

Response to Reviewers' Comments

We would like to thank the reviewers and meta-reviewer for their constructive comments. We have addressed all the comments in the revised paper. We highlight the major changes made on the manuscript in [blue](#).

Summary of Changes

- We have enhanced Section 4.1 and Section 4.4 to state that we are employing the cache line version scheme to prevent inconsistent reads in RedT-SI. In particular, we maintain multiple version numbers on each data item, and identify inconsistent reads by checking the version numbers on the obtained data. Further, we have clarified that RedT manages variable-length data using an overflow segment mechanism, where we utilize chained overflow segments to store data exceeding the capacity of fixed-size structures.
- We have improved Section 4.4 by adding Figure 8 to clarify the invocation timing of functions within the RDMA logical clock mechanism.
- We have revised Section 7.3 to clarify that we employ TLA+ to formally verify RedT's correctness regarding atomicity, durability, SI and SER isolation levels. We utilize the TLC model checker to perform 3 test runs for each specification and confirm that no correctness criteria are violated. All three TLA+ specifications and their corresponding execution logs are available in the GitHub repository [50].
- We have extensively enhanced Section 8 according to reviewers' comments. We have incorporated NCC and Calvin for performance comparisons with RedT-SER, as shown in Section 8.2. We have compared PostSI with RedT-SI in Section 8.3. We have added Section 8.4 to evaluate the impact of RDMA version array length and the RDMA logical clock mechanism on RedT-SI performance. We have provided additional experiments of read staleness in Section 8.6. We have compared RCBench-Cicada, an RDMA-based OCC algorithm, with RedT-SI in Section 8.7.
- We have enhanced Section 9 to incorporate the discussion of more related systems.
- For reference, we release our codes in <https://github.com/dbiir/RedT>, and provide a detailed technical report in <https://github.com/dbiir/RedT/blob/RedT/RedT.pdf>.

1 Meta Reviewer

The authors are asked to address the comments/concerns of the reviewers listed below. Many are related to clarifications and extensions of discussion. But the reviewers also ask for significant extensions to the evaluation section, including comparing with other systems. In particular, the following evaluation requests were given:

- * understand the cost of logical timestamp maintenance (R1)
- * provide information about abort rates (R1)
- * evaluate against a system that requires fewer network round trips (R1)
- * micro-experience examining the trade-off of using a larger version array (R2)
- * compare with more state-of-the art SER and SI protocols (R3)
- * compare with OCC-based RDMA protocols (R3)
- * analyze staleness/freshness in more detail, following eg. the solutions used in the OSDI'20 and SIGMOD'24 papers that R3 cited (R3)

Response: We have enhanced the clarity and completeness of our paper by: (1) adding Figure 8 and revising Section 4.4 to clarify the invocation time of functions in the RDMA-based logical clock mechanism (Response *R1:1*); (2) improving the explanation of the *latch* mechanism in Section 2.4, Section 4.1, and Section 4.2 (Response *R1:2*); (3) detailing the handling of variable-length data within our fixed-length structure in Section 4.1 (Response *R2:O2*); and (4) elaborating on the handling of read inconsistency in Section 4.1 and Section 4.4 (Response *R2:O1*). In addition, we have expanded the discussion in Section 6 to specify the replica failure threshold $\frac{1}{4}$ supported by RedT (Response *R1:3*), and incorporated descriptions of Spanner, Carousel, NOC-NOC, and NOCS into Section 9 (Response *R3:5*).

Further, we have added the following experiments accordingly:

1. We have evaluated the overhead of the RDMA-based logical clock mechanism. For further details, please refer to Response *R1:1*.
2. We have reported the abort rate in original Figure 8 (now Figure 11) as requested by Reviewer 1 (Response *R1:4*).
3. We have compared RedT-SER with Calvin, a system that requires fewer network round trips, in Figures 10-16 (Response *R1:5*).
4. We have added an experiment to evaluate the trade-offs in version array length in Figure 24(a) (Response *R2:O2*).
5. We have compared RedT-SER with NCC as requested by Reviewer 3 in Figures 10-16, and compared RedT-SI with PostSI, a more state-of-the-art algorithm satisfying the SI isolation level in Figures 17-23 (Response *R3:O3*).
6. We have compared RedT-SI with RCBench-Cicada, an RDMA-based OCC algorithm in an intra-DC environment, as shown in Figure 26(b) (Response *R3:O4*).
7. We have added an experiment to evaluate the read staleness resulting from read optimization in Figure 25(c) (Response *R3:O2*).

MR:R2. Furthermore, reviewers raised concerns regarding correctness. In particular, R1 mentions that a formal specification alone does not establish correctness. This could be addressed by using a model checker or theorem prover to at least show for some properties that they hold, and that these properties comply with the claims made in the paper. If the authors find it hard to specify and model-check the properties, they might want to consider remove some of their claims.

Response: We have revised Section 7.3 to clarify that we employ TLA+ to formally verify RedT’s correctness regarding atomicity, durability, SI and SER isolation levels. We utilize the TLC model checker to perform 3 test runs for each specification and confirm that no correctness criteria are violated. All three TLA+ specifications and their corresponding execution logs are available in the GitHub repository [50]. Please refer to Response *R3:1* for more details.

2 Reviewer 1

R1:1. It was interesting to read the solution for implementing RDMA-based logical clocks through the use of *GetTS*, *UpdateTS* and *UpdateRemoteTS*. The text on page 10 refers to these functions as being in Algorithm 4, but I believe they are actually described in Algorithm 3. The text-only description of when these functions are called in the different phases of transaction processing can be difficult to follow, which also makes it difficult to reason about the correctness of the protocol. Adding a time diagram to illustrate an example would help here. It would also be interesting to understand the cost of logical timestamp maintenance since that introduces the need for additional RDMA_CAS operations which have performance penalties relative to other RDMA operations.

Response: Following your suggestions, we have corrected the reference from Algorithm 4 to Algorithm 3, and added Figure 8 to clarify when *GetTS*, *UpdateTS*, and *UpdateRemoteTS* are invoked during transaction processing. We have also updated the statements in Section 4.4 accordingly. As shown in Figure 8, during the execution phase, the coordinator first acquires a snapshot timestamp $T_i.snapshot$ by invoking *GetTS* (①), and then dispatches the transaction to the corresponding data centers for execution. In each data center, upon receiving the transaction, the executor updates its local logical clock by calling *UpdateTS* (②) and then invokes *UpdateRemoteTS* to update the logical clocks on remote data nodes (③). In the commit phase, the coordinator collects the execution results returned by each data center, and updates its local logical clock using *UpdateTS* (④). Afterward, the coordinator determines if T_i can be committed. If more than $\frac{3}{4}$ of replicas per partition accessed by T_i return consistent data items and vote to commit, the coordinator makes the commit decision and retrieves the commit timestamp $T_i.cts$ using *GetTS* (⑤). Subsequently, it sends commit messages, along with $T_i.cts$, to all data centers to execute commit operations. Otherwise, the coordinator sends abort messages to all data centers. Upon receiving the commit/abort messages, the executors call *UpdateTS* (⑥) to update their local logical clock, and then utilize *UpdateRemoteTS* to update the remote logical clock while executing commit/abort operations on

remote data nodes (7). Finally, after all executors complete their commit/abort operations and send responses to the coordinator, the coordinator invokes *UpdateTS* to update its local logical clock.

We have conducted additional experiments to evaluate the overhead of the RDMA-based logical clock mechanism. Specifically, we compared two configurations: RedT-SI, which employs the RDMA-based logical clock mechanism, and RedT-SI (w/o LTS), a variant that replaces the RDMA-based logical clock service with local physical clocks. As shown in Figure 24(b), the performance difference between RedT-SI and RedT-SI (w/o LTS) is negligible. This is due to the fact that the primary performance bottleneck stems from the latency of inter-DC networks, rather than the additional RDMA one-sided verbs in the proposed logical clock mechanism.

R1:2. Algorithm 4 also introduces the idea of a latch, but the text does not explain what it is, what purpose it serves, and how it differs from a lock. Expanding on this would be helpful.

Response: We have enhanced Section 2.4, Section 4.1, and Section 4.2 to provide a clearer explanation of the *latch*. In our design, each data item maintains a *latch*, while each version maintains a *lock*. The *latch* protects concurrent access to the entire data item, whereas the *lock* manages concurrent access to each version. RedT-SER requires both the data item *latch* and the per-version *lock* to enable concurrency control at the version level. In contrast, RedT-SI performs concurrency control at the data item level, so that it only needs the *latch*. We would like to detail the use of *latch* and *lock* in RedT-SER and RedT-SI below: In RedT-SER, a transaction obtains the *latch* on the data item before acquiring a read or write lock on a specific version (line 7). This *latch* prevents concurrent lock acquisitions by other transactions, ensuring exclusive access to the data item’s metadata. Specifically, as detailed in Algorithm 1, after securing the *latch*, the transaction updates the version-specific metadata fields, i.e., the *lock* and *lockOwner* fields (lines 10-14), to obtain the desired lock on the version. In RedT-SI, as shown in Algorithm 4, a transaction acquires a data item’s *latch* before write operations (line 13), serving as an exclusive lock on the entire data item. Concurrent transactions can check whether the *latch* is already held by another transaction before performing a read operation (line 9), thus avoiding access to uncommitted data.

R1:3. Section 6 describes that, unlike Paxos/Raft, RedT can tolerate at most 1/4 replica failures. It is not clear to me why this stricter condition is required given that RedT only handles crash faults. Is this necessary to support failure recovery of coordinators? The reason needs to be included. In the “Failure recovery of coordinators” subsection, it mentions that if less than half of the replicas for each partition is still alive, then the status is aborted. Is that safe, or should the protocol instead stall until additional replicas become available before it makes a decision?

Response: RedT ensures that an inter-DC transaction can complete with only 2 network round trips, but in return, it employs a stricter requirement for fault tolerance, i.e., tolerating at most $\frac{1}{4}$ replica failures. Given that our primary focus is optimizing transaction performance in normal execution and that failures occur relatively infrequently, we consider this trade-off reasonable.

We would like to clarify that tolerating at most $\frac{1}{4}$ replica failures can enable RedT to effectively recover from the simultaneous failure of both the coordinator and replicas. In such a failure scenario, the coordinator loses its commit or abort decisions, and concurrent failures of the involving replicas may further result in the loss of commit/abort decisions for certain sub-transactions. This combined loss of information makes it infeasible to reconstruct the original decision during recovery. To address this challenge, both RedT-SER and RedT-SI require transactions to perform concurrency control on all replicas within a partition, and allow transactions to commit only if more than $\frac{3}{4}$ of the involved replicas vote to commit. This design ensures that, even in cases of simultaneous failures, a newly elected coordinator can reconstruct transaction states by collecting concurrency control outcomes from surviving replicas [73]. For example, let us consider a transaction *T* that has obtained commit votes from more than $\frac{3}{4}$ of replicas, and the coordinator has made the final commit decision accordingly. After that, the coordinator and fewer than $\frac{1}{4}$ of the replicas fail simultaneously. In this case, we trigger the recovery process: the new coordinator collects the state information from the remaining replicas, and it can still obtain more than $\frac{1}{2}$ commit votes ($\frac{3}{4} - \frac{1}{4} = \frac{1}{2}$), thereby enabling the coordinator to reconstruct the original commit decision. However, if the coordinator and more than $\frac{1}{4}$ of the replicas fail, the new coordinator needs to wait until a sufficient number (more than $\frac{3}{4}$) of replicas recover. To recover the failed replica, RedT follows the approach adopted in Paxos [31], which finds available data nodes and restores the failed replicas asynchronously.

We have revised Section 6 to include this discussion accordingly, and clearly state that if fewer than $\frac{3}{4}$ of the replicas for any partition are alive, RedT suspends the operations on this partition until enough (more than $\frac{3}{4}$) replicas recover.

R1:4. RedT was evaluated using two workloads: YCSB and TPC-C. However, it is not clear which workload was used to generate the different graphs in the evaluation section. I believe the VLDB paper did describe this in the graph captions, but that has been removed in this manuscript. This makes it difficult to compare results across graphs. Below are some questions I have about the evaluation results:

- The X-axis of Figure 8 shows the percentage of inter-DC transactions and stops at 1%. Should that be 100% instead of 1%? A similar graph in the VLDB paper goes from 0 to 100%.
- What is the reason for the significant throughput drop from increasing RTT in Figure 9? Is it due to higher abort rates? If it is, it would be helpful to show the abort rates. 100+ ms RTT is fairly common between datacenters. RedT's throughput improvements over the competing systems are fairly small at that point. Similarly, a skew factor of 0.5 or more is fairly common. RedT no longer has a performance advantage over the other systems at that point.
- Figure 24 shows that the read-only optimization is effective at significantly increasing the throughput when there are mostly read-only transactions. However, it is not clear why the results of the other systems in Figure 24 seem to be different from the results in Figure 8. In Figure 8, RedT-SER without read-only optimization outperforms TAPIR, yet the two systems perform about the same in Figure 24. Are there differences in the workload between those two figures?
- Figure 26 shows fairly minimal performance difference between RedT and RedT without RDMA when the network delay between data centers is only 30 ms. Is the complexity of using RDMA warranted given that most data centers will have a network delay higher than 30 ms in a geo-distributed deployment? It would be helpful to discuss this in the text.

Response: Following your suggestions, we have revised Section 8 to specify the workload used in the figure captions, and enhanced the explanation of experimental results. Below, we would like to address your concerns regarding the experimental results point by point:

1. We have corrected the X-axis range to 0% - 100% in Figure 10 (previously Figure 8), Figure 17, Figure 25(a), and Figure 25(b).
2. **Regarding the abort rate in Figure 11 (previously Figure 9)**, we have added Figure 11(d) to present the corresponding abort rates. The results indicate that the abort rates for all evaluated algorithms remain consistently low across varying network delays. This further confirms that the performance degradation observed in Figure 11 is not caused by changes in abort rates but is primarily due to the increased network delay.
3. **Regarding scenarios with RTT latency exceeding 100ms**, we have included Figure 11(b) to more clearly show the performance under high network latency. As observed, even under a 100ms network delay, RedT-SER still achieves 20% higher throughput than the next-best protocol. Similarly, Figure 18(b) illustrates that at 100ms latency, RedT-SI outperforms the next-best protocol by 50%.
4. **Regarding scenarios with skewness exceeding 0.5**, as depicted in Figure 14(a) and Figure 21(a), all algorithms except Calvin experience a significant decline when skewness exceeds 0.5. This can be attributed to the fact that under high contention scenarios, the concurrency control algorithm becomes the dominant factor affecting performance. Calvin employs deterministic concurrency control to reduce transaction aborts but relies on several preconditions, such as pre-knowledge of read/write sets and batch execution requirements. In contrast, RedT and other protocols do not impose additional constraints on transactions, leading to similarly high abort rates under high contention. Further, we would like to clarify that in Figure 14(a), in Figure 14(a), we maintain all other settings at their default values to isolate the impact of skewness on performance. However, this setup does not fully highlight the advantages of RedT's dedicated design for inter-DC networks. Therefore, we have conducted additional experiments under varying contention levels with 0-100ms inter-DC network jitter, another common setting in inter-DC deployments. The throughput and latency results are presented in Figure 1. As observed, RedT-SER outperforms most other algorithms (excluding Calvin) when the skew factor is below 0.8. Even under significantly high contention (skew factor is larger than 0.8), RedT's performance remains comparable to most other algorithms.
5. **Regarding the performance mismatch observed between Figure 10 and Figure 25 (previously Figure 24 and Figure 8)**, their X-axes are different while other settings remain by default. We would like to state that, at the point where the inter-DC transaction ratio is 100% and the write transaction ratio is 80%, the performance of all algorithms is consistent in both figures. Moreover, read-only optimization is enabled by default for both RedT-SER and RedT-SI, with the exceptions of RedT-SER-Base and RedT-SI-Base in Figure

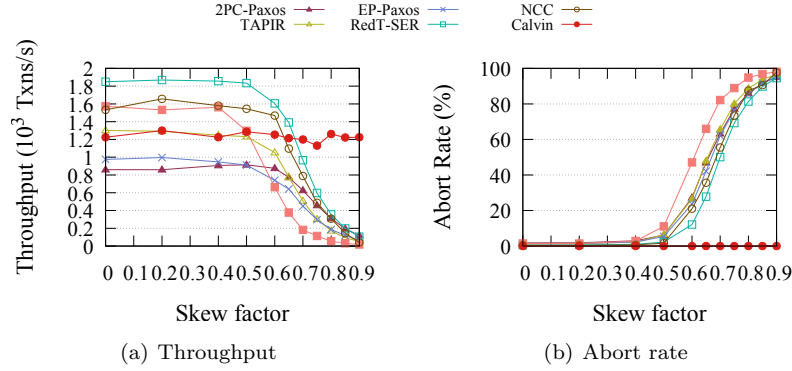


Fig. 1 Impact of contention level under jitter network - SER

25. Consequently, at the 100% cross-datacenter ratio and 80% write transaction ratio point, the performance of RedT-SER in Figure 10 aligns with that of RedT-SER in Figure 25. The performance of RedT-SER in Figure 10 differs from that of RedT-SER-Base in Figure 25. We have clarified these settings in Section 8.1.

6. **Regarding the benefit of RDMA**, while the performance benefits of RDMA diminish when network latency exceeds 30ms, it significantly enhances transaction performance in low-latency scenarios (less than 30ms), as shown in Figure 26(a). We would like to clarify that deploying inter-DC databases with sub-30ms network latency is common in practice. For example, PingCap adopts a deployment model with two data centers in Beijing and one data center in Xi'an [25], where the inter-DC network latency is approximately 20ms. In such typical deployment scenarios, utilizing RDMA can still achieve up to 80% performance gains. Furthermore, in a geo-distributed deployment where inter-DC network latency exceeds 30ms, most transactions can still be processed within a region, as countrywide distributed transactions are relatively infrequent.

R1:5. The related work section describes a number of systems that require fewer network round trips with the assumption that the read/write sets are known ahead of time. Given that this assumption holds for YCSB and may even hold for TPC-C if reconnaissance queries are used, it would be interesting to evaluate against one of these systems.

Response: We have included an additional baseline system, Calvin [57], which employs deterministic concurrency control to minimize network round trips, assuming transaction read/write sets are known in advance. We evaluate Calvin across various settings and report its performance in Figures 10–16. As observed, RedT-SER generally outperforms Calvin. This is due to the fact that, although Calvin can theoretically commit a transaction within a single network round trip, it mandates processing transactions in a predetermined execution sequence at each node. Therefore, Calvin requires a centralized scheduler responsible for determining the execution sequence, which can potentially become a single contention bottleneck. In contrast, RedT does not have this limitation and includes additional optimizations (e.g., pre-write-log mechanism, read-only optimization, etc.) for inter-DC transactions, thus achieving superior overall performance. However, as shown in Figure 14, Calvin achieves higher throughput than RedT in high-contention scenarios. This is primarily because Calvin avoids transaction aborts caused by conflicts in such scenarios, thereby maintaining stable throughput. Nevertheless, Calvin's advantage comes at the expense of several assumptions, including the requirement of prior knowledge of transaction operations, batch execution, and limitations in handling long transactions, among others. RedT does not impose these constraints.

R1:6. The related work section should also include a citation to Spanner, since it is a well known geo-distributed transaction processing system, and Carousel, which reduces the number of wide-area network round trips required for a Spanner-like system.

Response: We have improved Section 9 to discuss the difference between Spanner, Carousel, and RedT. Spanner [4, 13, 14], widely recognized as the first geo-distributed database, employs the 2PC-Paxos protocol (as detailed in Section 2.1) to manage distributed transactions. Therefore, Spanner does not explicitly optimize transaction performance under the inter-DC network, resulting in 6 inter-DC round trips per distributed transaction. Building

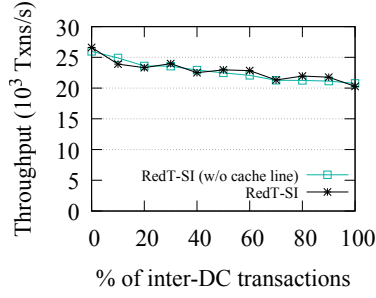


Fig. 2 Impact of inter-DC transaction ratio (RedT-SI vs RedT-SI without cache line version)

upon Spanner’s architecture, Carousel [66] merges the execution phase and prepare phase, thus reducing the inter-DC round trips from 6 to 4. Nevertheless, RedT introduces a novel pre-write-log mechanism, enabling distributed transactions to complete in 2 inter-DC round trips while minimizing communication overhead per round.

R1:7. There were also a number of (mostly minor) grammatical issues throughout the manuscript. The manuscript would benefit from a copy-editing pass with a focus on improving readability.

Response: Following your suggestion, we have thoroughly proofread the paper and corrected all grammar issues we spotted.

3 Reviewer 2

R2:O1. Regarding Algorithm 4, as I understand it, the read operation uses a single RDMA read to retrieve the entire version buffer. This approach could risk reading inconsistent or corrupted data, as RDMA reads do not guarantee sequential address order. For example, suppose a cache line in the middle of the version buffer is read first while it is being modified, followed by the writer unlocking and updating the version. In that case, the reader may receive an inconsistent state when it reads the remaining cache lines. This potential issue should be further clarified, since correctness is important. For more details on the issue, you can refer to this paper “Design Guidelines for Correct, Efficient, and Scalable Synchronization using One-Sided RDMA”

Response: Thank you for pointing out this read inconsistency issue. We opt to employ the cache line version scheme [78,16] to address this problem in RedT-SI. In particular, we store a version number, denoted as v , at the start and end of each data item, as well as in the header of each version, as shown in Figure 5. During transaction commit, when RedT-SI modifies a data item, it updates all version numbers v within that data item to $v + 1$. Subsequently, an RDMA_WRITE operation is issued to propagate these version number updates to the remote data item. As a result, when RedT-SI issues an RDMA_READ during a read operation, it checks if the version numbers (v) at the start, end of the data item, and the head of each version within the data item are identical. If any mismatch is detected, it indicates that another write operation is concurrently modifying the data item, leading to an inconsistent read. In this case, RedT-SI reissues RDMA_READ to retrieve a consistent data item.

We have evaluated the performance impact of introducing the cache line versioning scheme. We compare the performance of RedT-SI, and RedT-SI without the cache line version scheme, namely RedT-SI (w/o cache line), and plot the transaction throughput in Figure 3. The results indicate that the cache line version scheme has a minimal effect on performance. This is explainable because, in inter-DC deployments, the primary bottleneck is network latency rather than the additional consistency checking overhead incurred by the cache line version scheme. Further, we have analyzed the storage overhead of the cache line version scheme. Each version within a data item requires an additional 8 bytes for versioning. Given that typical version sizes range from 64 to 1000 bytes, this incurs a limited space overhead of only 12% to 0.8% (computed as $8/64$ to $8/1000$).

To reflect this mechanism, we have revised Algorithm 4, Section 4.1, and Section 4.4 accordingly. Additionally, we have rerun all experiments for RedT-SI and updated the corresponding figures.

R2:O2. One challenge I often see under-discussed in RDMA-focused papers is handling variable-sized tuples, such as strings, which play a critical role in practical applications. The paper would benefit from discussing how variable-sized data types could be managed within the proposed fixed-sized data structures.

Response: We have revised Section 4.1 to detail that we employ fixed-size *overflow segment* to manage variable-length data under heterogeneous networks, inspired by the out-of-table model used in herd [26]. As depicted in Figure 5, each data version x_v can be organized into a fixed-sized *data* structure and multiple overflow segments, allowing efficient storage of variable-sized data within fixed-size data structures. The *data* structure includes: (1) a data item type (*type*), indicating whether x_v is variable-length; (2) a segment count field (*count*), specifying the number of overflow segments x_v needs; (3) an address pointer (*addr*), referencing the initial overflow segment containing the remaining data content; (4) a field storing either a portion of the variable-length data or the complete data if it fits entirely within this field. As shown in Figure 5, the overflow segments containing the remaining data content are stored continuously. Each overflow segment has the following layout: The first and last 8 bytes store the version number v , which indicates whether the overflow segments are consistent with the entire data item, as discussed in Response R2:O1. The remaining portion of the segment contains the actual data value. Upon retrieving variable-length data, we first obtain the count of overflow segments (*count*) and the memory address (*addr*) of the first overflow segment. Subsequently, issue an RDMA_READ to retrieve all necessary overflow segments to reconstruct the complete variable-length data.

R2:O3. Along the same line, I suggest including a micro-experiment that examines the trade-off of using a larger version array. My expectation is that a larger version array may reduce transaction aborts but at the cost of potentially more expensive reads due to the increased size. Seeing this trade-off could improve the understanding of the protocol’s performance. Or at least a small discussion on the effect would be interesting.

Response: Following your suggestion, we have conducted additional experiments to investigate the impact of version array length. We adopt the default setting, with the version array length increasing from 1 to 32. As shown in Figure 24(a), the performance remains stable when the version array length increases. We attribute this to the fact that the version array length does not dominate performance in inter-DC transaction processing. In low-contention (skew factor = 0.2) scenarios, the primary bottleneck is network latency introduced by inter-DC deployments, while in high-contention scenarios (skew factor = 0.7), the primary bottleneck is transaction aborts caused by write-write conflicts, which is orthogonal to the version array length. We acknowledge that in cases where long-running read transactions access a large number of data items, an increased version array length may lead to higher abort rates, potentially impacting performance. Since RedT-SI is primarily designed and evaluated for OLTP workloads with short transactions, we opt to leave the investigation of long-running transactions as part of our future work.

4 Reviewer 3

R3:1. The claims regarding correctness guarantees appear overstated. While the TLA+ specification is a commendable effort, formal specification alone does not establish correctness guarantees.

- Did the authors use TLC (the model checker) or the TLA+’s theorem prover to analyze the properties? I did not find evidence of formal verification effort in the paper or source code.

- This concern extends beyond the incremental SI contribution to SER, atomicity, and durability guarantees as well.

Response: We have revised Section 7.3 to specify that we employ TLA+ to formally verify RedT’s correctness regarding atomicity, durability, SI and SER isolation levels. In particular, we provide three TLA+ specifications: `redtatom.tla`, verifying atomicity and durability; `RedTSER.tla`, verifying that RedT-SER correctly ensures SER; and `RedTSI.tla`, verifying that RedT-SI satisfies SI. We utilize the TLC model checker [65] to perform 3 test runs for each specification and confirm that no correctness criteria are violated. All three TLA+ specifications and their corresponding execution logs are available in the GitHub repository [50]. Due to space constraints, we provide the following detailed explanation of these three specifications in the README of our GitHub repository [50]. 1) In `redtatom.tla`, we simulate the simultaneous failure of the coordinator and fewer than $\frac{1}{4}$ of the replicas. Subsequently, following the proof in Section 7.2, we verify the *atomicity* invariant, which requires that after

the transaction finishes, more than $\frac{3}{4}$ of the replicas must reach a consistent state with the coordinator, either committed or aborted. Then, we verify the *durability* invariant, which mandates that throughout system execution, more than $\frac{3}{4}$ of the replicas for each data item must maintain consistent data. 2) In `RedTSI.tla`, we primarily examine two criteria to check whether transaction executions satisfy the requirements of SI: *snapshot read* criteria validates that after a read transaction accesses a data item, no version written with a timestamp preceding the read transaction’s snapshot time exists. Fulfillment of this condition, i.e., the absence of versions written with earlier timestamps, indicates that the read transaction satisfies the **SI1** condition as defined in Section 7.1.2. *snapshot isolation write* criteria verifies the absence of overlapping execution intervals between any two write transactions accessing the same data item. The absence of such overlaps indicates that these write transactions satisfy the **SI2** condition as defined in Section 7.1.2. 3) In `RedTSER.tla`, we examine the serializability criteria, which verifies whether cycles exist in the dependency graph constructed from concurrently running transactions.

R3:2. The idea of trading data freshness for isolation guarantees is not novel, though its application in optimizing read-only transactions is valid. However,

- without concrete measurements, it is difficult to assess the practical effectiveness of this optimization.
- while stale snapshots meet isolation guarantees, they may lack practical relevance. Empirical evaluation on data freshness is needed to justify the optimization’s utility.

Response: We have revised Section 5 to clarify that we use a threshold *max_tolerant_staleness*, to enhance the practical effectiveness of read-only optimization. To prevent read-only transactions from accessing excessively stale data, we enforce a threshold, *max_tolerant_staleness*, which limits the oldest accessible version for read-only transactions. If the timestamp difference between the readable version and the latest version exceeds *max_tolerant_staleness*, the read-only transaction aborts and re-executes using the non-optimized read-only mode. When *max_tolerant_staleness* = 0, read-only transactions must always read the most recent data, effectively disabling the read-only optimization. When *max_tolerant_staleness* = *infinity*, read-only transactions are required to always access the latest version. By incorporating *max_tolerant_staleness*, our read-only optimization can be applied to scenarios with less stringent data freshness requirements, such as social media applications (e.g., displaying likes and comments).

Regarding the efficiency of our read-only optimization, as shown in Figures 25(a) and Figure 25(b), RedT-SER and RedT-SI achieve throughput improvements of up to $2.34\times$ and $5\times$, respectively, when read-only optimization is enabled. This improvement stems from read-only transactions accessing local versions, thereby eliminating inter-DC round-trips.

We have also conducted additional experiments to evaluate the staleness of read-only transactions in RedT, with *max_tolerant_staleness* = *infinity*. As shown in Figure 25(c), with increasing skewness, increasing skewness leads to older versions being read by read-only transactions. However, in 90% of cases, the staleness remains within 150ms. This relatively low staleness is primarily due to our system being built on an in-memory database, where log replay is highly efficient, enabling read-only transactions to be able to access relatively up-to-date data.

R3:3. There are issues with the baselines used in the evaluation:

- TAPIR, though a novel protocol at its time, is nearly a decade old. More recent protocols, such as "NCC: Natural Concurrency Control for Strictly Serializable Datastores by Avoiding the Timestamp-Inversion Pitfall" (OSDI’23), would be included for comparison.
- Comparing RedT-SI (an SI protocol) to TAPIR (an SER protocol) is inherently unfair, as TAPIR provides stronger guarantees. To make performance improvements convincing, the authors should compare with state-of-the-art SI protocols.

Response: Following your advice, we have included two additional baselines in our evaluation, NCC [36], a state-of-the-art SER protocol, and PostSI [76], a state-of-the-art SI protocol. We have conducted additional experiments to compare RedT-SER with NCC [36] and RedT-SI with PostSI across various settings, and plot the results in Figures 11-16. As observed, RedT-SER demonstrates superior performance compared to NCC, primarily due to its lower network round trips for inter-DC transactions. NCC is designed mainly for intra-DC environments, where an inter-DC read-write transaction requires 4 rounds of inter-DC network communication. Further, as shown in Figures 18-23, RedT-SI outperforms PostSI. This is due to the fact that PostSI introduces an additional network round trip in 2PC to dynamically determine transaction order, which reduces abort rates but negatively impacts performance for inter-DC transactions.

R3:4. The evaluation includes a comparison with DSLR, a recent RDMA-based protocol, but there are two issues.

- For readers unfamiliar with DSLR, the isolation guarantee it provides should be explicitly stated to ensure the fairness of the comparison is clear.
- The exclusion of OCC-based RDMA protocols for "fair comparison" is unconvincing to me. Including such protocols would actually strengthen the claim of RedT-SI's competitiveness.

Response: We have revised Section 8.7 to clarify that DSLR [68] is a state-of-the-art lock-based RDMA protocol that achieves the serializable isolation level.

Following your suggestion, we have included RDMA-Cicada, an RDMA-based Optimistic Concurrency Control (OCC) protocol [75], in our evaluation. To ensure a fair comparison with RedT-SI, we set RDMA-Cicada to SI isolation level. As shown in Figure 26(b), RedT-SI outperforms RDMA-Cicada. This is expected because, compared to RedT-SI, RDMA-Cicada employs more complex logic and requires additional RDMA one-sided operations, leading to lower throughput compared to RedT-SI.

We would like to mention that since DSLR and RDMA-Cicada lack fault tolerance support, we disabled the fault tolerance mechanisms in RedT-SER and RedT-SI to facilitate a fair performance comparison with DSLR and RDMA-Cicada. Given that neither RDMA-Cicada nor DSLR is designed for inter-DC deployments, we conducted this evaluation in an intra-DC RDMA environment. We have revised Section 8.7 to clarify these settings accordingly.

R3:5. Improving database concurrency control is a critical research area. It would be better for the authors to contextualize their protocols in the landscape of both theoretical and practical efforts. For instance, it would be better to analyze the protocols through the lens of frameworks like NOCS (Performance-Optimal Read-Only Transactions, OSDI'23) and NOC-NOC (Towards Performance-Optimal Distributed Transactions, SIGMOD'24). Including such discussions in, e.g., the Related Work section, would enrich the paper and position the contribution more effectively.

Response: We have improved Section 9 to discuss the difference between NOCS, NOC-NOC, and RedT. Specifically, NOCS [37] introduces a version-clock mechanism to track the serializability order of read and write operations, which improves data freshness for read-only transactions. Building upon this, NOC-NOC proposes a dual views mechanism that processes read and write operations in a lock-free manner, further improving transaction performance. However, they are orthogonal to RedT, as they primarily focus on intra-DC transaction optimizations and do not explicitly address inter-DC transaction performance. While both NOCS and NOC-NOC require a similar number of network round trips as traditional 2PC for inter-DC transactions, RedT enables a transaction to commit in 2 network round trips.

An Efficient Two-Round Distributed Transaction Processing Approach over Heterogeneous Networks

Hongyao Zhao · Wei Lu · Zhanhao Zhao · Yinhao Hong · Quanqing Xu · Jinliang Xiao · Fusheng Han · Chuanhui Yang · Xiaoyong Du

Received: date / Accepted: date

Abstract Countrywide and worldwide business, like gaming and social networks, drives the popularity of inter-data-center transactions. To support inter-data-center transaction processing and data center fault tolerance simultaneously, existing protocols suffer from significant performance degradation due to high-latency and unstable networks. In this paper, we propose RedT, a novel distributed transaction processing protocol that works in heterogeneous networks. In detail, nodes within a data center are inter-connected via the RDMA-capable network, and nodes across data centers are inter-connected via TCP/IP networks. RedT extends two-phase commit (2PC) by decomposing transactions into sub-transactions in terms of the data center granularity, and proposing a pre-write-log mechanism that is able to reduce the number of inter-data-center round-trips from a maximal of 6 to 2. RedT supports both serializability and snapshot isolation. Extensive evaluation against state-of-the-art protocols shows that RedT can achieve up to $1.80\times$ higher throughputs and $0.41\times$ lower latency.

Keywords heterogeneous networks · distributed transaction · concurrency control · RDMA

Hongyao Zhao, Wei Lu, Zhanhao Zhao, Yinhao Hong and Xiaoyong Du
the Key Laboratory of Data Engineering and Knowledge Engineering, Ministry of Education, China, and School of Information, Renmin University of China, China.
E-mail: {hongyaozhao, lu-wei, zhanhaozhao, hongyh, duyong}@ruc.edu.cn

Quanqing Xu, Jinliang Xiao, Fusheng Han and Chuanhui Yang
OceanBase, Ant Group, China.
E-mail: {xuquanqing.xqq, xiaoshi.xjl, yanran.hfs, rizhao.ych}@oceanbase.com

1 Introduction

Modern distributed database systems achieve scalability and availability by partitioning and replicating the data over multiple nodes. Each partition has one *primary replica* and multiple *secondary replicas*. The consensus among replicas is guaranteed by Paxos [31] or Raft [43]. The standard approach to processing a distributed transaction T is called *2PC-Paxos* which executes T through three phases: 1) execution phase, 2) prepare phase, and 3) commit phase. In the execution phase, the coordinator decomposes T into sub-transactions that are distributed to the corresponding primary replicas (a.k.a. participants). Once each primary replica completes local execution, the coordinator follows 2PC to commit/abort the sub-transactions. In the prepare phase, first, the coordinator notifies each primary replica to check whether the sub-transaction is ready to commit; then, every primary replica synchronizes writes in terms of redo logs to the secondary replicas if the sub-transaction is ready to commit, and sends a response to the coordinator after the synchronization; subsequently, after receiving responses from all primary replicas, the coordinator decides whether T commits or aborts. Before entering the commit phase, the coordinator synchronizes its decision to coordinator backups to ensure the availability of the decision in case of failure. In the commit phase, the coordinator first notifies all primary replicas to commit/abort the sub-transactions, and each primary replica then synchronizes the decision to its secondary replicas. *2PC-Paxos* takes a maximum of 6 network round-trips, with 1 in the execution phase, 3 in the prepare phase, and 2 in the commit phase. Note that some network round-trips can be neglected for read-only transactions.

There is an increasing trend to support inter-data-center (abbreviated as inter-DC) transaction processing and data center fault tolerance simultaneously. On the one hand, the wisdom to facilitate fast data delivery is geo-locality, i.e., storing clients' data in the nearby data center. Yet, the countrywide business, e.g., e-commerce and gaming, requires to co-operate clients' data across data centers, and drives the popularity of inter-DC transactions, each of which reads/writes primary replicas located in multiple data centers. On the other hand, to achieve system availability that tolerates data center failures, replicas of the same partition are deployed across data centers. *Nevertheless, distributed transaction processing suffers from severe performance degradation due to high-latency and unstable networks.* Compared with the intra-data-center (abbreviated as intra-DC) network that typically takes $70\mu s \lesssim RTT \lesssim 1ms$, the inter-DC network has much higher latency, and is less stable. Figure 1(a) shows that the average round-trip time (abbreviated as RTT, a latency metric) from Beijing to Hong Kong ranges from a minimum of 40ms to a maximum of 183ms, and 13 packets are lost. We implement *2PC-Paxos* on the distributed framework Deneva [24], in which each partition is set to have one primary replica and two secondary replicas. Replicas of the same partition are placed in different data centers. Figure 1(b) plots the system throughput over YCSB benchmark by varying the percentage of inter-DC transactions. The throughput decreases by a factor of 33%, 44%, and 45%, under the inter-DC latency of 20ms, 40ms, and 80ms, respectively.

To efficiently support inter-DC transaction processing and data center fault tolerance simultaneously, a few works attempt to reduce either the number or the overhead of network round-trips. TAPIR [73] and G-PAC [40] unite 2PC and consensus protocols in a single framework to eliminate redundant coordination, reducing the number of inter-DC round-trips from 6 to 3 in common cases. MDCC [29] incorporates Fast Paxos [32] protocol so that the number of round-trips is reduced from 6 to 2 under some prerequisites, e.g, in the absence of conflicts, while two additional round-trips are required otherwise. Multi-level 2PC [42] reduces the number of expensive inter-DC communications by layering participants, but cannot reduce the number of inter-DC round-trips. Because the inter-DC network could be a bottleneck, we aim to improve the system performance by further reducing either the number or the overhead of network round-trips.

Our research is motivated by the fact that in both academia and industry, the widespread use of RDMA-capable networks has become a reality. In academia, numerous works [70, 68, 6, 63, 11, 62, 16, 17] show that the

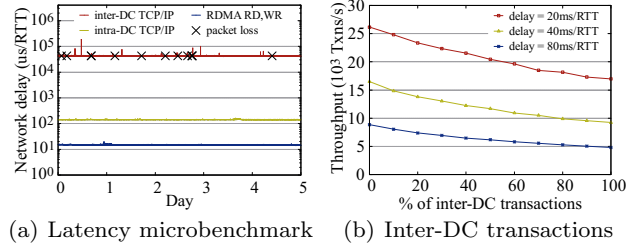


Fig. 1 Effect of the high-latency networks

performance of intra-DC distributed transaction processing can be improved by orders of magnitude over RDMA-capable networks. In industry, enterprises like Microsoft [77, 23], IBM [5], and Alibaba [22, 10] have reported boosting their database systems using RDMA-capable networks. Unfortunately, RDMA is built on lossless networks, and is currently considered only suitable for use in the local area networks (a.k.a. LAN) [77]. We realize heterogeneous networks are increasingly popular, where nodes within the same data center are inter-connected via RDMA networks, and nodes across multiple data centers are inter-connected via TCP/IP networks. Hence, redesigning distributed transaction processing in heterogeneous networks is of great necessity.

In this paper, we propose RedT, an RDMA-enhanced distributed transaction processing protocol with availability guarantees. Compared with 2PC-Paxos, RedT reduces the number of inter-DC round-trips from a maximum of 6 to 2. Specifically, we design a pre-write-log mechanism. In this mechanism, during the execution phase, every primary/secondary replica is required to perform concurrency control, persist the redo logs, and send a commit/abort message to the coordinator. After collecting messages from at least $\frac{3}{4}$ replicas of each partition, the coordinator makes a commit/abort decision of the transaction, which is then propagated to every replica. Upon receiving the decision, every primary/secondary replica persists the commit/abort log, writes the data items if the transaction is committed, and releases the locks. RedT eliminates the synchronization of prepare messages from the coordinator to primary replicas, redo/commit/abort logs from the primary replicas to secondary replicas in the prepare/commit phase, and the transaction decision among coordinator backups. Hence, RedT reduces up to 4 inter-DC round-trips.

We introduce two RedT variants, RedT-SER and RedT-SI, to support two widely used isolation levels: Serializability (SER) and Snapshot Isolation (SI) [7]. Considering the network in which nodes within the same data center are inter-connected via RDMA networks, RedT selects one executor in each data center responsible for concurrency control over all replicas of the same data center. RedT achieves this by directly lock-

ing/unlocking/reading/writing replicas in other nodes using RDMA verbs. From this perspective, RedT extends sub-transactions from the replica granularity to the data center granularity, leading to a lower coordination cost and fewer inter-DC communications. Further, we conduct a quantitative analysis of RedT and existing protocols in terms of the inter-DC round-trips and communication costs when processing different kinds of transactions, which show the advantages of RedT. We introduce an optimization for read-only transactions, which allows them to commit without costly inter-DC communications, and propose a fault tolerance mechanism to handle node and data center failures. Note that if we do not need to tolerate data center failures, for intra-DC transactions, RedT is reduced to a complete RDMA-boosted method. We also formulate a theoretical proof and the TLA+ specifications [30] to ensure the correctness of RedT.

In summary, we make the following contributions:

- We present a novel distributed transaction processing protocol RedT that works in heterogeneous networks. It achieves high throughput and data center fault tolerance.
- We propose a pre-write-log mechanism that reduces the number of inter-DC round-trips from a maximum of 6 to 2. Specifically, we employ the RDMA-capable network to relax the granularity of sub-transactions from the replica to the data center, thus resulting in fewer inter-DC communications.
- We implement two RedT variants, namely RedT-SER and RedT-SI, to support serializability and snapshot isolation levels, respectively.
- We optimize read-only transactions in RedT to commit without inter-DC round-trips.
- We conduct extensive evaluations to compare RedT and four state-of-the-art protocols over YCSB and TPC-C benchmarks. The results show that RedT can achieve up to $1.80\times$ higher throughputs and $0.41\times$ lower latency.

This paper extends our previous work [74] by introducing new content and contributions as follows. (1) Considering SI is a prevalent database isolation level, we further enhance RedT to support SI. This enhancement includes the introduction of multi-version concurrency control (MVCC) (Section 2.4), the development of an RDMA-based logical clock maintenance mechanism for allocating transaction snapshots (Section 4.4), and analysis of inter-DC round-trips and communication costs for RedT and other protocols under the snapshot isolation level (Section 4.5). (2) To enhance the performance of read-only transactions, we propose a read-only optimization mechanism, enabling the avoidance of inter-DC round-trips by reading old data from

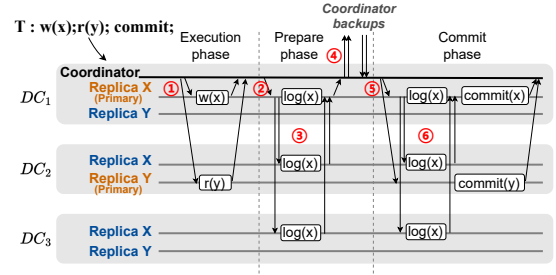


Fig. 2 Modern distributed transaction processing

the current data center (Section 5). (3) We demonstrated theoretical proof and the TLA+ specifications for both serializability and snapshot isolation levels to guarantee the correctness of RedT (Sections 7).

2 Preliminaries

2.1 Distributed Transaction Processing

As introduced in Section 1, the standard 2PC-Paxos takes a maximum of 6 network round-trips to process a distributed transaction. Take transaction T that consists of a write operation $w(x)$ and a read operation $r(y)$ in Figure 2 for an example. The primary replica of x/y is located in DC_1/DC_2 , respectively. x/y has one secondary replica in DC_2/DC_1 , and another secondary replica in DC_3/DC_3 , respectively. T first takes one round-trip to access the primary replica of x and y in DC_1 and DC_2 (labeled as ① in Figure 2). After successful execution, T notifies the primary replica of x to commit (②), synchronizes redo logs to x 's secondary replicas (③), and responds to the coordinator afterward. After collecting all responses and before entering the commit phase, T synchronizes its decision to the coordinator backups for fault tolerance (④). In the commit phase, T notifies the primary replica of x and y to commit (⑤), and x synchronizes the decision to its secondary replicas (⑥).

2.2 RDMA

RDMA is a network feature that enables remote direct access to the main memory of a remote data node without the involvement of its CPU. Compared with traditional TCP/IP networks, RDMA can be built on InfiniBand to enjoy the advantages of high-bandwidth and low-latency. It also provides three properties: (1) **Zero-copy** property. Applications can perform data transfers without the involvement of the network software stack. Data is sent and received directly to the buffers without being copied between the network layers. (2) **Ker-**

nel bypass property. Applications can perform data transfers directly from user space without kernel involvement. (3) **No CPU involvement** property. Applications can access remote memory without consuming any CPU time in the remote machine. RDMA provides two categories of verbs for programming: (1) one-sided RDMA verbs, including READ, WRITE, WRITE With Immediate, and two atomic operations: FETCH-And-ADD (a.k.a. FAA) and COMPARE-And-SWAP (a.k.a. CAS), and (2) two-sided RDMA verbs, including SEND and RECEIVE. Programming using two-sided RDMA verbs can enjoy zero-copy and kernel-passing properties, and one-sided RDMA verbs can enjoy all three properties of RDMA. Note that, CAS and FAA are confined to operating data of 64-bit size per invocation. As a result, using a single CAS/FAA to operate data items and metadata together may exceed the limitation of 64-bit size.

2.3 RDMA-based Transaction Processing

RDMA-based transaction processing is based on the shared-memory architecture [70, 17, 63, 68]. As opposed to decomposing read/write operations of a transaction into sub-transactions in distributed databases, this approach follows the paradigm in the centralized databases to process transactions without the involvement of 2PC. In RDMA-based transaction processing, each node is equipped with multiple *executors*, responsible for executing transactions, and a *memstore*, a shared-buffer used to manage data items and redo logs for fault tolerance. Nodes are inter-connected via low-latency and high-bandwidth featured RDMA networks. To process a distributed transaction, each executor works like its counterpart in the centralized system and directly locks/unlocks/reads/writes data items stored on the remote nodes. Once all the read/write operations are complete, the executor writes the redo logs into the remote memstore if necessary. The executor finally commits the transaction. Compared with traditional distributed transaction processing under TCP/IP networks, the performance of RDMA-based distributed transaction processing can be improved by a factor of at least $20\times$ [17, 61].

2.4 Design Choices of RedT

We aim to ensure that RedT applies to most practical database systems, such as PostgreSQL, MySQL, etc. These databases typically employ a multi-version concurrency control mechanism (MVCC), where each data item has multiple versions. Based on MVCC, these databases support multiple isolation levels, including

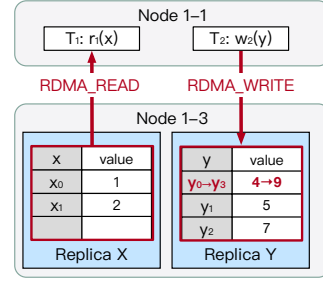


Fig. 3 The RDMA-friendly MVCC mechanism

serializability, snapshot isolation, etc. Serializability requires that the outcome of a transaction schedule matches that of a serial schedule. For snapshot isolation, a transaction T must consistently access a snapshot that reflects the state of data items at T 's (logical) start time, and T can only commit if no committed transaction has changed the data items that T intends to modify.

Given these observations, we first adopt the mechanism proposed in RCBench [75] to enable RDMA-friendly MVCC. In particular, each data item maintains a fixed-length version array. Transactions can then access the appropriate version through a single RDMA one-sided verb. As shown in Figure 3, when reading a version for a data item, the executor initiates an RDMA_READ to fetch the entire fixed-length version array and locates the appropriate version within it. When updating a data item, the new version is placed in an available slot within the version array. If the version array is full, the oldest version will be replaced. Based on this RDMA-friendly MVCC, we then design RedT to support both serializability and snapshot isolation, which will be further discussed in Section 4.

3 System Architecture

RedT is particularly designed for heterogeneous networks. An overview of the system architecture is shown in Figure 4. Each node is equipped with the following two kinds of components.

Multiple Txn Workers. A Txn Worker is able to act as either an **executor** or a **coordinator**. An executor executes sub-transactions by directly locking/unlocking/reading/writing data items that are located in other nodes of the same data center using RDMA verbs. A **coordinator** coordinates the execution of transactions.

MemStore. A MemStore is a pre-allocated RDMA-registered memory for maintaining replicas of different partitions. Besides, it maintains a circular buffer for logging. Any executors from other nodes in the same data center can directly write logs to this buffer via RDMA

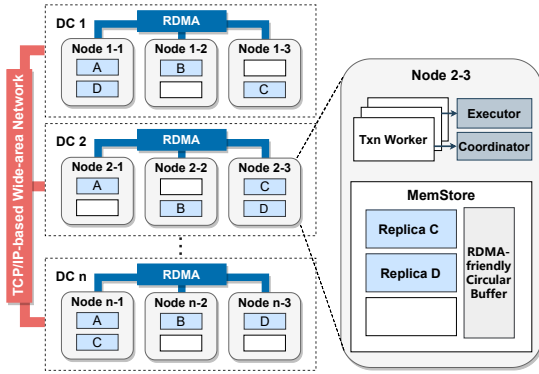


Fig. 4 An overview of the system architecture

verbs. For performance, we assume MemStore is deployed on Non-Volatile Memory (NVM) by default, as many other works do [72, 16, 28, 70]. Otherwise, logs are persisted in the disk before committing for durability.

In RedT, a distributed transaction T_i is executed through two phases: 1) execution phase, and 2) commit phase. In the execution phase, we propose a *pre-write-log* mechanism, in which each primary/secondary replica involves performing concurrency control and writing the redo logs. Similar to 2PC-Paxos, the coordinator decomposes T_i into multiple sub-transactions in terms of the data center granularity. To simplify the discussion, we assume that a partition in each data center has at most one replica (later we will discuss how to handle the case without this assumption). A sub-transaction consists of the reads/writes from/to replicas that locate in the same data center. For example, in Figure 2, transaction T is decomposed into three sub-transactions, each of which consists of two operations $w(x)r(y)$ and locates in every data center. The *pre-write-log* mechanism eliminates the prepare phase that synchronizes the prepare messages from the coordinator to primary replicas, redo logs from the primary replicas to secondary replicas, and the transaction decision among coordinator backups.

In each data center, one executor is selected to execute the sub-transaction, and follows the paradigm in the centralized databases to do concurrency control. We ensure RedT can support both serializability and snapshot isolation levels, which will be elaborated on in Section 4. To be specific, the executor directly reads/writes data items that locate in other nodes of the same data center using RDMA verbs. Any conflicts on the same data items would cause the sub-transaction to abort. If the sub-transaction is ready to commit, the executor first writes the redo logs and then sends a commit message to the coordinator; otherwise, if the sub-transaction is ready to abort, the executor sends an abort message to the coordinator. In the commit

phase, the coordinator collects the messages from each executor. Following the previous work [32], if more than $\frac{3}{4}$ of replicas per partition accessed by T_i have the consistent data items and vote to commit, the coordinator makes the commit decision, generates a commit timestamp, and notifies each executor to commit. Otherwise, the coordinator notifies each executor to abort. By doing this, RedT tolerates replica failures of no more than $\frac{1}{4}$ for any partition. We shall discuss the correctness of the number $\frac{3}{4}$ in Section 6.1 and Section 7. Note, for the sub-transaction votes to abort but the transaction decides to commit, the coordinator notifies its executor to re-write and commit the log. After receiving the message from the coordinator, each executor writes the data items if the coordinator decides to commit, commits/aborts the log, and releases the locks. We shall discuss the correctness in Section 4.3.

Discussion. We first consider the scenario where multiple replicas of each partition are deployed within a single data center, focusing on configurations that do not tolerate data center failures. Let n be the number of replicas per partition in this data center. We logically split the replicas that the transaction involves, into n groups, each of which takes one disjoint replica per partition. For each group, a designated executor is selected to execute the corresponding sub-transaction, following the previously described procedure.

4 Transaction processing

In this section, we first introduce the key data structures for RedT transaction processing. We then present the pre-write-log mechanism, which RedT uses to ensure efficient transaction processing, and the logging mechanism within RedT. After that, we perform an analysis by comparing RedT with several existing distributed transaction processing protocols in terms of inter-DC communication costs and round-trips.

4.1 Key Data Structures

To facilitate concurrency control, we propose an RDMA-friendly hash table that is used to efficiently retrieve data items located in the other nodes of the same data center, as well as some auxiliary information. To facilitate logging, we propose an RDMA-friendly circular buffer that maintains redo logs only.

Data items and the auxiliary information. In each node, we maintain a set of primary/secondary replicas in its MemStore. As depicted in Figure 5, for each data item x , x is associated with a version array comprising versions x_v of x , and a latch $x.latch$, which R1.2

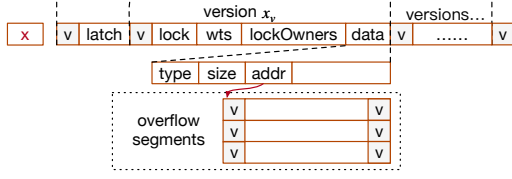


Fig. 5 Data structure of the data items

is utilized to protect concurrent access to the entire data item. Each version x_v maintains a lock $x_v.lock$, a timestamp $x_v.wts$, and a list $x_v.lockOwners$ of transactions that acquire the lock. The $x_v.lock$ is used to manage concurrent access to version x_v . Note $x_v.lock$ is a compound data object with two variables $lockType$ and $lockNum$. The value of $x_v.lock.lockType$ is either *NULL*, *EX*, or *SH*, meaning there is either no lock, an exclusive lock, or one/more shared locks on x_v , respectively. $x_v.lock.lockNum$ represents how many shared locks on x_v are granted. To facilitate the implementation using RDMA verbs, the size of list $x_v.lockOwners$ is fixed. In this way, when the list is full, any transaction that applies to acquire the lock on x_v would abort. If the transaction commits or aborts, it will be removed from the list $x_v.lockOwners$. $x_v.wts$ is the commit timestamp of transactions that have ever written x_v . It is used for the coordinator to check whether versions of x from different replicas of the same partition are consistent. To handle potential inconsistent read results from RDMA_READ, we employ the cache line version scheme [78,16]. As illustrated in Figure 5, we store a version number, denoted as v , at the start and end of each data item, as well as in the header of each version. These version numbers are used to indicate data consistency. If all version numbers (v) values within a data item have the same value, it signifies data consistency; otherwise, it implies that the data item is being modified by another RDMA_WRITE operation. Consequently, an RDMA_READ operation must be issued to re-read the data item. The specific methodology employed in RedT will be elaborated upon in Section 2.4.

Discussion. To manage variable-length data, we employ fixed-size *overflow segment*, inspired by the out-of-table model used in herd [26]. As depicted in Figure 5, each data version x_v can be organized into a fixed-sized *data* structure and multiple overflow segments, allowing efficient storage of variable-sized data within fixed-size data structures. The *data* structure includes: (1) a data item type (*type*), indicating whether x_v is variable-length; (2) a segment count field (*count*), specifying the number of overflow segments x_v needs; (3) an address pointer (*addr*), referencing the initial overflow segment containing the remaining data con-

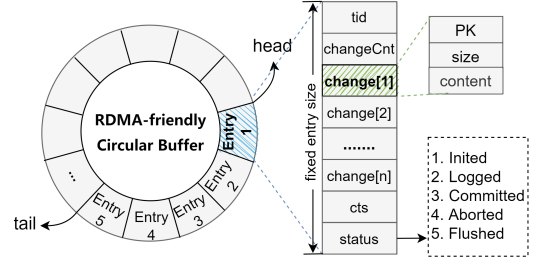


Fig. 6 Data structure of the circular buffer

tent; (4) a field storing either a portion of the variable-length data or the complete data if it fits entirely within this field. As shown in Figure 5, the overflow segments containing the remaining data content are stored continuously. Each overflow segment has the following layout: The first and last 8 bytes store the version number v , which indicates whether the overflow segments are consistent with the entire data item. The remaining portion of the segment contains the actual data value. Upon retrieving variable-length data, we first obtain the count of overflow segments (*count*) and the memory address (*addr*) of the first overflow segment. Subsequently, issue an RDMA_READ to retrieve all necessary overflow segments to reconstruct the complete variable-length data.

RDMA-friendly hash table. To make direct access to each data item x in any replicas from other nodes in the same data center using RDMA verbs, we follow [41] to build a special RDMA-friendly hash table where each key is the primary key, $x.PK$, of x , and its corresponding value is the address $x.addr$ of x in MemStore. Using $x.addr$, we can issue RDMA verbs to fetch x and its auxiliary information. Given a primary key $x.PK$, it is theoretically shown that, based on this hash table, it takes an average of 1.6 and a maximum of 3 RDMA invocations to obtain $x.addr$ [41].

RDMA-friendly circular buffer. We maintain a single RDMA-friendly circular buffer in the MemStore of each node. It stores redo logs only. An executor in one node can directly write log entries to the circular buffer of another node in the same data center using RDMA verbs. An overview of the circular buffer is shown in Figure 6. The buffer has a fixed number of slots, each of which can contain one log entry. In our design, a write is modeled as a change. a primary key $x.PK$ of data item x , *size* and *content* of the write. A log entry comprises *tid*, the ID of the transaction T , the number *changeCnt* of changes, the commit timestamp *cts* of T , and the status *status* of the transaction. To facilitate the writing of logs, we maintain two variables *head* and *tail* for each circular buffer. Because of the concurrent writes from different nodes, we carefully design the log-

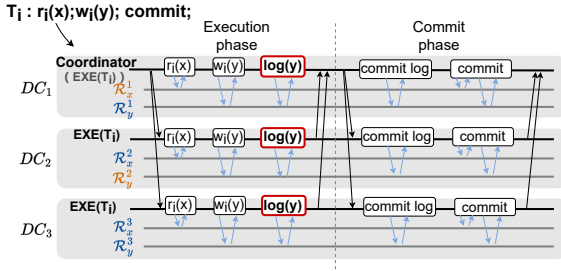


Fig. 7 Inter-DC transaction processing

ging algorithm, which will be elaborated on in Section 4.3, to make sure the writes are safe.

Discussion. The circular buffer of our work has two differences from that in Active-Memory [72]. On the one hand, Active-Memory uses the undo log, while ours uses a redo log. On the other hand, we employ a single circular buffer per node, while Active-Memory employs a set of circular buffers per node with each for a separate target node to be communicated. For this reason, we can make better use of memory space, especially when the workloads to nodes are skewed. As the cost, concurrent updates to the buffer from multiple nodes could become an overhead. To solve this problem, we make a careful design of our logging algorithm, which will be discussed in Section 4.3.

4.2 Pre-Write-Log Mechanism and RedT-SER

In this section, we present the pre-write-log mechanism, which leverages the RDMA-based shared-memory architecture to process sub-transactions in each data center. For clarity, we illustrate this mechanism based on RedT-SER, the variant of RedT that guarantees serializability. RedT-SER performs concurrency control based on the two-phase locking protocol (2PL) and uses the No-Wait deadlock resolution policy [8]. Given a data item x , let $\mathcal{R}_x^j$ be the j -th replica, $\mathcal{N}(\mathcal{R}_x^j)$ be the node that $\mathcal{R}_x^j$ locates in, and $\mathcal{N}(\mathcal{R}_x^j).DC$ be the data center that $\mathcal{N}(\mathcal{R}_x^j)$ locates in. In $\mathcal{N}(\mathcal{R}_x^j).DC$, we select one executor $EXE(T_i)$ to handle concurrency control and logging for transaction T_i . The transaction processing logic in RedT-SER is as follows.

In the execution phase, for any read operation $r_i(x)$, we read data item x after acquiring a shared lock on x across all replicas in its partition. Similarly, for any write operation $w_i(x)$, we acquire an exclusive lock on x across all replicas in its partition before performing the write. To achieve this, the coordinator first distributes T_i to each data center to conduct read and write operations on all replicas for each data item required by T_i . After receiving $r_i(x)$ or $w_i(x)$, $EXE(T_i)$ first issues an

RDMA verb, namely RDMA_CAS, to acquire a *latch* on x from replica $(\mathcal{R}_x^j)$, which prevents x from being modified simultaneously by other transactions. If it fails to acquire the latch, T_i aborts; otherwise, $EXE(T_i)$ issues another RDMA verb, namely RDMA_READ, to read x and locate its most recent version x_l along with its corresponding lock $x_l.lock$, which retains all locks related to x_l . If there exists a conflict with T_i on x_l , T_i releases the latch on x and aborts; otherwise, $EXE(T_i)$ acquires a new lock on x_l and releases the latch on x by issuing another RDMA verb, namely RDMA_WRITE, in the remote node. After completing all operations, the executor $EXE(T_i)$ writes the redo logs, and sends a commit vote with the data items read in the data center, to the coordinator. If the sub-transaction aborts, $EXE(T_i)$ sends an abort vote to the coordinator.

In the commit phase, the coordinator collects the commit/abort messages from each executor $EXE(T_i)$. If more than $\frac{3}{4}$ of replicas per partition accessed by T_i have consistent data items and vote to commit, the coordinator makes the commit decision, sets the commit timestamp of T_i to $\max(wts)+1$, which is larger than the maximum *wts* among all data items accessed, and notifies each executor to commit the sub-transaction. Otherwise, it makes the abort decision and notifies each executor to abort the sub-transaction. The reason why we check how many replicas have the consistent data items is that due to the failure reasons, the minor portion of replicas may not be updated by the latest commit transaction, causing the later transactions to read the inconsistent data items. For each executor $EXE(T_i)$ that previously voted to commit, after receiving the commit message from the coordinator, $EXE(T_i)$ writes the commit log, makes every write $w_i(x)$ to each replica, and releases all granted locks. If $EXE(T_i)$ previously voted to abort, it additionally records the redo logs that were not written during the execution phase. The redo and commit logs will be replayed to apply the modification made by T_i according to RedT's logging mechanism, which will be further discussed in Section 4.3. Note that $EXE(T_i)$ writes an abort log if it receives an abort message.

For illustration purposes, we list some key functions in Algorithm 1. Function *GetItemAddr* is used to obtain $x.addr$ of x by taking primary key $x.PK$ based on the RDMA-friendly hash table (lines 1–2). Function *AcqLatch* is used to acquire a latch on x to prevent x from being modified by other transactions simultaneously (lines 3–4). Function *AcqLock* is used to acquire an exclusive or shared lock on x 's most recent version x_l . *EXE(T_i)* first acquires the *latch* on the data item before acquiring a read or write lock on a specific version (line 7). This *latch* prevents concurrent lock ac-

R1:2

Algorithm 1: The functions of RedT-SER

```

1 Function GetItemAddr( $x.PK$ ):
2   return  $x.addr \leftarrow hashTable(x.PK)$ ;
3 Function AcqLatch( $x.addr$ ):
4   return  $RDMA.CAS(x.addr, x.latch, 0, 1)$ ;
5 Function AcqLock( $x.PK, T_i, newLock$ ):
6    $x.addr \leftarrow GetItemAddr(x.PK)$ ;
7   while  $\neg AcqLatch(x.addr)$  do goto line 7;
8    $x \leftarrow RDMA.READ(x.addr)$ ;
9    $x_l \leftarrow LatestVersion(x)$ ;
10  if  $conflict(x_l.lock, newLock)$  then
11    return  $RDMA.CAS(x.addr, x.latch, 1, 0)$ ;
12   $x_l.lock.lockType \leftarrow newLock.lockType$ ;
13   $x_l.lock.lockNum ++$ ;
14   $x_l.lockOwners \leftarrow T_i.tid \cup x_l.lockOwners$ ;
15   $x.latch \leftarrow 0$ ;
16  return  $RDMA.WRITE(x.addr, x)$ ;
17 Function RlsLock( $x.PK, T_i, newValue$ ):
18   $x.addr \leftarrow GetItemAddr(x.PK)$ ;
19  while  $\neg AcqLatch(x.addr)$  do goto line 19;
20   $x \leftarrow RDMA.READ(x.addr)$ ;
21   $x_l \leftarrow LatestVersion(x)$ ;
22   $x_l.lock.lockNum --$ ;
23   $x_l.lockOwners \leftarrow x_l.lockOwners - T_i.tid$ ;
24  if  $x_l.lock.lockNum = 0$  then
25     $x_l.lock.lockType \leftarrow NULL$ ;
26  if  $newValue \neq NULL$  then
27    create new version  $x_{l+1}$  in  $x$ ;
28     $x_{l+1}.data \leftarrow newValue$ ;  $x_{l+1}.wts \leftarrow T_i.cts$ ;
29   $x.latch \leftarrow 0$ ;
30  return  $RDMA.WRITE(x.addr, x)$ ;
31 Function Read( $x.PK, T_i$ ):
32   $x.addr \leftarrow GetItemAddr(x.PK)$ ;
33   $newLock.lockType \leftarrow Sh$ ;
34  if  $\neg AcqLock(x.PK, T_i, newLock)$  then Abort  $T_i$ ;
35   $x \leftarrow RDMA.READ(x.addr)$ ;
36   $T_i.rs \leftarrow \{x\} \cup T_i.rs$ ;
37 Function Write( $x.PK, newValue, T_i$ ):
38   $newLock.lockType \leftarrow Ex$ ;
39  if  $\neg AcqLock(x.PK, T_i, newLock)$  then Abort  $T_i$ ;
40   $T_i.ws \leftarrow \{x\} \cup T_i.ws$ ;
41 Function Commit( $T_i$ ):
42  foreach  $k \in T_i.rs$  do
43     $RlsLock(k.PK, T_i, NULL)$ 
44  foreach  $k \in T_i.ws$  do
45     $RlsLock(k.PK, T_i, newValue)$ 

```

quisitions by other transactions, ensuring exclusive access to the data item's metadata. Then, $EXE(T_i)$ issues an $RDMA.READ$ to obtain x and its most recent version x_l (lines 8–9). If there is an exclusive lock on x_l , or T_i attempts to acquire an exclusive lock while $x_l.lock.lockType = Sh$, it fails (lines 10,11); otherwise, $EXE(T_i)$ updates $x_l.lock$ and $x_l.lockOwners$ to acquire a new lock and releases the latch by issuing $RDMA.WRITE$ (lines 12–16). Following a reverse logic of function $AcqLock$, function $RlsLock$ is used to release the

remote lock and update the data item for write operations properly (lines 17–30). Functions $Read$ and $Write$ are used to perform remote read and write. For read, $EXE(T_i)$ first tries to acquire a shared lock on x using function $AcqLock$ (line 34). If the lock is acquired, $EXE(T_i)$ issues another $RDMA.READ$ to read the whole x and adds x to the read set $T_i.rs$ (lines 35–36). For write, $EXE(T_i)$ uses function $AcqLock$ to acquire a remote exclusive lock on x , and adds x to the write set $T_i.ws$ if the lock is acquired (lines 37–40). Upon committing or aborting, $EXE(T_i)$ sequentially releases the locks (lines 41–45).

4.3 Logging

Logging is performed in each data center to persist the outcome of concurrency control and the data written by sub-transactions. To do this, $EXE(T_i)$ first performs redo log replication to write redo logs to the log buffer in each data node containing corresponding replicas. Note that we ensure that the redo log replication is finished before entering the commit phase. After receiving the commit request from the coordinator, $EXE(T_i)$ then triggers commit log replication to update the status of T_i to committed/aborted. Finally, each data node has a log replay thread to update data items according to the persisted redo logs asynchronously.

Redo log replication. $EXE(T_i)$ performs the concurrency control over each replica separately. After the accomplishment of locking/reading/writing over replicas, $EXE(T_i)$ writes the redo logs to RDMA-friendly circular buffers in MemStores of the nodes that these replicas locate in using RDMA verbs. Take transaction T_i with two operations $r_i(x)$ and $w_i(y)$ in Figure 7 for an example. Because DC_1 , DC_2 , and DC_3 contain the replicas of x and y , we then select one executor in each data center to process the sub-transaction of T_i . During the execution phase, first, the coordinator packs $r_i(x)$ and $w_i(y)$ into the sub-transaction, and sends it to $EXE(T_i)$ for each data center; second, $EXE(T_i)$ performs the concurrency control over the replicas, and writes the redo logs only to the nodes with replicas to be written (highlighted in red of Figure 7), i.e., $EXE(T_i)$ in $DC_1/DC_2/DC_3$ writes the redo log of $w_i(y)$ to the circular buffer in $\mathcal{N}(\mathcal{R}_y^1)/\mathcal{N}(\mathcal{R}_y^2)/\mathcal{N}(\mathcal{R}_y^3)$ using RDMA verbs. If $EXE(T_i)$ fails to acquire the lock on a data item, it skips writing the redo logs. Finally, $EXE(T_i)$ sends the commit/abort messages to the coordinator.

We present how to directly write redo logs to other data nodes of the data center using RDMA verbs in Algorithm 2. For each write $w_i(x)$ in a write set ws , $EXE(T_i)$ builds a change chg for x (line 13) using function $GetChange$ (lines 1–4). $EXE(T_i)$ collects the

Algorithm 2: RedT's log write functions

```

1 Function GetChange( $x$ ):
2    $chg.PK \leftarrow x.PK$ ;  $chg.size \leftarrow x.size$ ;
3    $chg.content \leftarrow x.content$ ;
4   return  $chg$ ;
5 Function GetEntry( $chgs, tid, status$ ):
6   foreach  $i \in [0, chgs.size)$  do
7      $ent.change[i] \leftarrow chgs[i]$ ;
8    $ent.changeCnt \leftarrow chgs.size$ ;
9    $ent.tid \leftarrow tid$ ;  $ent.cts \leftarrow 0$ ;  $ent.status \leftarrow status$ ;
10  return  $ent$ ;
11 Function WriteLog( $T_i, ws, isActiveExe$ ):
12  foreach  $x \in ws$  do
13     $chg \leftarrow GetChange(x)$ ;
14     $chgs[\mathcal{N}(\mathcal{R}_x^s)] \leftarrow chgs[\mathcal{N}(\mathcal{R}_x^s)] \cup chg$ ;
15  foreach node  $\mathcal{N}$  and  $\neg chgs[\mathcal{N}].empty()$  do
16     $ent \leftarrow GetEntry(chgs[\mathcal{N}], T_i.tid, Logged)$ ;
17     $idx \leftarrow RDMA\_FAA(\mathcal{N}.logBuf.tail.addr, 1)$ ;
18    while  $idx - logHead[\mathcal{N}] \geq BUFSIZE$  do
19       $head\_addr \leftarrow \mathcal{N}.logBuf.head.addr$ ;
20       $logHead[\mathcal{N}] \leftarrow RDMA\_READ(head\_addr)$ ;
21     $ent\_addr \leftarrow \mathcal{N}.logBuf.entry[idx \% BUFSIZE].addr$ ;
22     $RDMA\_WRITE(ent\_addr, ent)$ ;
23     $T_i.logSet \leftarrow T_i.logSet \cup \{ent\_addr, ent\}$ ;
24 Function CommitLog( $T_i$ ):
25  foreach  $\langle ent\_addr, ent \rangle \in T_i.logSet$  do
26     $ent.cts \leftarrow T_i.cts$ ;  $ent.status \leftarrow T_i.status$ ;
27     $RDMA\_WRITE(ent\_addr, ent)$ ;
28 Function ReplayLog( $\mathcal{N}$ ):
29   $move \leftarrow true$ ;  $h \leftarrow \mathcal{N}.logBuf.head$ ;
30   $t \leftarrow \min(\mathcal{N}.logBuf.tail, h + BUFSIZE)$ ;
31  foreach  $idx \in [h, t)$  do
32     $ent \leftarrow \mathcal{N}.logBuf.entry[idx \% BUFSIZE]$ ;
33    if  $ent.status = Committed$  then
34      Apply the changes of  $ent$  in node  $\mathcal{N}$  locally;
35    if  $ent.status =$ 
      Committed or Aborted or Flushed then
36       $ent.status \leftarrow Flushed$ ;
37      if  $move$  then
38         $ent.status \leftarrow Inited$ ;
39         $++\mathcal{N}.logBuf.head$ ;
40    else  $move \leftarrow false$ ;

```

changes into a batch that is sent to the same node (lines 12–14). For each node $\mathcal{N}$ in the data center, $EXE(T_i)$ builds a log entry ent based on the changes $chgs[\mathcal{N}]$, and writes ent directly to the circular buffer of $\mathcal{N}$ (lines 15–23). To do this, $EXE(T_i)$ reads the tail position of the target buffer by issuing an RDMA.FAA (line 17). If the tail position exceeds the head position by one round, existing log entries will be overwritten. To solve this problem, $EXE(T_i)$ issues an RDMA.READ to fetch the head position, and if an overwrite exists,

$EXE(T_i)$ continuously issues RDMA.READs to fetch the head position until it is updated by the target node to make the tail position safely writable (lines 18–20). Note that, concurrent reads to the same buffer from different executors could happen. RDMA.FAA guarantees the atomicity that concurrent reads can return different tail positions. If the tail position is safely writable, $EXE(T_i)$ issues an RDMA.WRITE to write the log entry to the circular buffer (lines 21–23). By realizing that directly writing the log entry to the tail position without checking the head position is often safe, we make an optimization to reduce the number of RDMA.READs of head positions. we design a lazy check mechanism, in which $EXE(T_i)$ maintains a local head position $logHead$ (lines 18–20). Only if the tail position exceeds $logHead$ by one round, $EXE(T_i)$ then issues an RDMA.READ to read the head position and update $logHead$ properly (lines 19–20). Due to a later update of logs in the commit phase, $EXE(T_i)$ locally maintains all memory addresses of log entries in other nodes (lines 23).

Commit log replication. After receiving the message from the coordinator, each executor enters the commit phase. We first discuss the case when the coordinator decides to commit. If $EXE(T_i)$ votes to commit, it writes the commit log by updating cts and $status$ of the redo log to $T_i.cts$ and *Committed*; otherwise, $EXE(T_i)$ modifies the status of the sub-transaction from aborted to committed, and follows the logic of function WriteLog to write a redo log with $cts = T_i.cts$ and $status = Committed$. We now discuss why it is correct to directly modify the status of the sub-transaction. Because the commit decision of T_i indicates that T_i still holds locks on more than $\frac{3}{4}$ of replicas for each partition, and we follow 2PL to do concurrency control, for any concurrent transaction T_j with conflicts with T_i , the serializable order between T_i and T_j is either $T_j \rightarrow T_i$ if T_j has already committed or otherwise $T_i \rightarrow T_j$. To help write the data items in less than $\frac{1}{4}$ of replicas that vote to abort, we write the redo log to the circular buffers in the corresponding data nodes and replay the logs asynchronously after the commit of T_i . If $(T_j \rightarrow T_i)$ or $(T_i \rightarrow T_j)$ and T_j has not committed, replaying the log does not affect the correctness of T_j ; If $T_i \rightarrow T_j$ and T_j have committed, replaying the logs would overwrite the data items that T_j writes. However, by replaying the logs through creating new versions, i.e., generating a new version for T_i , it will not overwrite the new version created by T_j . We then discuss the case when the coordinator decides to abort. If $EXE(T_i)$ votes to commit, it writes the abort log by setting $status$ of the redo log to *Aborted*; otherwise, it returns. Details of commit log replication in each executor are given in function *CommitLog* of Algorithm 2, which is self-explained.

Asynchronous redo log replay. Because data items in replicas might not be updated even after the transaction commits due to the node failures, to make them consistent with the other replicas, we create a separate thread in each node that continuously scans the circular buffer and replays redo logs for replicas if its status is committed. In the *ReplayLog* function of Algorithm 2, we scan all entries from head to tail in the local log buffer (lines 29–31). Upon encountering an entry with a *Committed* status, we apply all changes of this entry by creating new versions. (lines 33–34). If all versions of a data item x have their wts greater than the commit timestamp cts , we skip this entry following the Thomas write rule [56]. Otherwise, we create a new version for x by replacing the oldest existing version of x . Finally, we update the status of each entry with *Aborted* or *Committed* to *Flushed*, and if the head is movable and the entry of the head position can be released for use, we set the status of the entry to *Inited* and move forward of the head by a slot (lines 35–39). The head cannot move forward any further during this round of scan once encountering an entry with the *Logged* or *Inited* status (line 40).

4.4 RedT-SI

In this section, we introduce RedT-SI, a variant of RedT that supports snapshot isolation. As discussed in Section 2.4, snapshot isolation requires a consistent snapshot, typically represented by a timestamp. However, in an inter-DC setting, acquiring a timestamp from a conventional centralized timestamp oracle [45] incurs costly inter-DC communications. We therefore opt to employ logical clocks [55, 33] in RedT-SI. However, implementing logical clocks in an RDMA-based system is not trivial as RDMA bypasses CPUs. In particular, when an executor interacts with data nodes via RDMA to perform reads or writes of a transaction T_i , the logical clocks on these data nodes might need to be updated, which is not supported by conventional techniques relying on local CPU participation.

To tackle this challenge, we introduce the RDMA-based logical clock maintenance mechanism, which maintains the logical clock in the registered memory of data nodes. Executors can then utilize RDMA to update the logical clocks of data nodes remotely. Specifically, we maintain a 64-bit logical clock Clk in the memory of each node. Then, we design three Functions *GetTS*, *UpdateTS*, and *UpdateRemoteTS* in Algorithm 3 to update Clk locally and remotely. The function *GetTS* increments the local logical clock Clk by 1 (lines 2,3) and returns the new timestamp (lines 5). The function

Algorithm 3: Timestamp allocation mechanism

```

1 Function GetTS():
2    $TS \leftarrow Clk + 1$ ;
3   if  $\neg RDMA\_CAS(Clk.addr, Clk, tTS)$  then
4     goto 2;
5   return  $TS$ ;
6 Function UpdateTS( $TS_C$ ):
7    $temp \leftarrow \max\{TS_C, Clk\} + 1$ ;
8   if  $\neg RDMA\_CAS(Clk.addr, Clk, temp)$  then
9     goto 7;
10 Function UpdateRemoteTS( $Clk_R.addr$ ):
11    $temp \leftarrow Clk$ ;
12   // read remote clock of other node;
13    $Clk_R \leftarrow RDMA\_READ(Clk_R.addr)$ ;
14    $temp \leftarrow \max\{Clk_R, temp\} + 1$ ;
15   if  $\neg RDMA\_CAS(Clk_R.addr, Clk_R, temp)$  then
16     goto 11;
17    $UpdateTS(temp)$ ;

```

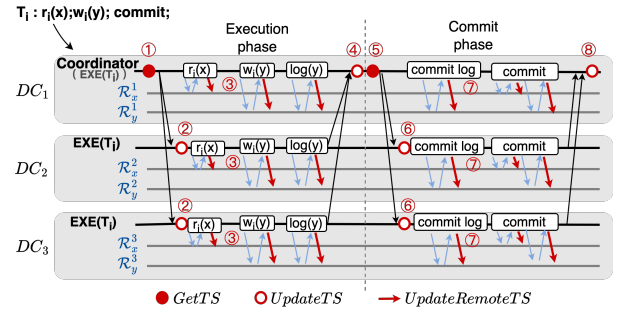


Fig. 8 An example of RedT-SI.

UpdateTS updates local logical clock Clk based on a remote timestamp TS_C , setting Clk to $\max\{TS_C, Clk\} + 1$ (lines 7–9). Further, the function *UpdateRemoteTS* adjusts the remote logical clock Clk_R of a remote data node $\mathcal{N}_R$ according to the local clock Clk , setting Clk_R to $\max\{Clk, Clk_R\} + 1$ (lines 11–15). Afterward, the local logical clock Clk is updated with Clk_R (line 17).

With the RDMA-based logical clock maintenance mechanism, we outline the execution logic of RedT-SI, which also involves two distinct phases to execute a transaction T_i : the execution phase and the commit phase. As shown in Figure 8, during the execution phase, the coordinator first acquires a snapshot timestamp $T_i.snapshot$ by invoking *GetTS* (①), and then dispatches the transaction to the corresponding data centers for execution. In each data center, upon receiving the transaction, the executor updates its local logical clock by calling *UpdateTS* (②) and then invokes *UpdateRemoteTS* to update the logical clocks on remote data nodes (③). In the commit phase, the coordinator collects the execution results returned by

R1:1

R1:1

Algorithm 4: RedT-SI

```

1 Function RlsLatch( $x.addr$ ):
2   return  $RDMA.CAS(x.addr, x.latch, 1, 0)$ 
3 Function Read( $x.PK, T_i$ ):
4    $x.addr \leftarrow GetItemAddr(x.PK)$ ;
5    $x \leftarrow RDMA.READ(x.addr)$ ;
6   if  $x$  is inconsistent then goto 5 ;
7    $x_v \leftarrow getProperVersion(x, T_i)$ ;
8   if  $x_v = NULL$  then Abort  $T_i$  ;
9    $x_l \leftarrow LatestVersion(x)$ ;
10  if  $x_v = x_l$  and  $x.latch = 1$  then Abort  $T_i$  ;
11   $T_i.rs \leftarrow \{x_v\} \cup T_i.rs$ ;
12 Function Write( $x.PK, newValue, T_i$ ):
13   $x.addr \leftarrow GetItemAddr(x.PK)$ ;
14  if  $\neg AcqLatch(x.addr)$  then
15    Abort  $T_i$ ;
16   $x \leftarrow RDMA.READ(x.addr)$ ;
17   $x_l \leftarrow LatestVersion(x)$ ;
18  if  $x_l.wts > T_i.snapshot$  then
19    RlsLatch( $x.addr$ );
20    Abort  $T_i$ ;
21   $T_i.ws \leftarrow \{\langle x, newValue \rangle\} \cup T_i.ws$ ;
22 Function Commit( $T_i$ ):
23  foreach  $\langle k, newValue \rangle \in T_i.ws$  do
24     $k.addr \leftarrow GetItemAddr(k.PK)$ ;
25    create new version  $k_{l+1}$  in  $k$ ;
26     $k_{l+1}.data \leftarrow newValue$ ;
27     $k_{l+1}.wts \leftarrow T_i.cts$ ;
28     $v' \leftarrow k.v + 1$ ;
29    update all version number  $v$  with  $v'$ ;
30     $k.latch \leftarrow 0$ ;
31     $RDMA.WRITE(k.addr, k)$ ;

```

each data center, and updates its local logical clock using *UpdateTS* (④). Afterward, the coordinator determines if T_i can be committed. If more than $\frac{3}{4}$ of replicas per partition accessed by T_i return consistent data items and vote to commit, the coordinator makes the commit decision and retrieves the commit timestamp $T_i.cts$ using *GetTS* (⑤). Subsequently, it sends commit messages, along with $T_i.cts$, to all data centers to execute commit operations. Otherwise, the coordinator sends abort messages to all data centers. Upon receiving the commit/abort messages, the executors call *UpdateTS* (⑥) to update their local logical clock, and then utilize *UpdateRemoteTS* to update the remote logical clock while executing commit/abort operations on remote data nodes (⑦). Finally, after all executors complete their commit/abort operations and send responses to the coordinator, the coordinator invokes *UpdateTS* to update its local logical clock.

R1:2 RedT-SI differs from RedT-SER in the read and write operations and metadata utilization. In particular, RedT-SER requires both the data item *latch* and the per-version *lock* to enable concurrency control

at the version level. In contrast, RedT-SI performs concurrency control at the data item level, so that it only needs the *latch*. As shown in Algorithm 4, the functions for each operation are detailed below. The function *Write* illustrates the write operations of RedT-SI.

Transaction T_i first acquires a data item's *latch* before write operations (line 13), serving as an exclusive lock on the entire data item. Then T_i examines the write timestamp *wts* of version x_l . If $x_l.wts$ is greater than $T_i.snapshot$, it indicates that data item x was updated by other transactions that have already committed after transaction T_i began; thus, T_i will abort (lines 17–19). Otherwise, transaction T_i can proceed to modify the current data item. The function *Read* illustrates the read operations of RedT-SI. Transaction T_i first obtains the remote data item x by issuing *RDMA.READ*. If T_i reads an inconsistent data item x , that is, all the version numbers v in x do not have the same value, T_i re-reads data item x until reads a consistent data item. Then, transaction T_i utilizes its snapshot time $T_i.snapshot$ to retrieve the most recent version x_v of data item x from all versions, where $x_v.wts$ is less than $T_i.snapshot$ (lines 6). If the version x_v does not exist, indicating that the version intended to be read by T_i has been overwritten by the newer versions, T_i aborts (line 7). If T_i needs to read the latest version and the *latch* is already held by another transaction, transaction T_i will abort (lines 8,9). Otherwise, transaction T_i reads version x_v (line 10). The function *Commit* describes the commit operations of RedT-SI. For each data item k that requires updating by transaction T_i , T_i generates a new version k_{l+1} , assigns $k_{l+1}.data = newValue$, and $k_{l+1}.wts = T_i.cts$. Besides, T_i increments all version numbers v within the data item by 1 (line 28–29). Subsequently, T_i resets $k.latch$ to 0, executes *RDMA.WRITE* to update the remote data item k , and releases the *latch* on k .

4.5 Analysis on Inter-DC Round-trips

In this section, we conduct a thorough analysis by comparing RedT with four representative distributed transaction processing protocols in terms of inter-DC communication costs and round-trips.

Analysis under serializability. To ensure a fair comparison, we assume that all protocols use 2PL-based concurrency control algorithms with serializability guarantees. Given a read-write inter-DC transaction T_{ne} , let $\mathcal{DC}(T_{ne})$ be the collection of data centers that T_{ne} involves. For brevity, symbol n is set to be the number of data centers in $\mathcal{DC}(T_{ne})$, i.e., $n = |\mathcal{DC}(T_{ne})|$, m ($m > 0$) is the fixed number of secondary replicas per partition, and M ($M > 0$) is the fixed number of backups for

Table 1 A comparison of existing protocols with RedT under serializability in terms of inter-DC communications ($\mathcal{C}$) and round-trips. T_{ne} is a read-write inter-DC transaction, T_{re} is a read-only inter-DC transaction, T_{na} is a read-write intra-DC transaction, and T_{ra} is a read-only intra-DC transaction. $K = \sum_{i=2}^n (a_i + b_i)$.

Protocol	Phase	# of $\mathcal{C}$	# of round-trips			
			T_{ne}	T_{re}	T_{na}	T_{ra}
2PC-Paxos	exe	$2K$	1	1	0	0
	prep	$2\sum_{i=2}^n a_i + 2m\sum_{i=1}^n a_i + 2M$	$2\sim 3$	1	2	1
	com	$2K + 2m\sum_{i=1}^n a_i$	2	1	1	0
EP-Paxos	exe	$2K + 2m\sum_{i=1}^n a_i + 2M$	3	2	2	1
	com	$2K + 2m\sum_{i=1}^n a_i$	2	1	1	0
MDCC	com	$2\sum_{i=2}^n (a_i + b_i + c_i + d_i)$	1	1	1	1
		$2K + 2m\sum_{i=1}^n (a_i + b_i)$	2	2	1	1
TAPIR G-PAC	exe	$2\sum_{i=2}^n (a_i + b_i + c_i + d_i)$	1	1	1	1
	prep	$2\sum_{i=2}^n (a_i + c_i)$	1	0	1	0
	com	$2\sum_{i=2}^n (a_i + b_i + c_i + d_i)$	1	1	1	1
NCC	exe	$2K + 2m\sum_{i=1}^n a_i$	2	2	1	1
	com	$2K + 2m\sum_{i=1}^n a_i$	2	0	1	0
Calvin	seq	n/B	1	1	1	1
	exe	n	0~1	0~1	0	0
RedT-SER	exe	$2(n-1)$	1	1	1	1
	com	$2(n-1)$	1	1	1	1

each coordinator. In each $DC_i \in \mathcal{DC}(T_{ne})$, suppose that T_{ne} writes/reads data items to/from primary (secondary) replicas in a collection of a_i (c_i) / b_i (d_i) nodes, respectively. That is, $a_i = |\{\mathcal{N}(\mathcal{R}_x^p) | x \in T_{ne}.ws\}|$, $b_i = |\{\mathcal{N}(\mathcal{R}_x^s) | x \in T_{ne}.rs\}|$, $c_i = |\{\mathcal{N}(\mathcal{R}_x^s) | x \in T_{ne}.ws\}|$ and $d_i = |\{\mathcal{N}(\mathcal{R}_x^p) | x \in T_{ne}.rs\}|$. By default, if there are overlaps, nodes are counted only once with the priority of a_i , b_i , c_i , and d_i in descending order. For illustration, let $K = \sum_{i=2}^n (a_i + b_i)$. B denotes the batch size, representing the number of transactions per batch in Calvin.

Table 1 summarizes the number of inter-DC communications ($\mathcal{C}$) and inter-DC round-trips to process T_{ne} for RedT and other state-of-the-art protocols. Without loss of generality, the coordinator of T_{ne} is assumed to be located in $DC_1 \in \mathcal{DC}(T_{ne})$. Compared with that of inter-DC communications over TCP/IP network, the cost of intra-DC communications using RDMA-capable networks is negligible (see Figure 1(a)). Thus, we only take the former into account.

For 2PC-Paxos, there are 5 to 6 inter-DC round-trips in total to process T_{ne} . We now step through the processing with reference to Figure 2. For round-trip ① in the execution (abbreviated as exe) phase, $2K$ communications are required for the coordinator to send/re-

ceive messages to/from the primary replicas. The prepare (abbreviated as prep) phase takes a maximum of 3 round-trips including ②, ③ and ④, for which $2\sum_{i=2}^n a_i$, $2m\sum_{i=1}^n a_i$ and $2M$ communications are required, respectively. For the special case that all primary replicas in $T_{ne}.ws$ located in DC_1 , i.e., $\sum_{i=2}^n a_i = 0$, round-trip ② can be eliminated, resulting in only 2 inter-DC round-trips in the prepare phase. Similarly, in the commit (abbreviated as com) phase, processing T_{ne} requires 2 inter-DC round-trips that are ⑤ and ⑥, and $2K + 2m\sum_{i=1}^n a_i$ communications in total, where $2K$ is the number of communications that are used to release the locks on data items in other nodes. As an optimization of the standard 2PC-Paxos protocol, Early Prepare [53] reduces one potential round-trip by combining the prepare phase with the execution phase. Early Prepare also writes redo logs in the execution phase, as RedT does. However, unlike Early Prepare, which runs sub-transactions in data node granularity, RedT runs sub-transactions in data center granularity, and thus achieves lower coordination overhead and fewer inter-DC communications, which is verified in Table 3. Besides, Early Prepare is proposed for systems without replication, while RedT is designed for replicated systems. To make a fair comparison, we extend it to the replicated system by introducing the standard Paxos protocol for replication. We name this implementation EP-Paxos for brevity, which requires 5 inter-DC round-trips in total to process T_{ne} . As opposed to 2PC-Paxos, MDCC [29] tries to commit T_{ne} after the execution phase. In the absence of conflict or under the assumption of commutative operations, MDCC can commit with a single round-trip and $2\sum_{i=2}^n (a_i + b_i + c_i + d_i)$ communications by accessing all replicas directly. To be fair, we do not consider the commutative optimization. In the case that any conflict exists, processing T_{ne} using MDCC requires 2 additional network round-trips and $2K + 2m\sum_{i=1}^n (a_i + b_i)$ communications, which are similar to that in the commit phase of standard 2PC-Paxos. NCC[36] is designed mainly for intra-DC environments, where an inter-DC read-write transaction requires 4 rounds of inter-DC network communication to process T_{ne} . This is because NCC needs to make the modification on the metadata of data items and synchronize these modification to all replicas in each phase. Despite NCC having an asynchronous commit mechanism, which allows immediate result return to clients after execution, the subsequent asynchronous commit phase still involves message exchanges, potentially affecting throughput. Since this mechanism is broadly applicable to protocols like those in Table 3, this paper will account for the message overhead of asynchronous commits to ensure a fair performance evaluation. TAPIR

R3:3

[73] or G-PAC [40] requires 3 inter-DC round-trips to process T_{ne} . In the execution phase, the coordinator sends messages to all replicas involved in a single round-trip. In the prepare phase, the coordinator directly replicates the redo logs to the primary and secondary replicas. As a result, it requires 1 round-trip and at least $2 \sum_{i=2}^n (a_i + c_i)$ communications. Similar to the execution phase, in the commit phase, the coordinator needs $2 \sum_{i=2}^n (a_i + b_i + c_i + d_i)$ communications with all replicas accessed in other data centers in a single round-trip.

R1:5 Calvin, a deterministic algorithm, requires 1-2 inter-DC round-trips. It employs a sequencer (abbreviated as seq) to pre-order transactions, batching and partitioning them into sub-transactions based on partitions before distributing them to nodes. Nodes then execute sub-transactions independently after acquiring locks, with an additional round-trip for read-set synchronization only when cross-sub-transaction dependencies exist. Despite its low round-trip count, Calvin relies on pre-defined read/write sets and batch processing, limiting its applicability. RedT requires exactly 2 inter-DC round-trips. In the execution phase, the coordinator sends/receives messages to/from $EXE(T_{ne})$ in other $n - 1$ data centers in 1 round-trip. The coordinator then decides the committed/aborted status of T_{ne} locally by collecting the votes from the messages. In the commit phase, the decision is sent to each $EXE(T_{ne})$ in other $n - 1$ data centers. After the decision is synchronized, each $EXE(T_{ne})$ notifies the coordinator with a total of $n - 1$ communications.

We extend the analysis from T_{ne} to other three kinds of transactions T_{re} , T_{na} , and T_{ra} in Table 1. For read-only transactions T_{re} and T_{ra} , we have $a_i = 0$ and $c_i = 0$, for $1 \leq i \leq n$, meaning that it is unnecessary to synchronize redo logs among writes. For intra-DC transactions T_{na} and T_{ra} , we have $a_i = 0$ and $b_i = 0$, for $2 \leq i \leq n$, meaning that $K = 0$ and it is unnecessary for the coordinator to send messages to primary replicas in other data centers. For reference, we list the number of inter-DC round-trips for T_{re} , T_{na} , and T_{ra} in the last three columns of Table 1, respectively. To summarize, in most cases, RedT enjoys a lower or at least equal number of inter-DC round-trips and inter-DC communications compared with existing protocols.

Analysis under snapshot isolation. We then summarize the number of inter-DC communications ($\mathcal{C}$) and inter-DC round-trips of the six protocols under the snapshot isolation level in Table 2. The results are similar to Table 1, except for the round-trip number concerning read-only transactions T_{re} and T_{ra} . Under the snapshot isolation level, T_{re} does not require the synchronization of redo logs or the acquisition of locks, and therefore, T_{re} can commit directly after accessing

Table 2 A comparison of existing protocols with RedT under snapshot isolation in terms of inter-DC communications ($\mathcal{C}$) and round-trips. T_{ne} is a read-write inter-DC transaction, T_{re} is a read-only inter-DC transaction, T_{na} is a read-write intra-DC transaction, and T_{ra} is a read-only intra-DC transaction. $K = \sum_{i=2}^n (a_i + b_i)$.

Protocol	Phase	# of $\mathcal{C}$	# of round-trips			
			T_{ne}	T_{re}	T_{na}	T_{ra}
2PC-Paxos	exe	$2K$	1	1	0	0
	prep	$2 \sum_{i=2}^n a_i + 2m \sum_{i=1}^n a_i + 2M$	2~3	0	2	0
	com	$2 \sum_{i=2}^n a_i + 2m \sum_{i=1}^n a_i$	2	0	1	0
PostSI	exe	$2K$	1	1	0	0
	prep	$2 \sum_{i=2}^n a_i + 2L + 2m \sum_{i=1}^n a_i + 2M$	2~3	0	2~3	0
	com	$2 \sum_{i=2}^n a_i + 2m \sum_{i=1}^n a_i$	2	0	1	0
EP-Paxos	exe	$2K + 2m \sum_{i=1}^n a_i + 2M$	3	1	2	0
	com	$2 \sum_{i=2}^n a_i + 2m \sum_{i=1}^n a_i$	2	0	1	0
MDCC	com	$2 \sum_{i=2}^n (a_i + b_i + c_i + d_i)$	1	1	1	1
		$2 \sum_{i=2}^n a_i + 2m \sum_{i=1}^n (a_i)$	2	0	1	0
TAPIR G-PAC	exe	$2 \sum_{i=2}^n (a_i + b_i + c_i + d_i)$	1	1	1	1
	prep	$2 \sum_{i=2}^n (a_i + c_i)$	1	0	1	0
	com	$2 \sum_{i=2}^n (a_i + c_i)$	1	0	1	0
RedT-SI	exe	$2(n - 1)$	1	1	1	1
	com	$2 \sum_{i=2}^n (a_i + c_i)$	1	0	1	0

the data item with one round of inter-DC round-trips. Regarding T_{ra} , 2PC-Paxos and EP-Paxos can commit T_{ra} without any inter-DC communications, as T_{ra} can access data directly from the primary replicas within the current data center. **R3:3** PostSI [76] is state-of-the-art SI protocol based on 2PC-Paxos. PostSI introduces an additional network round trip in 2PC to dynamically determine transaction order. In this round, PostSI communicates with nodes (denoted as L in Table 2) associated with transactions it depends on to adjust the timestamp intervals of those dependent transactions, thereby reflecting the dependency relationships.

In TAPIR/G-PAC, MDCC, and RedT, T_{ra} requires a single inter-DC round-trip to read from all the replicas to guarantee data consistency, since these protocols do not have distinct primary replicas.

5 Optimization for Read-Only Transactions

In this section, we optimize read-only transactions in RedT to commit without requiring inter-DC round-

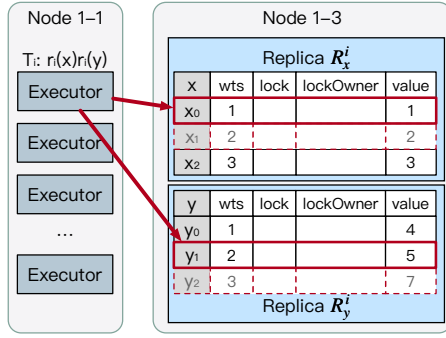


Fig. 9 An example of inconsistent reads with MVCC

trips. As discussed in Section 4.2, RedT transactions typically access all the replicas for each data item to read a consistent data item, which incurs inter-DC round-trips. However, for read-only transactions, these costly inter-DC communications can be avoided by reading from a past consistent snapshot available within a single data center. We therefore introduce a watermark-based read mechanism to efficiently establish a consistent snapshot for read-only transactions without additional inter-DC communications.

Obtaining a consistent snapshot from the current data center is not trivial. As discussed in Section 4, for each transaction, we ensure that more than $\frac{3}{4}$ of the replicas per partition agree on the consistent data. However, the replicas involved may vary between transactions, which can cause the version array of data items to be discontinuous in some replicas. For example, as shown in Figure 9, node $\mathcal{N}_{1-3}$ contain versions x_0 and x_2 of the data item x and versions y_0 and y_1 of the data item y , while versions x_1 and y_2 are absent. If a read-only transaction T_i employs a snapshot timestamp of 2 to access the previous versions of data items x and y on nodes $\mathcal{N}_{1-3}$, it mistakenly reads the inconsistent versions x_0 and y_1 , without being aware of the absence of version x_1 .

To address this challenge, we introduce a watermark $x.watermark$ for each data item x , representing the version up to which the version array is continuous in the current data node. Continuing the example, on node $\mathcal{N}_{1-3}$, data item x in Figure 9 maintains its watermark on $\mathcal{N}_{1-3}$ at 1, while data item y maintains its watermark on $\mathcal{N}_{1-3}$ at 2. Consequently, when the read-only transaction T_i accesses data items x and y , it can retrieve the consistent versions x_0 and y_0 with write timestamps ($x_0.wts$ and $y_0.wts$) that are lower than $x.watermark$ and $y.watermark$.

Next, we will discuss how RedT updates the watermark for each data item in node $\mathcal{N}_k$. Specifically, when replaying a redo log entry ent , the data node $\mathcal{N}_k$ will update the watermark for each data item x that is re-

quired to be updated by ent , setting $x.watermark$ to $\max\{x.watermark, ent.cts\}$. However, since each transaction can commit once over $\frac{3}{4}$ of the replicas per partition return success, some replicas may not have written the transaction's redo log, and therefore, the log can be discontinuous. To address this issue, before asynchronously replaying the logs, the data node $\mathcal{N}_k$ must retrieve logs from other replicas based on the partitions stored on the node to ensure that the local logs are complete. To minimize the amount of log retrieval, the remote node maintains a mark $head_k$, which represents the position in the circular buffer of the remote node up to which logs have been sent to $\mathcal{N}_k$. When $\mathcal{N}_k$ retrieves logs from the remote node, only the logs from $head_k$ to $tail$ need to be retrieved, and $head_k$ in the remote node is updated to the value of $tail$ afterward.

Discussion. To prevent read-only transactions from accessing excessively stale data, we enforce a threshold, $max_tolerant_staleness$, which limits the oldest accessible version for read-only transactions. If the timestamp difference between the readable version and the latest version exceeds $max_tolerant_staleness$, the read-only transaction aborts and re-executes using the non-optimized read-only mode. When $max_tolerant_staleness = 0$, read-only transactions must always read the most recent data, effectively disabling the read-only optimization. When $max_tolerant_staleness = infinity$, read-only transactions are required to always access the latest version. By incorporating $max_tolerant_staleness$, our read-only optimization can be applied to scenarios with less stringent data freshness requirements, such as social media applications (e.g., displaying comments). R3:4

6 Fault Tolerance

6.1 Node Fault Tolerance

In our design, a node can act as either a data node, an executor, or a coordinator. Therefore, when a node fails, we need to perform failure recovery for a loss of data nodes and coordinators/executors.

Failure recovery of data nodes. As opposed to Paxos/Raft, RedT follows a stricter condition in which as long as more than $\frac{3}{4}$ of the replicas for each partition are still alive, the transaction keeps running normally; otherwise, if less than $\frac{3}{4}$ of the replicas for any partition vote to commit within a pre-defined timeout, the coordinator directly enters the commit phase and notifies the replicas to abort. Upon failure, it is unnecessary for the replicas in the failed nodes to release the locks. Instead, RedT finds new data nodes, restores the failed replicas, and synchronizes the logs from the other replicas to the new replicas, following the logic of Paxos.

Failure recovery of executors. In each data center, there is an executor that is responsible for acquiring/releasing locks on all replicas operated by T in the same data center, and these locks are directly maintained with the replicas. In this way, the executor is stateless. For this reason, once the executor fails, the coordinator chooses another executor in the same data center to replace the failed executor and lets it manage the locks on the replicas.

Failure recovery of coordinators. In RedT, the coordinator maintains the operations and current status of the transaction T and is responsible for coordinating the execution of T . In the execution phase, the coordinator sends the operations of T to all executors. Once the coordinator fails, one executor is selected to act as the new coordinator. Because the status of T located in the failed coordinator is lost, we need to restore the status in the new coordinator. To do this, we adapt the mechanism used in TAPIR [73]. The new coordinator restores the status by collecting the status of every healthy replica. If more than $\frac{1}{2}$ of the replicas for each partition are still alive and have the same commit decisions, it means that the status is committed.

R1:3 Conversely, if more than $\frac{1}{2}$ of replicas for each partition are still alive but do not report a consistent commit decision, then the transaction status is determined as aborted. If more than $\frac{1}{4}$ of replicas have failed, the coordinator must wait for a sufficient number of replicas to recover before proceeding with status restoration.

The reason is as follows. The status of a transaction is committed only if more than $\frac{3}{4}$ of replicas for each partition vote to commit, meaning that at the worst case, $\frac{1}{4}$ of replicas vote to commit fail. In this way, $\frac{1}{2}$ ($\frac{3}{4} - \frac{1}{4} = \frac{1}{2}$) of replicas still vote to commit. Thus, by collecting the decisions from at least $\frac{1}{2}$ of replicas for each partition, the new coordinator can restore the status of the transaction. If more than $\frac{1}{4}$ of the replicas fail, the new coordinator needs to wait until partial replicas are recovered.

6.2 Data Center Fault Tolerance

When a minor portion of data centers fail, in the worst case, the coordinator and some executors of T , and some replicas accessed by T are lost. In this way, among the remaining healthy executors, one executor is selected to act as the new coordinator following the same logic discussed in Section 6.1. If the number of failed replicas does not exceed $\frac{1}{4}$ of replicas for each partition, as discussed in Section 6.1, the new coordinator does not need to restore the executor and replicas of T ; otherwise, when multiple data centers fail, causing more than $\frac{1}{4}$ of replicas for some partitions lost, the new

coordinator needs to wait until the replicas recover. To keep the system having enough healthy replicas, RedT follows the logic of Paxos to find new data nodes and restore the failed replicas asynchronously.

Discussion. RedT ensures that an inter-DC transaction can complete with only 2 network round trips, but in return, it employs a more strict requirement for fault tolerance, i.e., tolerating at most $\frac{1}{4}$ replica failures. Given that our primary focus is optimizing transaction performance in normal execution and that failures occur relatively infrequently, we consider this trade-off reasonable. By accommodating up to $\frac{1}{4}$ replica failures, RedT can recover from the simultaneous failure of both the coordinator and replicas, thus eliminating an additional round-trip to synchronize the coordinator state. For example, consider a transaction T where its replicas obtained more than $\frac{3}{4}$ commit votes, but subsequently fewer than $\frac{1}{4}$ of the replicas and the coordinator fail. When the new coordinator collects the status from the remaining replicas, it can still obtain more than $\frac{1}{2}$ commit votes from the remaining replicas, specifically $\frac{3}{4} - \frac{1}{4} = \frac{1}{2}$.

R1:3

7 Correctness

To prove correctness, we show that RedT maintains the following three properties 1) isolation, 2) atomicity, and 3) durability given up to $\frac{1}{4}$ of replicas failure in each partition. Further, we provide the TLA+ specification [30] for RedT, demonstrating that RedT maintains data consistency even when crash failures occur.

7.1 Isolation

7.1.1 RedT-SER satisfies serializability.

We first prove that RedT-SER can ensure serializability. In RedT, any two conflicting transaction, T_i and T_j , that violate the serializability cannot commit both. That is, T_i and T_j access the same data item x , and satisfy the order $T_i \rightarrow T_j$ and $T_j \rightarrow \dots \rightarrow T_i$. As discussed, in RedT, a transaction can commit only after acquiring locks on more than $\frac{3}{4}$ of replicas for each partition. T_i and T_j cannot both acquire locks on more than $\frac{3}{4}$ of replicas. Therefore, with the order $T_i \rightarrow T_j$, T_j can acquire the lock on more than $\frac{3}{4}$ of replicas only after T_i commits and releases its locks. By using RedT-SER, the order $T_j \rightarrow \dots \rightarrow T_i$ cannot be held because T_i cannot acquire other locks after releasing the locks. Even if less than $\frac{1}{4}$ of the replicas fail, more than $\frac{1}{2}$ of the replicas still maintain the correct order. Because we

can compute the transaction status based on the replicas, failures of the coordinator and the executors do not affect the serializable scheduling of transactions.

7.1.2 RedT-SI satisfies SI.

We then prove that RedT-SI satisfies the two properties of SI for a transaction T (SI1-2).

SI1: Transaction T can always read data items from a consistent snapshot.

There are only two possible scenarios if transaction T does not access data items in a consistent snapshot. Let us denote the snapshot that a transaction is supposed to read as $T.snapshot$. 1) First, transaction T may read data item x in a newer snapshot than $T.snapshot$. However, RedT-SI requires transaction T to always read the latest version valid as of the logical time T started, or as of the watermark, as discussed in Section 5. Thus, this case is infeasible under RedT-SI. 2) Second, transaction T may read data item x in an older snapshot than $T.snapshot$. This situation arises when the new version of data item x required by T has not been created yet. Consequently, T will encounter a lock on x and must abort. As both cases are infeasible under RedT-SI, we conclude that RedT-SI ensures SI1.

SI2: Transaction T can only commit if, at its commit time, the data that T aims to update has not been modified by any other concurrent transactions.

According to the function *Write* in Algorithm 4, transaction T will validate whether there exist the situations described in SI2 by comparing the *wts* of data items with the snapshot timestamp of T . If such a situation is identified, transaction T will abort, and therefore, RedT-SI satisfies property SI2.

7.2 Atomicity and Durability.

Atomicity. For atomicity, we must guarantee that T is finally committed if T enters the commit phase. Barring failures, the coordinator would commit T normally; otherwise, the remaining executor restores a new coordinator, and the new coordinator also computes the same transaction status following the logic in Section 6.1 and then commits T normally.

Durability. For any committed transaction T , upon a failure, the original/new coordinators obtain the same status of T and let executors do the same writes to the replicas. Note that for the replicas that vote to abort and the coordinator decides to commit, the executor modifies the status of the sub-transaction and writes a redo log with a commit timestamp. Then, the asynchronous thread replays the redo logs according to the

order of the commit timestamp, thus maintaining the original orders.

7.3 TLA+ specification

We provide TLA+ specifications [30] to formally verify RedT's correctness regarding atomicity, durability, SI and SER isolation levels. In particular, we provide three TLA+ specifications: `redtatom.tla`, verifying atomicity and durability; `RedTSER.tla`, verifying that RedT-SER correctly ensures SER; and `RedTSI.tla`, verifying that RedT-SI satisfies SI. We utilize the TLC model checker [65] to perform 3 test runs for each specification file and confirm that no invariants are violated. All three TLA+ specifications and their corresponding execution logs are available in the GitHub repository [50] and technical report [3].

R3:1

8 Evaluation

In this section, we compare RedT-SER's performance with six distributed transaction processing protocols under the serializable isolation level. These protocols include 2PC-Paxos, TAPIR, EP-Paxos, MDCC, NCC, and Calvin[57], a deterministic concurrency control protocol. Subsequently, we compare the performance of RedT-SI with 2PC-Paxos, TAPIR, EP-Paxos, MDCC, and PostSI under snapshot isolation. We first describe the experimental setup and then present the results in terms of throughput, latency, and abort rate.

R3:3

8.1 Experimental Setup

We conduct the experiments over 8 or 16 machines of an RDMA-capable EDR cluster. Each machine is equipped with one Intel(R) Xeon(R) Gold 5220 CPU @ 2.20GHz (18 cores×2 HT) processor, 128GB RAM, and one ConnectX-5 EDR 100Gb/s InfiniBand MT27800. All experiments are conducted over the following two benchmarks.

YCSB [12] is a comprehensive benchmark that simulates large-scale Internet applications. Its dataset contains a single 10-column relation, in which each data item occupies 1KB. The relation is then horizontally partitioned, and partitions are distributed to data nodes in a round-robin manner. In our evaluation, each data node maintains 4 million data items, resulting in a 4GB storage space per data node. Each transaction of the workloads is set to have a fixed number of 10 read-/write operations that access data items following the Zipfian distribution. A larger Zipfian value results in a

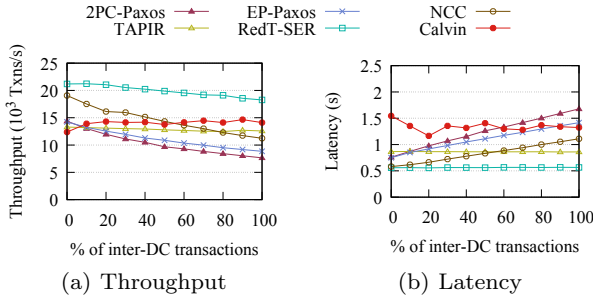


Fig. 10 Impact of inter-DC transaction ratio **under YCSB - SER**

higher contention workload. Unless otherwise specified, we set the skew factor to 0.2, and the write ratio to 0.8, i.e., there are 20% read-only transactions and 80% read-write transactions.

TPC-C [58] is a popular OLTP benchmark simulating a warehouse order processing application. Its dataset is composed of 9 relations, in which each warehouse has 100MB of data. By default, we set the number of warehouses per node to 32. TPC-C contains 5 types of transactions, among which Payment, New-order, and Delivery are read-write transactions, Stock-level and Order-status are read-only transactions. Following the previous works [24, 69], we only evaluate Payment and New-order transactions in the experiments and omit the other three types which are local transactions.

By default, we set up 4 data centers, with 2 nodes in each. To support the data center fault tolerance, we set the replication factor to 3, and replicas of each partition are placed in 3 distinct data centers. As we mainly focus on the processing of inter-DC transactions, unless otherwise specified, the ratio of inter-DC transactions is set to 1 in YCSB and 0.5 in TPC-C. Each transaction is confined to 2 nodes. We simulate the inter-DC round-trip network delay as 40 ms, which is reasonable. For example, RTT from New York to Dallas is 40ms [34]. Furthermore, read-only optimization is enabled by default for both RedT-SER and RedT-SI, except for Section 8.6. To ensure fair comparisons, all protocols are implemented and evaluated in Deneva [24], which is open-sourced by MIT. For reference, we release the source code of implementations via [49].

8.2 Evaluation under SER

8.2.1 Impact of Inter-DC Transaction Ratio

We first evaluate the performance by varying the percentage of inter-DC transactions in Figure 10.

RedT-SER demonstrates superior performance, outperforming NCC, Calvin, TAPIR, EP-Paxos, and 2PC-

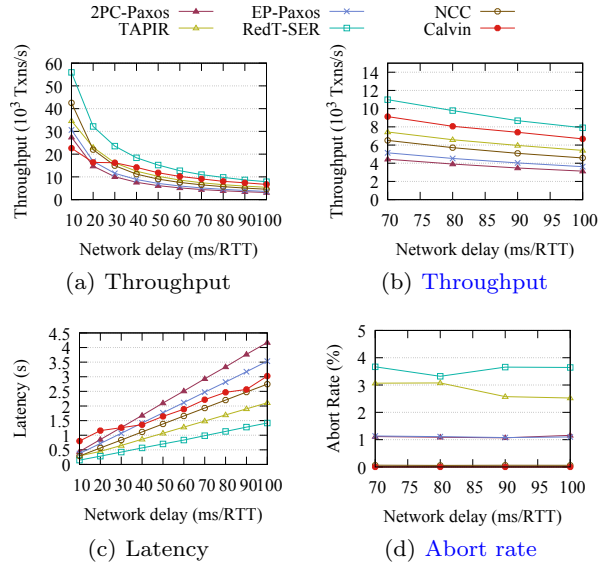


Fig. 11 Impact of inter-DC network delay **under YCSB - SER**

Paxos, primarily due to its minimal inter-DC round-trips and read-only transaction optimization. As expected, EP-Paxos surpasses 2PC-Paxos by eliminating the phase barrier. In the absence of inter-DC transactions, NCC's performance is comparable to RedT-SER, as both require two round-trips in this scenario. RedT-SER, NCC, 2PC-Paxos, and EP-Paxos exhibit a performance decline with increasing inter-DC transaction ratios, stemming from the greater inter-DC communication overhead. For RedT-SER, increased inter-DC transactions reduce the effectiveness of read-only optimizations. Conversely, TAPIR and Calvin maintain relatively stable performance. TAPIR's stability is attributed to its consistent inter-DC round-trip count for both transaction types. Calvin's deterministic nature allows for batched sub-transaction distribution, bypassing 2PC/Paxos for sub-transactions and requiring only two communication rounds: batch distribution and inter-node read-write set synchronization.

8.2.2 Impact of Inter-DC Network Delay

To illustrate the adverse effects of high-latency inter-DC networks, we use the traffic control emulator *tc* provided by Linux to simulate a fixed inter-DC RTT ranging from 10ms to 100ms. Figure 11 demonstrates the throughput and latency over YCSB. The throughput of all protocols sharply degrades with increasing delay as inter-DC networks become a bottleneck. For instance, 2PC-Paxos throughput drops by 87% from 10ms to 100ms delay. This degradation is primarily attributed to network latency, not abort rates, which remain below 5% (Figure 11(d)). RedT-SER, with fewer inter-DC

R1:4

R3:3

R1:5

R1:4

Table 3 An experimental comparison of all protocols in terms of inter-DC communications ($\mathcal{C}$) and round-trips to process read-write inter-DC committed transactions.

Protocol	Phase	$\mathcal{C}$		round-trips	
		avg #	max #	avg #	max #
2PC-Paxos	exe	2.00	2	1	1
	prep	13.49	14	2.97	3
	commit	9.60	10	2	2
EP-Paxos	exe	13.59	14	3	3
	commit	9.60	10	2	2
MDCC [†]	commit	7.43 (9.24)	20 (20)	1.02 (1.38)	3 (3)
TAPIR G-PAC	exe	7.33	10	1	1
	prep	7.01	10	1	1
	commit	7.34	10	1	1
NCC	exe	8.06	10	2	2
	commit	8.04	6	2	2
Calvin	seq	0.001	2	1	1
	exe	0	0	0	0
PostSI	exe	2.00	2	1	1
	prep	11.55	16	2.97	3
	commit	8.04	10	2	2
RedT	exe	5.87	10	1	1
	commit	5.87	6	1	1

[†] For MDCC, we also report the results under high contention (i.e., the skew factor is set to 0.8) in parenthesis. For all other protocols, the skew factor is set to 0.2 by default.

round-trips, outperforms 2PC-Paxos, EP-Paxos, NCC, and TAPIR in both throughput and latency across all tested delays. Calvin exhibits less sensitivity to delay due to batch processing, but RedT-SER still achieves superior performance, maintaining a 20% throughput advantage even at 100ms latency (Figure 11(b)), primarily due to read-only optimizations.

8.2.3 Impact of Inter-DC Network Communications

We then use *tc* to simulate a jittered inter-DC RTT (10ms to 100ms) to evaluate the impact of inter-DB network communications. As shown in Table 3, its result is consistent with our theoretical analysis in Tables 1 and 2. Except for Calvin, RedT exhibits the lowest average/maximum communications per transaction (11.74/12). In contrast, 2PC-Paxos, EP-Paxos, TAPIR, NCC, and PostSI require significantly more communications (25.09/26, 23.19/24, 21.68/30, 16.10/20, and 21.59/28, respectively). MDCC's communication numbers are discussed in Section 8.2.4.

To demonstrate the benefits of reduced communication, Figure 12 presents throughput and latency as partitions accessed per transaction increase under jittered network conditions (10ms-100ms). In this experiment, 16 machines are used to form 4 data centers, and primary replicas accessed by each transaction are confined

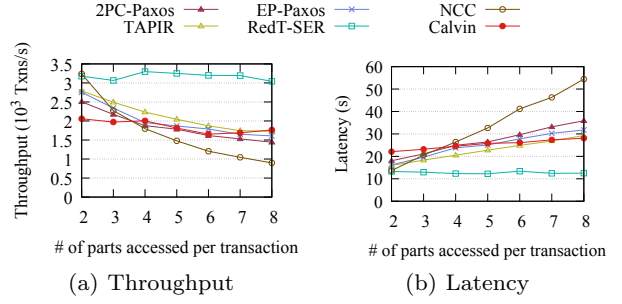


Fig. 12 Impact of partitions accessed under YCSB - SER

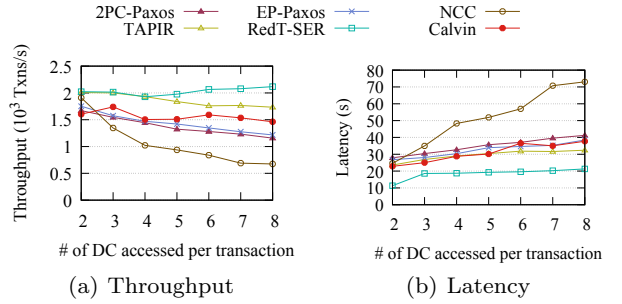


Fig. 13 Impact of data centers accessed under YCSB - SER

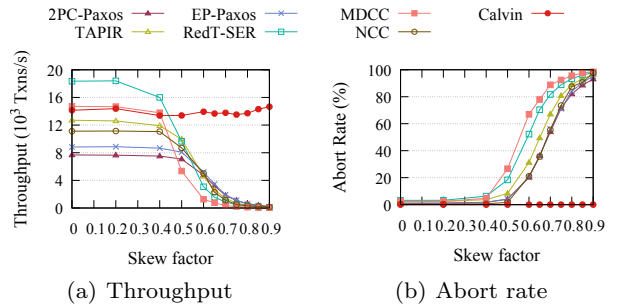
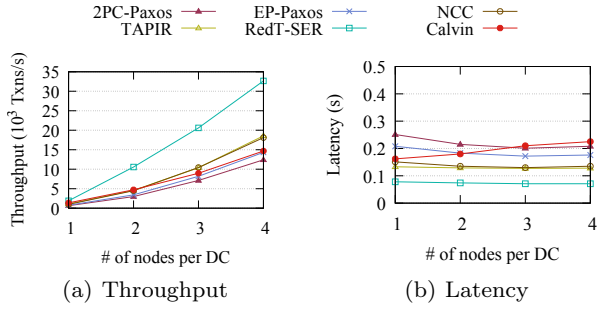
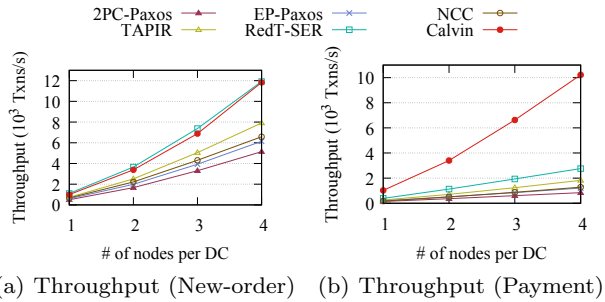


Fig. 14 Impact of contention level under YCSB - SER

to 2 data centers. With partitions accessed rising from 2 to 8, protocols except RedT-SER and Calvin experience increased inter-DC communication. Throughput for 2PC-Paxos, EP-Paxos, TAPIR, and NCC drops by 42%, 41%, 38%, and 73% respectively, while latency increases by 99%, 96%, 83%, and 298%. Figure 13 shows that increasing the number of data centers involved per transaction degrades performance across protocols due to increased communication. However, RedT is less exposed to the unstable inter-DC network by decomposing transactions in terms of data center granularity. Consequently, 2PC-Paxos, EP-Paxos, and TAPIR exhibit performance drops as the number of involved data centers increases from 2 to 8, while Calvin and RedT-SER maintain stable performance.

8.2.4 Impact of Contention Level

At low contention (skew factor < 0.4), protocols exhibit low abort rates and stable performance, primarily dic-

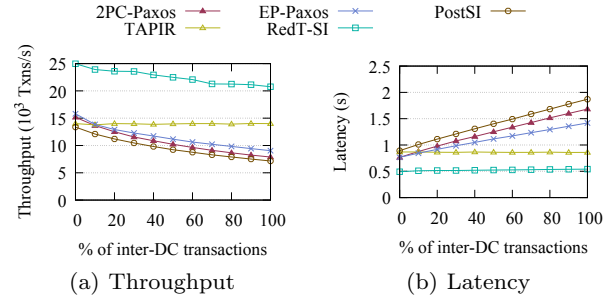
**Fig. 15** Scalability under YCSB - SER**Fig. 16** Scalability under TPC-C - SER

tated by inter-DC round-trips. RedT-SER and MDCC, requiring the fewest round-trips, perform best.

We evaluate the performance by varying the contention level over YCSB in Figure 14. Under low contention levels (i.e., skew factor < 0.4), all protocols exhibit low abort rates and stable performance. In this situation, the number of inter-DC round-trips almost dominates the performance. RedT-SER and MDCC, requiring the fewest round-trips, perform best. In moderate contention ($0.4 \leq \text{skew factor} < 0.6$), abort rates sharply increase for most protocols except Calvin. MDCC's performance degrades most significantly due to both abort overhead and additional round-trips for conflict resolution (Table 3). Under high contention (skew factor ≥ 0.6), performance severely degrades across all protocols except Calvin. This can be attributed to the fact that under high contention scenarios, the concurrency control algorithm becomes the dominant factor affecting performance. Calvin employs deterministic concurrency control to reduce transaction aborts but relies on several preconditions, such as pre-knowledge of read/write sets and batch execution requirements. In contrast, RedT and other protocols do not impose additional constraints on transactions, leading to similarly high abort rates under high contention.

8.2.5 System Scalability

We study the system scalability over both YCSB and TPC-C, by varying the number of nodes in a data center from 1 to 4. Figure 15 shows near-linear throughput

**Fig. 17** Impact of inter-DC transaction ratio under YCSB - SI

scaling and stable latency for all protocols on YCSB. We plot the results over YCSB in Figure 15. As we can see, with a growing number of nodes, RedT-SER consistently achieves the highest throughput, with its performance advantage amplified at a larger scale. Similar linear scaling is observed for TPC-C New-order and Payment (Figures 16(a), 16(b)). However, due to the inherently high contention in TPC-C Payment and New-order workloads, Calvin demonstrates the best performance, succeeded by RedT-SER.

8.3 Evaluation under SI

We then compared RedT-SI with other protocols, including PostSI, under the snapshot isolation level. Figure 17 shows RedT-SI outperforming other protocols when varying inter-DC transaction percentages, primarily due to its minimal inter-DC round-trips. Latency sensitivity under SI was also evaluated, following the setup in Section 8.2.2. As shown in Figure 18, RedT-SI again demonstrates superior performance, with all protocols exhibiting expected performance degradation as network latency increases, consistent with findings in Section 8.2.2.

We assess the influence of the number of inter-DC network communications in scenarios with fluctuating latency, and plot the results in Figure 19 and Figure 20. RedT-SI is less affected by network fluctuations, consistent with serializable isolation results, and attributed to its minimal inter-DC communication. Similarly, contention experiments (Figure 21) display a trend similar to that under the serializable isolation level. RedT-SI's linear scalability is demonstrated in Figures 22 and 23.

8.4 Impact of version array length

To evaluate the impact of version array length, we increase the version array length from 1 to 32 under both low-contention (skew factor = 0.2) and high-contention

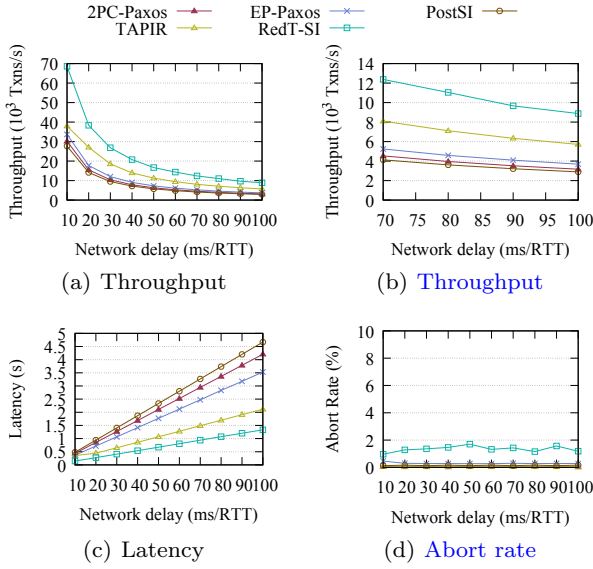


Fig. 18 Impact of inter-DC network delay under YCSB - SI

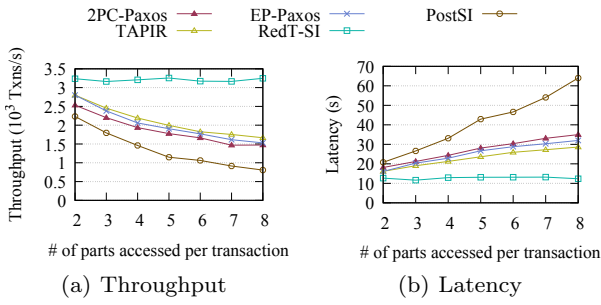


Fig. 19 Impact of partitions accessed under YCSB - SI

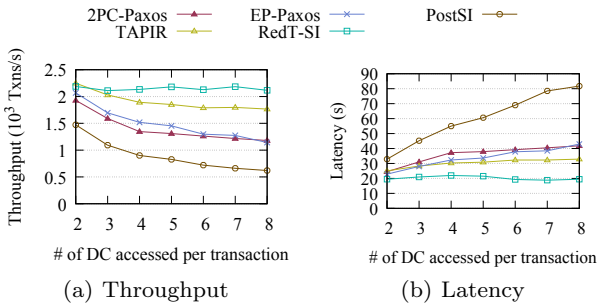


Fig. 20 Impact of data centers accessed under YCSB - SI

(skew factor = 0.7) scenarios. As shown in Figure 24(a), the performance remains stable when the version array length increases. This is because the version array length does not dominate performance in inter-DC transaction processing. In low-contention scenarios, the primary bottleneck is network latency introduced by inter-DC deployments, while in high-contention scenarios, the primary bottleneck is transaction aborts caused

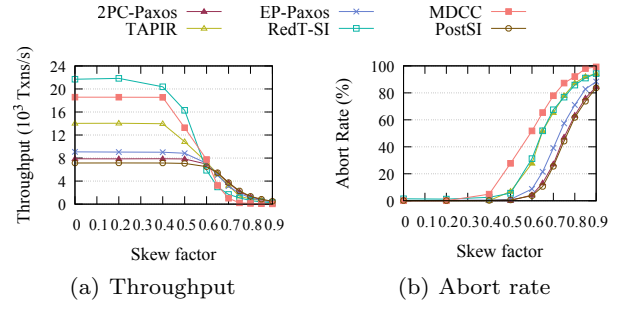


Fig. 21 Impact of contention level under YCSB - SI

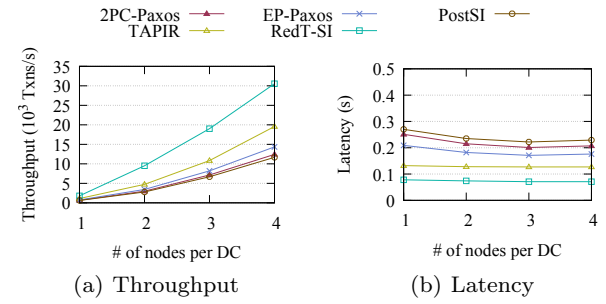


Fig. 22 Scalability under YCSB - SI

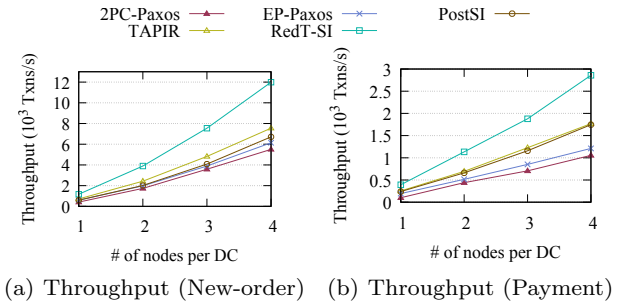


Fig. 23 Scalability under TPC-C - SI

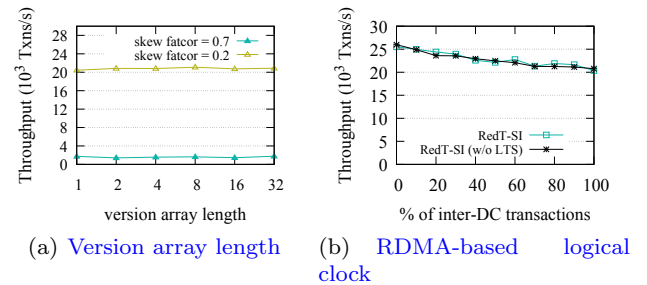


Fig. 24 Impact of version array length and RDMA-based logical clock under YCSB

by write-write conflicts, which is orthogonal to the version array length.

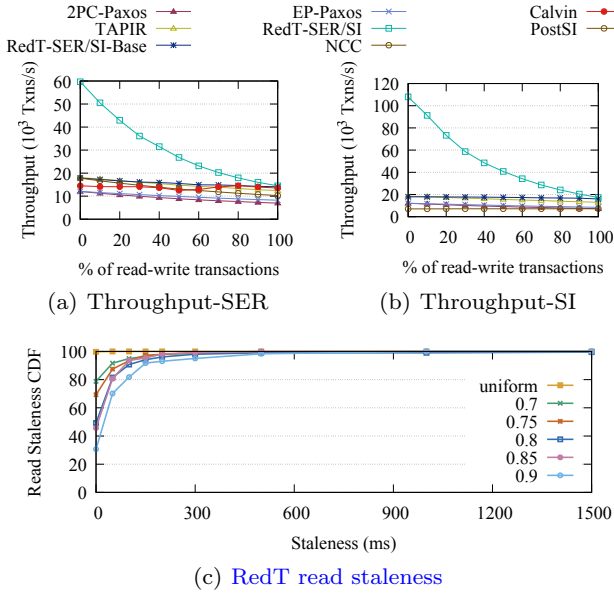


Fig. 25 Impact of read-only optimization under YCSB

8.5 Impact of RDMA-based logical clock

R1:1 We evaluate the overhead of the RDMA-based logical clock mechanism. Specifically, we compared two configurations: RedT-SI, which employs the RDMA-based logical clock mechanism, and RedT-SI (w/o LTS), a variant that replaces the RDMA-based logical clock service with local physical clocks. As shown in Figure 24(b), the performance difference between RedT-SI and RedT-SI (w/o LTS) is negligible. This is because the primary performance bottleneck stems from the inherent latency of inter-DC networks, rather than the additional RDMA one-sided verbs associated with the logical clock mechanism.

8.6 Impact of read-only optimization

We evaluate the effectiveness of read-only optimization for both RedT-SER and RedT-SI by varying the write ratio using the YCSB benchmark. Figure 25(a) reports the throughput under serializable isolation level. RedT-SER outperforms RedT-SER without read-only optimization (RedT-SER-Base), demonstrating the benefit of the optimization. This advantage is attributed to read-only transactions accessing local data and minimizing inter-DC round-trips at lower write ratios. As the write ratio increases, the optimization's impact diminishes, and RedT-SER's performance approaches RedT-SER-Base. Consistent performance trends are observed for RedT-SI under snapshot isolation (Figure 25(b)).

R3:2 As shown in Figure 25(c), with increasing skewness, increasing skewness leads to older versions being read

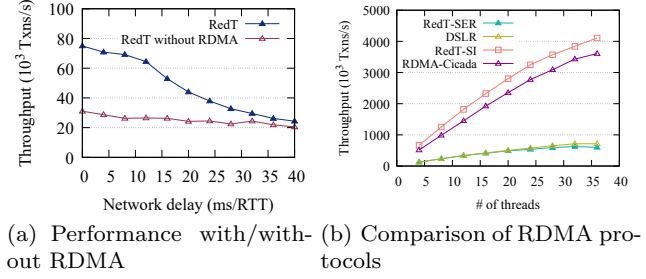


Fig. 26 Impact of RDMA under YCSB

by read-only transactions. However, in 90% of cases, the staleness remains within 150ms. This relatively low staleness is primarily due to our system being built on an in-memory database, where log replay is highly efficient, enabling read-only transactions to be able to access relatively up-to-date data.

8.7 Impact of RDMA

We compare the performance of RedT and RedT without RDMA, which replaces RDMA networks with TCP/IP networks and is similar to the idea of multi-level 2PC [42]. The results are given in Figure 26(a). For low or moderate inter-DC network delays, i.e., when the data centers involved are not so far away, RedT significantly outperforms RedT without RDMA. For high inter-DC network delays, however, their throughput converges, as the contribution of RDMA grows negligible compared to the bottlenecked inter-DC networks. As a result, RDMA is shown to be effective when the replicas are deployed in nearby data centers. Note that deploying inter-DC databases with sub-30ms network latency is common in practice. For example, PingCap adopts a deployment model with two data centers in Beijing and one data center in Xi'an [25], where the inter-DC network latency is approximately 20ms. In such typical deployment scenarios, utilizing RDMA can still achieve up to 80% performance gains. Furthermore, in a geo-distributed deployment where inter-DC network latency exceeds 30ms, most transactions can still be processed within a region, as countrywide distributed transactions are relatively infrequent.

To show the efficiency of our RDMA-based implementation, we have included an evaluation to compare RedT-SER and RedT-SI with other RDMA-based transaction protocols in Figure 26(b). Specifically, RedT-SER is compared to DSLR [68], a state-of-the-art lock-based RDMA protocol, under the serializable isolation level. Since no RDMA-based OCC protocol exists at the snapshot isolation level, we selected RDMA-Cicada, an RDMA-based OCC protocol from RCBench [75]. To en-

sure a fair comparison, we lowered the isolation level of RDMA-Cicada in RCBench to SI, thus evaluating both RedT-SI and RDMA-Cicada under SI isolation. Given that DSLR and RDMA-Cicada lack inherent fault tolerance, we disabled the fault tolerance mechanisms in RedT-SER and RedT-SI to facilitate a more fair performance comparison with DSLR and RDMA-Cicada. As shown in Figure 26(b), RedT-SI outperforms RDMA-Cicada. This is expected because, compared to RedT-SI, RDMA-Cicada employs more complex logic and requires additional RDMA one-sided operations, leading to lower throughput compared to RedT-SI. Regarding RedT-SER and DSLR, the performance of RedT-SER is generally comparable to that of DSLR.

9 Related Work

Substantial efforts have focused on transforming distributed transaction processing into localized transactions. These efforts design the static or dynamic data partitioning schemes [18, 44, 48, 15, 71, 59, 54, 51, 1, 2] to re-locate all data partitions accessed by a transaction onto a single node. However, static partitioning is effective only when the optimal data placement is known a priori and remains constant. Dynamic partitioning schemes require migrating the replicas accessed by a distributed transaction to a single node before execution, resulting in data migration overhead. As a remedy, an alternative method [39] is to arrange one super node that takes all replicas of the whole data partitions. All distributed transactions in a batch are then assigned to this super node, and before any of them starts to execute, all involved secondary replicas in this node are promoted as primary replicas so that they are transformed to local transactions [39]. Nevertheless, for this method, the super node may become the bottleneck when the workloads are skewed or the size of the whole data partitions is prohibitively huge.

As opposed to eliminating distributed transactions, quite a few works attempt to optimize distributed transaction processing by reducing the number of network round-trips in conventional networks [67]. **Spanner** [4, 13, 14], widely recognized as the first geo-distributed database, employs the 2PC-Paxos protocol (as detailed in Section 2.1) to manage distributed transactions. Therefore, Spanner does not explicitly optimize transaction performance under the inter-DC network, resulting in 6 inter-DC round trips per distributed transaction. Multi-level 2PC [42] reduces the expensive inter-DC communication by layering participants, but at the cost of a higher intra-DC coordination overhead. Early Prepare [53] further merges the execution and prepare phase, where each participant enters the prepare phase after

the read/write execution without waiting for the notification from the coordinator. Building upon Spanner's architecture, Carousel [66] merges the execution phase and prepare phase, thus reducing the inter-DC round trips from 6 to 4. Parallel Commit [60] turns some synchronous round-trips to asynchronous and is specially designed for MVCC-based write intents in CockroachDB. A series of classic algorithms are deterministic-based [57, 38, 47, 21, 46, 20, 19, 35] with several assumptions like the requirement of prior knowledge of transaction operations. Commutativity is a different constraint from determinism, with which MDCC [29] can commit in a single synchronous round-trip. Without commutativity of operations and in case of concurrent updates, however, primary replicas must step in to resolve conflicts in two additional round-trips. TAPIR [73] and G-PAC [40] attempt to unify the commitment and consensus protocols in a single framework. Specifically, instead of synchronizing log replications from primary replicas to secondary replicas, in the prepare/commit phase, the coordinator directly sends log replications to secondary replicas, reducing the number of round-trips. Specifically, NOCS [37] introduces a version-clock mechanism to track the serializability order of read and write operations, which improves data freshness for read-only transactions. Building upon this, NOC-NOC proposes a dual views mechanism that processes read and write operations in a lock-free manner, further improving transaction performance. However, they are orthogonal to RedT, as they primarily focus on intra-DC transaction optimizations and do not explicitly address inter-DC transaction performance. As a result, both NOCS and NOC-NOC require a similar number of network round trips as traditional 2PC for inter-DC transactions. Compared with these works, RedT concentrates on the reduced number of not only network round-trips, but also communication per round trip, and proposes a pre-write-log mechanism that is able to overlap the prepare and execution phase and eliminate redundant synchronization.

It has been a hotspot to use RDMA to optimize the distributed transaction processing. Foundational studies [9, 79, 64, 27, 70] provide insights into effective RDMA utilization, including comparisons of one-sided and two-sided verbs, request handling optimizations, and system architecture designs. Leveraging its advantages, RDMA is widely adopted to enhance distributed transaction systems. Numerous works utilize RDMA for high availability through replication [72, 17, 11], monotonic global timestamp generation [52], and concurrency control algorithms, encompassing OCC-based [62, 16] and 2PL-based [68, 6, 63] approaches. However, these solutions primarily focus on single data center deployments. RedT

R1.6

R1:6

R3:5

presents a first-of-its-kind attempt to support inter-data-center transaction processing and data center fault tolerance simultaneously.

10 Conclusions

This paper introduces RedT, a novel distributed transaction processing protocol designed for concurrent inter-data-center transactions and data center fault tolerance. RedT extends the 2PC protocol by decomposing transactions into data-center-level sub-transactions and employing a pre-write-log mechanism to eliminate log synchronization during the prepare phase. Specifically tailored for high-latency and unreliable inter-data-center networks, RedT enhances performance by minimizing inter-data-center round-trips. Extensive evaluations using YCSB and TPC-C benchmarks demonstrate RedT's superiority over state-of-the-art protocols, achieving a $1.80\times$ throughput improvement, a $0.41\times$ latency reduction, and more stable performance in diverse scenarios.

11 Acknowledgement

The work is supported by the National Natural Science Foundation of China under Grant No. 61972403, and Public Computing Cloud, Renmin University of China. We sincerely thank Qian Zhang and Jingyao Li for their early contributions.

References

1. Abebe, M., Glasbergen, B., Daudjee, K.: Dynamast: Adaptive dynamic mastering for replicated systems. In: 36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020, pp. 1381–1392. IEEE (2020)
2. Abebe, M., Glasbergen, B., Daudjee, K.: Morphosys: Automatic physical design metamorphosis for distributed database systems. *Proc. VLDB Endow.* **13**(13), 3573–3587 (2020)
3. et al., H.Z.: <https://github.com/dbiir/RedT/blob/RedT/RedT.pdf> (2024)
4. Bacon, D.F., Bales, N., Bruno, N., Cooper, B.F., Dickinson, A., Fikes, A., Fraser, C., Gubarev, A., Joshi, M., Kogan, E., Lloyd, A., Melnik, S., Rao, R., Shue, D., Taylor, C., van der Holst, M., Woodford, D.: Spanner: Becoming a sql system. In: Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD '17, p. 331–343. Association for Computing Machinery, New York, NY, USA (2017)
5. Barshai, V., Chan, Y., Lu, H., Sohal, S., et al.: Delivering continuity and extreme capacity with the IBM DB2 pureScale feature. IBM Redbooks (2012)
6. Barthels, C., Müller, I., Taranov, K., Alonso, G., Hoefler, T.: Strong consistency is not hard to get: Two-phase locking and two-phase commit on thousands of cores. *Proc. VLDB Endow.* **12**(13), 2325–2338 (2019)
7. Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., O'Neil, P.: A critique of ansi sql isolation levels. *SIGMOD Rec.* **24**(2), 1–10 (1995)
8. Bernstein, P.A., Goodman, N.: Concurrency control in distributed database systems. *ACM Comput. Surv.* **13**(2), 185–221 (1981)
9. Binnig, C., Crotty, A., Galakatos, A., Kraska, T., Zamanian, E.: The end of slow networks: It's time for a redesign. *Proc. VLDB Endow.* **9**(7), 528–539 (2016)
10. Cao, W., Zhang, Y., Yang, X., Li, F., Wang, S., Hu, Q., Cheng, X., Chen, Z., Liu, Z., Fang, J., Wang, B., Wang, Y., Sun, H., Yang, Z., Cheng, Z., Chen, S., Wu, J., Hu, W., Zhao, J., Gao, Y., Cai, S., Zhang, Y., Tong, J.: PolarDB serverless: A cloud native database for disaggregated data centers. In: Proceedings of the 2021 International Conference on Management of Data, SIGMOD '21, pp. 2477–2489. Association for Computing Machinery, New York, NY, USA (2021)
11. Chen, Y., Wei, X., Shi, J., Chen, R., Chen, H.: Fast and general distributed transactions using rdma and htm. In: Proceedings of the Eleventh European Conference on Computer Systems, EuroSys '16. Association for Computing Machinery, New York, NY, USA (2016)
12. Cooper, B.F., Silberstein, A., Tam, E., Ramakrishnan, R., Sears, R.: Benchmarking cloud serving systems with ycsb. In: Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC '10, p. 143–154. Association for Computing Machinery, New York, NY, USA (2010)
13. Corbett, J.C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J., Ghemawat, S., Gubarev, A., Heiser, C., Hochschild, P., Hsieh, W., Kanthak, S., Kogan, E., Li, H., Lloyd, A., Melnik, S., Mwaura, D., Nagle, D., Quinlan, S., Rao, R., Rolig, L., Saito, Y., Szymaniak, M., Taylor, C., Wang, R., Woodford, D.: Spanner: Google's Globally-Distributed database. In: 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12), pp. 261–264. USENIX Association, Hollywood, CA (2012)
14. Corbett, J.C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J.J., Ghemawat, S., Gubarev, A., Heiser, C., Hochschild, P., Hsieh, W., Kanthak, S., Kogan, E., Li, H., Lloyd, A., Melnik, S., Mwaura, D., Nagle, D., Quinlan, S., Rao, R., Rolig, L., Saito, Y., Szymaniak, M., Taylor, C., Wang, R., Woodford, D.: Spanner: Google's globally distributed database. *ACM Trans. Comput. Syst.* **31**(3) (2013)
15. Curino, C., Jones, E., Zhang, Y., Madden, S.: Schism: A workload-driven approach to database replication and partitioning. *Proc. VLDB Endow.* **3**(1-2), 48–57 (2010)
16. Dragojević, A., Narayanan, D., Castro, M., Hodson, O.: FaRM: Fast remote memory. In: 11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14), pp. 401–414. USENIX Association, Seattle, WA (2014)
17. Dragojević, A., Narayanan, D., Nightingale, E.B., Renzelmann, M., Shamis, A., Badam, A., Castro, M.: No Compromises: Distributed Transactions with Consistency, Availability, and Performance, pp. 54–70. Association for Computing Machinery, New York, NY, USA (2015)
18. Elmore, A.J., Arora, V., Taft, R., Pavlo, A., Agrawal, D., El Abbadi, A.: Squall: Fine-grained live reconfiguration for partitioned main memory databases. In: Proceedings

- of the 2015 ACM SIGMOD International Conference on Management of Data, SIGMOD '15, p. 299–313. Association for Computing Machinery, New York, NY, USA (2015)
19. Faleiro, J.M., Abadi, D.J.: Rethinking serializable multi-version concurrency control. *Proc. VLDB Endow.* **8**(11), 1190–1201 (2015)
 20. Faleiro, J.M., Abadi, D.J., Hellerstein, J.M.: High performance transactions via early write visibility. *Proc. VLDB Endow.* **10**(5), 613–624 (2017)
 21. Faleiro, J.M., Thomson, A., Abadi, D.J.: Lazy evaluation of transactions in database systems. In: Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, SIGMOD '14, p. 15–26. Association for Computing Machinery, New York, NY, USA (2014)
 22. Gao, Y., Li, Q., Tang, L., Xi, Y., Zhang, P., Peng, W., Li, B., Wu, Y., Liu, S., Yan, L., Feng, F., Zhuang, Y., Liu, F., Liu, P., Liu, X., Wu, Z., Wu, J., Cao, Z., Tian, C., Wu, J., Zhu, J., Wang, H., Cai, D., Wu, J.: When cloud storage meets RDMA. In: 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21), pp. 519–533. USENIX Association (2021)
 23. Guo, C., Wu, H., Deng, Z., Soni, G., Ye, J., Padhye, J., Lipshteyn, M.: Rdma over commodity ethernet at scale. In: Proceedings of the 2016 ACM SIGCOMM Conference, SIGCOMM '16, p. 202–215. Association for Computing Machinery, New York, NY, USA (2016)
 24. Harding, R., Van Aken, D., Pavlo, A., Stonebraker, M.: An evaluation of distributed concurrency control. *Proc. VLDB Endow.* **10**(5), 553–564 (2017)
 25. Inc, P.: <https://docs-download.pingcap.com/pdf/tidb-v5.1-zh-manual.pdf> (2024)
 26. Kalia, A., Kaminsky, M., Andersen, D.G.: Using rdma efficiently for key-value services. In: Proceedings of the 2014 ACM Conference on SIGCOMM, SIGCOMM '14, pp. 295–306. Association for Computing Machinery, New York, NY, USA (2014)
 27. Kalia, A., Kaminsky, M., Andersen, D.G.: Design guidelines for high performance RDMA systems. In: 2016 USENIX Annual Technical Conference (USENIX ATC 16), pp. 437–450. USENIX Association, Denver, CO (2016)
 28. Kalia, A., Kaminsky, M., Andersen, D.G.: FaSST: Fast, scalable and simple distributed transactions with Two-Sided (RDMA) datagram RPCs. In: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), pp. 185–201. USENIX Association, Savannah, GA (2016)
 29. Kraska, T., Pang, G., Franklin, M.J., Madden, S., Fekete, A.: Mdcc: Multi-data center consistency. In: Proceedings of the 8th ACM European Conference on Computer Systems, EuroSys '13, pp. 113–126. Association for Computing Machinery, New York, NY, USA (2013)
 30. Lamport, L.: The temporal logic of actions. *ACM Trans. Program. Lang. Syst.* **16**(3), 872–923 (1994)
 31. Lamport, L.: The part-time parliament. *ACM Trans. Comput. Syst.* **16**(2), 133–169 (1998)
 32. Lamport, L.: Fast paxos. *Distributed Computing* **19**, 79–103 (2006)
 33. Lamport, L.: Time, clocks, and the ordering of events in a distributed system, p. 179–196. Association for Computing Machinery, New York, NY, USA (2019)
 34. Latency, N.: <http://ipnetwork.windstream.net/> (2022)
 35. Lin, Y.S., Tsai, C., Lin, T.Y., Chang, Y.S., Wu, S.H.: Don't look back, look into the future: Prescient data partitioning and migration for deterministic database systems. In: Proceedings of the 2021 International Conference on Management of Data, SIGMOD '21, p. 1156–1168. Association for Computing Machinery, New York, NY, USA (2021)
 36. Lu, H., Mu, S., Sen, S., Lloyd, W.: Ncc: Natural concurrency control for strictly serializable datastores by avoiding the timestamp-inversion pitfall. In: USENIX Symposium on Operating Systems Design and Implementation (2023)
 37. Lu, H., Sen, S., Lloyd, W.: Performance-optimal read-only transactions. In: Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation, OSDI'20. USENIX Association, USA (2020)
 38. Lu, Y., Yu, X., Cao, L., Madden, S.: Aria: A fast and practical deterministic oltp database. *Proc. VLDB Endow.* **13**(12), 2047–2060 (2020)
 39. Lu, Y., Yu, X., Madden, S.: Star: Scaling transactions through asymmetric replication. *Proc. VLDB Endow.* **12**(11), 1316–1329 (2019)
 40. Maiyya, S., Nawab, F., Agrawal, D., Abbadi, A.E.: Unifying consensus and atomic commitment for effective cloud data management. *Proc. VLDB Endow.* **12**(5), 611–623 (2019)
 41. Mitchell, C., Geng, Y., Li, J.: Using One-Sided RDMA reads to build a fast, CPU-Efficient Key-Value store. In: 2013 USENIX Annual Technical Conference (USENIX ATC 13), pp. 103–114. USENIX Association, San Jose, CA (2013)
 42. Mohan, C., Lindsay, B., Obermarck, R.: Transaction management in the r* distributed database management system. *ACM Trans. Database Syst.* **11**(4), 378–396 (1986)
 43. Ongaro, D., Ousterhout, J.: In search of an understandable consensus algorithm. In: 2014 USENIX Annual Technical Conference (USENIX ATC 14), pp. 305–319. USENIX Association, Philadelphia, PA (2014)
 44. Pavlo, A., Curino, C., Zdonik, S.: Skew-aware automatic database partitioning in shared-nothing, parallel oltp systems. In: Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data, SIGMOD '12, p. 61–72. Association for Computing Machinery, New York, NY, USA (2012)
 45. Peng, D., Dabek, F.: Large-scale incremental processing using distributed transactions and notifications. In: Proceedings of the 9th USENIX Conference on Operating Systems Design and Implementation, OSDI'10, p. 251–264. USENIX Association, USA (2010)
 46. Qadah, T., Gupta, S., Sadoghi, M.: Q-store: Distributed, multi-partition transactions via queue-oriented execution and communication. In: EDBT, pp. 73–84 (2020)
 47. Qin, D., Brown, A.D., Goel, A.: Caracal: Contention management with deterministic concurrency control. In: Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, SOSP '21, p. 180–194. Association for Computing Machinery, New York, NY, USA (2021)
 48. Quamar, A., Kumar, K.A., Deshpande, A.: Sword: Scalable workload-aware data placement for transactional workloads. In: Proceedings of the 16th International Conference on Extending Database Technology, EDBT '13, p. 430–441. Association for Computing Machinery, New York, NY, USA (2013)
 49. RedT: <https://github.com/dbiir/RedT/> (2024)
 50. RedT-TLA+: <https://github.com/dbiir/RedT/tree/RedT/tla+/> (2025)
 51. Serafini, M., Taft, R., Elmore, A.J., Pavlo, A., Aboul-naga, A., Stonebraker, M.: Clay: Fine-grained adaptive

- partitioning for general database schemas. *Proc. VLDB Endow.* **10**(4), 445–456 (2016)
52. Shamis, A., Renzelmann, M., Novakovic, S., Chatzopoulos, G., Dragojević, A., Narayanan, D., Castro, M.: Fast general distributed transactions with opacity. In: *Proceedings of the 2019 International Conference on Management of Data, SIGMOD '19*, p. 433–448. Association for Computing Machinery, New York, NY, USA (2019)
 53. Stamos, J., Cristian, F.: A low-cost atomic commit protocol. In: *Proceedings Ninth Symposium on Reliable Distributed Systems*, pp. 66–75 (1990)
 54. Taft, R., Mansour, E., Serafini, M., Duggan, J., Elmore, A.J., Abounaga, A., Pavlo, A., Stonebraker, M.: E-store: Fine-grained elastic partitioning for distributed transaction processing systems. *Proc. VLDB Endow.* **8**(3), 245–256 (2014)
 55. Taft, R., Sharif, I., Matei, A., VanBenschoten, N., Lewis, J., Grieger, T., Niemi, K., Woods, A., Birzin, A., Poss, R., Bardea, P., Ranade, A., Darnell, B., Gruneir, B., Jaffray, J., Zhang, L., Mattis, P.: Cockroachdb: The resilient geo-distributed sql database. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, SIGMOD '20*, p. 1493–1509. Association for Computing Machinery, New York, NY, USA (2020)
 56. Thomas, R.H.: A majority consensus approach to concurrency control for multiple copy databases. *ACM Trans. Database Syst.* **4**(2), 180–209 (1979)
 57. Thomson, A., Diamond, T., Weng, S.C., Ren, K., Shao, P., Abadi, D.J.: Calvin: Fast distributed transactions for partitioned database systems. In: *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data, SIGMOD '12*, p. 1–12. Association for Computing Machinery, New York, NY, USA (2012)
 58. TPC-C: <http://www.tpc.org/tpcc/> (2022)
 59. Tran, K.Q., Naughton, J.F., Sundarmurthy, B., Tsirogiannis, D.: Jecb: A join-extension, code-based approach to oltp data partitioning. In: *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, SIGMOD '14*, p. 39–50. Association for Computing Machinery, New York, NY, USA (2014)
 60. VanBenschoten, N.: Parallel Commits: An Atomic Commit Protocol For Globally Distributed Transactions. URL <https://www.cockroachlabs.com/blog/parallel-commits/>. (2019, November 7)
 61. Wang, C., Qian, X.: Rdma-enabled concurrency control protocols for transactions in the cloud era. *IEEE Transactions on Cloud Computing* **PP**(01), 1–1 (5555)
 62. Wei, X., Dong, Z., Chen, R., Chen, H.: Deconstructing RDMA-enabled distributed transactions: Hybrid is better! In: *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pp. 233–251. USENIX Association, Carlsbad, CA (2018)
 63. Wei, X., Shi, J., Chen, Y., Chen, R., Chen, H.: Fast in-memory transaction processing using rdma and htm. In: *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP '15*, p. 87–104. Association for Computing Machinery, New York, NY, USA (2015)
 64. Wei, X., Xie, X., Chen, R., Chen, H., Zang, B.: Characterizing and optimizing remote persistent memory with RDMA and NVM. In: *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pp. 523–536. USENIX Association (2021)
 65. www.tlapl.us: <https://github.com/tlaplus/tlaplus.git/> (2025)
 66. Yan, X., Yang, L., Zhang, H., Lin, X.C., Wong, B., Salem, K., Brecht, T.: Carousel: Low-latency transaction processing for globally-distributed data. In: *SIGMOD Conference*, pp. 231–243. ACM (2018)
 67. Yang, Z., Yang, C., Han, F., Zhuang, M., Yang, B., Yang, Z., Cheng, X., Zhao, Y., Shi, W., Xi, H., Yu, H., Liu, B., Pan, Y., Yin, B., Chen, J., Xu, Q.: Oceanbase: A 707 million tpmc distributed relational database system. *Proc. VLDB Endow.* **15**(12), 3385–3397 (2022)
 68. Yoon, D.Y., Chowdhury, M., Mozafari, B.: Distributed lock management with rdma: Decentralization without starvation. In: *Proceedings of the 2018 International Conference on Management of Data, SIGMOD '18*, pp. 1571–1586. Association for Computing Machinery, New York, NY, USA (2018)
 69. Yu, X., Bezerra, G., Pavlo, A., Devadas, S., Stonebraker, M.: Staring into the abyss: An evaluation of concurrency control with one thousand cores. *Proc. VLDB Endow.* **8**(3), 209–220 (2014)
 70. Zamanian, E., Binnig, C., Harris, T., Kraska, T.: The end of a myth: Distributed transactions can scale. *Proc. VLDB Endow.* **10**(6), 685–696 (2017)
 71. Zamanian, E., Binnig, C., Salama, A.: Locality-aware partitioning in parallel database systems. In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, SIGMOD '15*, p. 17–30. Association for Computing Machinery, New York, NY, USA (2015)
 72. Zamanian, E., Yu, X., Stonebraker, M., Kraska, T.: Rethinking database high availability with rdma networks. *Proc. VLDB Endow.* **12**(11), 1637–1650 (2019)
 73. Zhang, I., Sharma, N.K., Szekeres, A., Krishnamurthy, A., Ports, D.R.K.: Building consistent transactions with inconsistent replication. In: *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP '15*, p. 263–278. Association for Computing Machinery, New York, NY, USA (2015)
 74. Zhang, Q., Li, J., Zhao, H., Xu, Q., Lu, W., Xiao, J., Han, F., Yang, C., Du, X.: Efficient distributed transaction processing in heterogeneous networks. *Proc. VLDB Endow.* **16**(6), 1372–1385 (2023)
 75. Zhao, H., Li, J., Lu, W., Zhang, Q., Yang, W., Zhong, J., Zhang, M., Li, H., Du, X., Pan, A.: Rcbench: an rdma-enabled transaction framework for analyzing concurrency control algorithms. *The VLDB Journal* **33**(2), 543–567 (2023)
 76. Zhou, X., Zhou, X., Yu, Z., Tan, K.L.: Posterior snapshot isolation. In: *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*, pp. 797–808 (2017)
 77. Zhu, Y., Eran, H., Firestone, D., Guo, C., Lipshteyn, M., Liron, Y., Padhye, J., Raindel, S., Yahia, M.H., Zhang, M.: Congestion control for large-scale rdma deployments. In: *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, SIGCOMM '15*, p. 523–536. Association for Computing Machinery, New York, NY, USA (2015)
 78. Ziegler, T., Nelson-Slivon, J., Leis, V., Binnig, C.: Design guidelines for correct, efficient, and scalable synchronization using one-sided rdma. *Proc. ACM Manag. Data* **1**(2) (2023)
 79. Ziegler, T., Tumkur Vani, S., Binnig, C., Fonseca, R., Kraska, T.: Designing distributed tree-based index structures for fast rdma-capable networks. In: *Proceedings of the 2019 International Conference on Management of Data, SIGMOD '19*, p. 741–758. Association for Computing Machinery, New York, NY, USA (2019)

EXTENDS *Naturals, Sequences, FiniteSets, TLC*

CONSTANTS *N*

CONSTANTS *M*

CONSTANTS *C*

CONSTANTS *FAILEDPOINT1*

CONSTANTS *FAILEDPOINT2*

CONSTANTS *FailedC*

CONSTANTS *FailedN*

--algorithm *RedT*

variables

Replicas = [*item* ∈ 1 .. *N* ↦ [*rep* ∈ 1 .. *M* ↦ {[*value* ↦ 0]}]],

Transactions = [*tid* ∈ 1 .. *C* ↦
 LET *accessSet* $\triangleq$ {*item* ∈ 1 .. *N* : TRUE} IN
 [*access* ↦ *accessSet*,
 newValue ↦ *tid*,
 coordinator_id ↦ *tid*]],

Messages = {},

CoordinatorMessages = {},

TransactionStatusCoor = [*c* ∈ (1 + (*N* * *M*)) .. (*C* + (*N* * *M*)) ↦
 [*tid* ∈ 1 .. *C* ↦ [*status* ↦ "NotStarted"]]],

TransactionStatus = [*rep* ∈ 1 .. (*N* * *M*) ↦ [*tid* ∈ 1 .. *C* ↦
 [*status* ↦ "NotStarted"]]],

SuccessCounts = [*c* ∈ (1 + (*N* * *M*)) .. (*C* + (*N* * *M*)) ↦
 [*item* ∈ 1 .. *N* ↦ 0]];

FailureCounts = [*c* ∈ (1 + (*N* * *M*)) .. (*C* + (*N* * *M*)) ↦
 [*item* ∈ 1 .. *N* ↦ 0]];

ReadResults = [*c* ∈ (1 + (*N* * *M*)) .. (*C* + (*N* * *M*)) ↦
 [*tid* ∈ 1 .. *C* ↦ [*item* ∈ 1 .. *N* ↦ {}]]];

ReadConsistency = [*c* ∈ (1 + (*N* * *M*)) .. (*C* + (*N* * *M*)) ↦ TRUE];

DoneCounts = [*c* ∈ (1 + (*N* * *M*)) .. (*C* + (*N* * *M*)) ↦
 [*item* ∈ 1 .. *N* ↦ 0]];

NodeStatus = [*nid* ∈ (1 + (*N* * *M*)) .. (*C* + (*N* * *M*))
 ∪ (1 .. (*N* * *M*)) ↦ "Active"];

FailedCounts = [*nid* ∈ 1 .. (*C* + (*N* * *M*)) ↦ 0];

CommitCounts = [*c* ∈ (1 + (*N* * *M*)) .. (*C* + (*N* * *M*)) ↦ 0];

AbortCounts = [*c* ∈ (1 + (*N* * *M*)) .. (*C* + (*N* * *M*)) ↦ 0];

SystemTerminated = FALSE;

MessageLimit = *N* * *M*;

define

ConsistentFinishedTransactions $\triangleq$ TRUE

NoConflictingTransactionStatus $\triangleq$ TRUE

MessageConstraint $\triangleq$

$\wedge \text{Cardinality}(\text{Messages}) \leq \text{MessageLimit}$
 $\wedge \text{Cardinality}(\text{CoordinatorMessages}) \leq \text{MessageLimit}$

$\text{Durability} \triangleq$
 $\forall tid \in 1 \dots C :$
 $\forall cid \in (1 + (N * M)) \dots (C + (N * M)) :$
 LET
 $\text{ActiveReplicas}(item) \triangleq \{rep \in 1 \dots (N * M) :$
 $\wedge \text{NodeStatus}[rep] = \text{"Active"}$
 $\wedge rep \geq ((item - 1) * M + 1)$
 $\wedge rep \leq (item * M)\}$
 $\text{UpdatedReplicas}(item) \triangleq \{rep \in \text{ActiveReplicas}(item) :$
 $\text{Replicas}[item][((rep - 1) \% M) + 1].value = tid\}$
 IN
 $\text{TransactionStatusCoor}[cid][tid].status = \text{"FinCommit"} \Rightarrow$
 $\forall item \in \text{Transactions}[tid].access :$
 $\text{Cardinality}(\text{UpdatedReplicas}(item)) > M \div 2$

$\text{Atomicity} \triangleq$
 $\forall tid \in 1 \dots C :$
 $\forall cid \in (1 + (N * M)) \dots (C + (N * M)) :$
 LET
 $\text{ActiveReplicas}(item) \triangleq \{rep \in 1 \dots (N * M) :$
 $\wedge \text{NodeStatus}[rep] = \text{"Active"}$
 $\wedge rep \geq ((item - 1) * M + 1)$
 $\wedge rep \leq (item * M)\}$
 Committed
 $\text{CommittedCount}(item) \triangleq \text{Cardinality}(\{rep$
 $\in \text{ActiveReplicas}(item) :$
 $\text{TransactionStatus}[rep][tid].status = \text{"Committed"}\})$
 Aborted
 $\text{AbortedCount}(item) \triangleq \text{Cardinality}(\{rep$
 $\in \text{ActiveReplicas}(item) :$
 $\text{TransactionStatus}[rep][tid].status = \text{"Aborted"}\})$
 IN
 $\vee \text{TransactionStatusCoor}[cid][tid].status = \text{"FinCommit"} \Rightarrow$
 $\forall item \in \text{Transactions}[tid].access :$
 $\text{CommittedCount}(item) > M \div 2$
 $\vee \text{TransactionStatusCoor}[cid][tid].status = \text{"FinAbort"} \Rightarrow$
 $\forall item \in \text{Transactions}[tid].access :$
 $\text{AbortedCount}(item) > M \div 2$

end define ;

```

procedure NNodeFail(id)
variables msg = {}, type = "", fail_occur = FALSE;
begin
  ProcSetNodeFail:
  if FailedN ≠ 0 ∧ id = FailedN ∧ FailedCounts[id] < 1 then
    fail_occur := CHOOSE fail ∈ {TRUE, FALSE} : TRUE;
    if fail_occur = TRUE then
      NodeStatus[id] := "Failed" ;
      FailedCounts[id] := FailedCounts[id] + 1 ;
      TransactionStatus[id] := [tid ∈ 1 .. C ↦
        [status ↦ "NotStarted",
         CommitTS ↦ 0]] ;
    end if ;
  end if ;
  ProcSetNodeFailB:
  return ;
end procedure ;

procedure CNodeFail(id)
variables msg = {}, type = "" ;
begin
  ProcSetCNodeFail:
  TransactionStatusCoor[id] :=
    [tid ∈ 1 .. C
     ↦ [status ↦ "Recover", CommitTS ↦ 0]] ;
  SuccessCounts[id] := [item ∈ 1 .. N ↦ 0] ;
  FailureCounts[id] := [item ∈ 1 .. N ↦ 0] ;
  DoneCounts[id] := [item ∈ 1 .. N ↦ 0] ;
  CommitCounts[id] := 0 ;
  AbortCounts[id] := 0 ;
  return ;
end procedure ;

  CalculateACKCounts
procedure CalculateACKCounts(t, msg, cid)
variables countsUpdated = FALSE;
begin
  ProcCalculateCounts:
  if msg.type = "Process-ack" then
    if msg.status = "Success" then
      SuccessCounts[cid][msg.item] := SuccessCounts[cid][msg.item] + 1 ;
    elsif msg.status = "Failed" then
      FailureCounts[cid][msg.item] := FailureCounts[cid][msg.item] + 1 ;
    end if ;
  end if ;

```

```

end if ;
ProcCalculateCountsB:
if msg.type = "Report" then
  if msg.status = "Success" then
    SuccessCounts[cid][msg.item] := SuccessCounts[cid][msg.item] + 1 ;
  elseif msg.status = "Failed" then
    FailureCounts[cid][msg.item] := FailureCounts[cid][msg.item] + 1 ;
  elseif msg.status = "Committed" then
    CommitCounts[cid] := CommitCounts[cid] + 1 ;
  elseif msg.status = "Aborted" then
    AbortCounts[cid] := AbortCounts[cid] + 1 ;
  end if ;
end if ;
ProcCalculateCountsC:
return ;
end procedure ;

procedure WaitForACK(txnId, cid, mtype, quorum, minority)
variables msg = {} ;
begin
  WaitForACKs:
  while ( $\exists i5 \in \text{Transactions}[\text{txnId}].\text{access} : \text{SuccessCounts}[\text{cid}][i5] +$ 
     $\text{FailureCounts}[\text{cid}][i5] < M$ )  $\wedge$ 
    ( $\exists m \in \text{Messages} : m.\text{coordinator\_id} = \text{cid} \wedge m.\text{transaction} = \text{txnId}$ ) do
    if  $\exists m\_ack \in \text{CoordinatorMessages} : m\_ack.\text{coordinator\_id} = \text{cid}$ 
       $\wedge m\_ack.\text{transaction} = \text{txnId} \wedge m\_ack.\text{type} = \text{mtype}$  then
      msg := CHOOSE  $m\_ack \in \text{CoordinatorMessages} : m\_ack.\text{coordinator\_id} = \text{cid}$ 
         $\wedge m\_ack.\text{transaction} = \text{txnId} \wedge m\_ack.\text{type} = \text{mtype}$  ;
      CoordinatorMessages := CoordinatorMessages \ {msg} ;
      call CalculateACKCounts(txnId, msg, cid) ;
    end if ;
  end while ;

  ****
  ProcExecutorDecide:
  if CommitCounts[cid] > 0 then
    decision := "Commit" ;
  elseif AbortCounts[cid] > 0 then
    decision := "Abort" ;
  elseif  $\exists i5 \in \text{Transactions}[\text{txnId}].\text{access} :$ 
     $\vee \text{FailureCounts}[\text{cid}][i5] > \text{minority}$ 
     $\vee \text{SuccessCounts}[\text{cid}][i5] \leq \text{quorum}$  then
    decision := "Abort" ;
  else
    decision := "Commit" ;

```



```

end if ;
ProcExecutorDecide2:
return ;
end procedure ;

procedure SendDecision(txnId, cid, decision)
variables i7 = 0, j7 = 0 ;
begin
  ProcExecutorSendDecideOuterLoop:
  if decision = "Commit" then
    TransactionStatusCoor[cid][txnId] := [status  $\mapsto$  "Committed",
                                              CommitTS  $\mapsto$  commitTimestamp] ;
    ProcExecutorSendCommitOuterLoop:
    while i7 < N do
      i7 := i7 + 1 ;
      if i7  $\in$  Transactions[txnId].access then
        j7 := 0 ;
        ProcExecutorSendCommitInnerLoop:
        while j7 < M do
          j7 := j7 + 1 ;
          targetReplica := ((i7 - 1) * M) + j7 ;
          s_msg := [item  $\mapsto$  i7, replica  $\mapsto$  j7,
                    transaction  $\mapsto$  txnId, type  $\mapsto$  "Commit",
                    timestamp  $\mapsto$  commitTimestamp,
                    coordinator_id  $\mapsto$  Transactions[txnId].coordinator_id,
                    targetReplica  $\mapsto$  targetReplica] ;
          Messages := Messages  $\cup$  {s_msg} ;
        end while ;
      end if ;
    end while ;
  elseif decision = "Abort" then
    TransactionStatusCoor[cid][txnId].status := "Aborted" ;
    ProcExecutorSendAbortOuterLoop:
    while i7 < N do
      i7 := i7 + 1 ;
      if i7  $\in$  Transactions[txnId].access then
        j7 := 0 ;
        ProcExecutorSendAbortInnerLoop:
        while j7 < M do
          j7 := j7 + 1 ;
          targetReplica := ((i7 - 1) * M) + j7 ;
          s_msg := [item  $\mapsto$  i7, replica  $\mapsto$  j7,
                    transaction  $\mapsto$  txnId, type  $\mapsto$  "Abort",
                    timestamp  $\mapsto$  commitTimestamp,
                    coordinator_id  $\mapsto$  Transactions[txnId].coordinator_id,

```

```

        targetReplica  $\mapsto$  targetReplica];
    Messages := Messages  $\cup$  {s_msg};
end while ;
end if ;
end while ;
end if ;
ProcExecutorSendDecideOuterLoopEnd:
return ;
end procedure ;

ExecuteTransaction
procedure ExecuteTransaction(txnId)
variables i1 = 0, j1 = 0, targetReplica = 0,
           s_msg = {}, cid = 0, decision = "Pending",
           i7 = 0, j7 = 0, commitTimestamp = 0, msg = {};
begin
ProcExecuteTransaction:
    cid := Transactions[txnId].coordinator_id ;

    CurrentTransaction[cid] := txnId;
    TransactionStatusCoor[cid][txnId].status := "Pending" ;
    SuccessCounts[cid] := [item  $\in$  1 .. N  $\mapsto$  0];
    FailureCounts[cid] := [item  $\in$  1 .. N  $\mapsto$  0];
    DoneCounts[cid] := [item  $\in$  1 .. N  $\mapsto$  0];
    CommitCounts[cid] := 0;
    AbortCounts[cid] := 0;

    **: Read/Write ()**
ProcExecutorSendRequestsOuterLoop:
while i1 < N do
    i1 := i1 + 1;
    if i1  $\in$  Transactions[txnId].access then
        j1 := 0;
        ProcExecutorSendRequestsInnerLoop:
        while j1 < M do
            j1 := j1 + 1;
            targetReplica := ((i1 - 1) * M) + j1;
            s_msg := [item  $\mapsto$  i1,
                    replica  $\mapsto$  j1,
                    transaction  $\mapsto$  txnId,
                    type  $\mapsto$  "Access",
                    coordinator_id  $\mapsto$  cid,
                    targetReplica  $\mapsto$  targetReplica];
            Messages := Messages  $\cup$  {s_msg};
        end while ;
    end if ;
end while ;

```

```

    end if ;
end while ;

** Process-ack () **
call WaitForACK(txnId, cid, "Process-ack",  $(3 * M) \div 4$ ,  $M \div 4$ ) ;

** 1: Commit/Abort **
FailurePoint1:
if FAILEDPOINT1 then TRUE
    if id = FailedC + ( $N * M$ ) then
        call CNodeFail(cid) ;
        FailurePoint11:
        call CoordinatorRecover(cid) ;
        FailurePoint12:
        call WaitForACK(txnId, cid, "Report",  $M \div 2$ ,  $M \div 2$ ) ;
    end if ;
end if ;

** 2: Commit/Abort Fin **
ProcExecutorSendDecideOuterLoop:
call SendDecision(txnId, cid, decision) ;

** 2: Commit/Abort Fin **
FailurePoint2:
if FAILEDPOINT2 then TRUE
    if id = FailedC + ( $N * M$ ) then
        call CNodeFail(cid) ;
        FailurePoint21:
        call CoordinatorRecover(cid) ;
        FailurePoint22:
        call WaitForACK(txnId, cid, "Report",  $M \div 2$ ,  $M \div 2$ ) ;
        FailurePoint23:
        call SendDecision(txnId, cid, decision) ;
    end if ;
end if ;

** Fin () **
WaitForFins:
while ( $\exists i6 \in Transactions[txnId].access : DoneCounts[cid][i6] < (3 * M) \div 4$ )
    ^
    ( $\exists m \in Messages : m.coordinator\_id = cid \wedge m.transaction = txnId$ ) do
    if  $\exists m\_fin \in CoordinatorMessages : m\_fin.coordinator\_id = cid$ 
        ^  $m\_fin.transaction = txnId \wedge m\_fin.type = "Fin"$  then
        msg := CHOOSE  $m\_fin \in CoordinatorMessages : m\_fin.coordinator\_id = cid$ 
            ^  $m\_fin.transaction = txnId \wedge m\_fin.type = "Fin"$  ;
        CoordinatorMessages := CoordinatorMessages \ {msg} ;
        DoneCounts[cid][msg.item] := DoneCounts[cid][msg.item] + 1 ;
    end if ;
end while ;

```

```

    end if ;
end while ;

****
ProcExecutorFinA:
if decision = "Commit" then
    TransactionStatusCoor[cid][txnId].status := "FinCommit" ;
else
    ProcExecutorFinB:
    TransactionStatusCoor[cid][txnId].status := "FinAbort" ;
end if ;
ProcExecutorFinC:
return ;
end procedure ;

HandleAccess
procedure HandleAccess(msg, id)
variables i = 0, j = 0, t = 0, res_msg = {}, operation_status ;
begin
    ProcHandleRead:
    i := msg.item ;
    j := msg.replica ;
    t := msg.transaction ;

    operation_status := CHOOSE status_result ∈ { "Success", "Failed" } : TRUE ;
    TransactionStatus[id][t] := [status ↦ operation_status] ;
    res_msg := [item ↦ i, replica ↦ j, transaction ↦ t,
        type ↦ "Process-ack", status ↦ operation_status,
        value ↦ Replicas[i][j].value,
        coordinator_id ↦ msg.coordinator_id] ;
    CoordinatorMessages := CoordinatorMessages ∪ { res_msg } ;
    ProcHandleReadB:
    return ;
end procedure ;

procedure HandleInquire(msg, id)
variables i = 0, j = 0, t = 0, res_msg = {} ;
begin
    ProcHandleInquire:
    i := msg.item ;
    j := msg.replica ;
    t := msg.transaction ;
    operation_status := TransactionStatus[id][t].status ;
    res_msg := [item ↦ i, replica ↦ j, transaction ↦ t,
        type ↦ "Report", status ↦ operation_status,
        value ↦ Replicas[i][j].value,

```

```

        coordinator_id  $\mapsto$  msg.coordinator_id];
    CoordinatorMessages := CoordinatorMessages  $\cup$  {res_msg};
    ProcHandleWriteB:
    return;
end procedure ;

    HandleCommit
procedure HandleCommit(msg, id)
variables i = 0, j = 0, t = 0, idx = 0, res_msg = {};
begin
    ProcHandleCommit:
    i := msg.item;
    j := msg.replica;
    t := msg.transaction;
    TransactionStatus[id][t] := [status  $\mapsto$  "Committed"];
    Replicas[i][j] := [value  $\mapsto$  Transactions[t].newValue];
    res_msg := [item  $\mapsto$  i, replica  $\mapsto$  j, transaction  $\mapsto$  t,
        type  $\mapsto$  "Fin", coordinator_id  $\mapsto$  msg.coordinator_id];
    CoordinatorMessages := CoordinatorMessages  $\cup$  {res_msg};
    ProcHandleCommitB:
    return;
end procedure ;

procedure HandleAbort(msg, id)
variables i = 0, j = 0, t = 0, res_msg = {};
begin
    ProcHandleAbort:
    i := msg.item;
    j := msg.replica;
    t := msg.transaction;
    TransactionStatus[id][t].status := "Aborted";
    res_msg := [item  $\mapsto$  i, replica  $\mapsto$  j, transaction  $\mapsto$  t,
        type  $\mapsto$  "Fin", coordinator_id  $\mapsto$  msg.coordinator_id];
    CoordinatorMessages := CoordinatorMessages  $\cup$  {res_msg};
    ProcHandleAbortB:
    return;
end procedure ;

procedure CoordinatorRecover(id)
variables s_msg = {}, i8 = 0, j8 = 0, t = 0;
begin
    ProcCoordinatorRecover:
    NodeStatus[id] := "Recover";
    t := 1;
    ProcCoordinatorRecoverOuterLoop:
    while i8 < N do

```

```

i8 := i8 + 1 ;
if i8 ∈ Transactions[t].access then
  j8 := 0 ;
  ProcCoordinatorRecoverInnerLoop:
  while j8 < M do
    j8 := j8 + 1 ;
    targetReplica := ((i8 - 1) * M) + j8 ;
    s_msg := [item ↦ i8, replica ↦ j8,
              transaction ↦ t, type ↦ "Inquire",
              timestamp ↦ 0,
              coordinator_id ↦ Transactions[t].coordinator_id,
              targetReplica ↦ targetReplica] ;
    Messages := Messages ∪ {s_msg} ;
  end while ;
end if ;
end while ;
ProcCoordinatorRecoverB:
return ;
end procedure ;

procedure ReplicaReceiveRequest(id)
variables msg = {}, type = "" ;
begin
  ProcRepRecReq:
  if ∃ m ∈ Messages : m.targetReplica = id ∧ m.type ≠ "SyncAck" then
    msg := CHOOSE m ∈ Messages : m.targetReplica = id
      ∧ m.type ≠ "SyncAck" ;
    Messages := Messages \ {msg} ;
    if NodeStatus[id] = "Failed" then
      ProcRepRecReqD:
      return ;
    end if ;
    ProcRepRecReqB:
    if msg.type = "Access" then
      call HandleAccess(msg, id)
    elsif msg.type = "Commit" then
      call HandleCommit(msg, id)
    elsif msg.type = "Abort" then
      call HandleAbort(msg, id)
    elsif msg.type = "Inquire" then
      call HandleInquire(msg, id)
    end if ;
  end if ;
  ProcRepRecReqC:
  return ;

```



```

end procedure ;

process Main = 1
variable cid = 0;
begin
  MainProcess:
    cid := Transactions[1].coordinator_id;
    call ExecuteTransaction(1);
  MainTerminate:
    SystemTerminated := TRUE;
end process ;

process Replica  $\in 1 \dots (N * M)$ 
variable id = self;
begin
  ReplicaMain:
    while SystemTerminated = FALSE do
      call ReplicaReceiveRequest(id);
    ReplicaMainB:
      call NNodeFail(id);
    end while ;
end process ;
end algorithm ;

```

MODULE *RedTSI*

EXTENDS *Integers, FiniteSets, Sequences, TLC*

CONSTANTS *KEY*,
 CLIENT

CONSTANTS *REPLICA_COUNT*,
 QUORUM_SIZE, ($3/4 * \text{REPLICA_COUNT}$)
 MAX_TXN_PER_CLIENT,
 MAX_OPS_PER_TXN

ASSUME $\wedge \text{REPLICA_COUNT} > 0$
 $\wedge \text{QUORUM_SIZE} \leq \text{REPLICA_COUNT}$
 $\wedge \text{QUORUM_SIZE} > \text{REPLICA_COUNT} \div 2$

VARIABLES *next_ts*,
 client_state,
 client_ts,
 client_read_set,
 client_write_set,
 key_replica_data,
 key_replica_lock,
 key_replica_ts,
 key_replica_last_read_ts,
 key_replica_si,
 client_txn_count,
 client_op_count

vars $\triangleq$ $\langle \text{next_ts}, \text{client_state}, \text{client_ts}, \text{client_read_set},$
 client_write_set,
 key_replica_data, *key_replica_lock*, *key_replica_ts*,
 key_replica_last_read_ts, *key_replica_si*, *client_txn_count*,
 client_op_count
 $\rangle$

States $\triangleq$ { "init", "active", "failed", "committed", "aborted" }

Init $\triangleq$
 $\wedge \text{next_ts} = 0$
 $\wedge \text{client_state} = [c \in \text{CLIENT} \mapsto \text{"init"}]$
 $\wedge \text{client_ts} = [c \in \text{CLIENT} \mapsto [\text{start_ts} \mapsto 0, \text{commit_ts} \mapsto 0]]$
 $\wedge \text{client_read_set} = [c \in \text{CLIENT} \mapsto \{\}]$
 $\wedge \text{client_write_set} = [c \in \text{CLIENT} \mapsto \{\}]$
 $\wedge \text{key_replica_data} = [k \in \text{KEY} \mapsto$
 $[r \in 1 \dots \text{REPLICA_COUNT} \mapsto \langle [\text{ts} \mapsto 0, \text{val} \mapsto \text{"initial"}] \rangle]]$
 $\wedge \text{key_replica_lock} = [k \in \text{KEY} \mapsto [r \in 1 \dots \text{REPLICA_COUNT} \mapsto \{\}]]$

$\wedge \text{key_replica_ts} = [k \in \text{KEY} \mapsto [r \in 1 \dots \text{REPLICA_COUNT} \mapsto 0]]$
 $\wedge \text{key_replica_si} = [k \in \text{KEY} \mapsto [r \in 1 \dots \text{REPLICA_COUNT} \mapsto \text{TRUE}]]$
 $\wedge \text{key_replica_last_read_ts} = [k \in \text{KEY} \mapsto [r \in 1 \dots \text{REPLICA_COUNT} \mapsto 0]]$
 $\wedge \text{client_txn_count} = [c \in \text{CLIENT} \mapsto 0]$ $\parallel$
 $\wedge \text{client_op_count} = [c \in \text{CLIENT} \mapsto 0]$

$\text{HasQuorum}(k, \text{pred}(_)) \triangleq$
 $\text{LET } \text{replicas} \triangleq \{r \in 1 \dots \text{REPLICA_COUNT} : \text{pred}(r)\}$
 $\text{IN } \text{Cardinality}(\text{replicas}) \geq \text{QUORUM_SIZE}$

$\text{checkSnapshotIsolation}(k, \text{commit_ts}) \triangleq$
 $\wedge \forall r \in 1 \dots \text{REPLICA_COUNT} :$
 $\quad \text{IF } \text{key_replica_last_read_ts}[k][r] \geq \text{commit_ts}$
 $\quad \quad \text{THEN } \text{key_replica_si}' = [\text{key_replica_si} \text{ EXCEPT } ![k][r] = \text{FALSE}]$
 $\quad \quad \text{ELSE UNCHANGED } \text{key_replica_si}$

$\parallel$
 $\text{Start}(c) \triangleq$
 $\wedge \text{client_state}[c] = \text{"init"}$
 $\wedge \text{next_ts}' = \text{next_ts} + 1$
 $\wedge \text{client_state}' = [\text{client_state} \text{ EXCEPT } ![c] = \text{"active"}]$
 $\wedge \text{client_ts}' = [\text{client_ts} \text{ EXCEPT } ![c].\text{start_ts} = \text{next_ts}']$
 $\wedge \text{client_op_count}' = [\text{client_op_count} \text{ EXCEPT } ![c] = 0]$ $\parallel$
 $\wedge \text{UNCHANGED } \langle \text{client_read_set}, \text{client_write_set},$
 $\quad \text{key_replica_data}, \text{key_replica_ts}, \text{key_replica_lock},$
 $\quad \text{key_replica_last_read_ts}, \text{key_replica_si}, \text{client_txn_count} \rangle$

$\parallel$
 $\text{Read}(c, k) \triangleq$
 $\wedge \text{client_state}[c] = \text{"active"}$
 $\wedge \text{client_op_count}[c] < \text{MAX_OPS_PER_TXN}$
 $\wedge k \notin \text{client_read_set}[c]$ $\parallel$
 $\wedge \vee \wedge \text{HasQuorum}(k, \text{LAMBDA } r : \text{key_replica_lock}[k][r] = \{\})$
 $\quad \wedge \exists \text{data} \in \text{DOMAIN } \text{key_replica_data}[k][1] :$
 $\quad \quad \wedge \text{key_replica_data}[k][1][\text{data}].\text{ts} \leq \text{client_ts}[c].\text{start_ts}$
 $\quad \quad \wedge \neg \exists \text{newer} \in \text{DOMAIN } \text{key_replica_data}[k][1] :$
 $\quad \quad \quad \wedge \text{key_replica_data}[k][1][\text{newer}].\text{ts} \leq \text{client_ts}[c].\text{start_ts}$
 $\quad \quad \quad \wedge \text{key_replica_data}[k][1][\text{newer}].\text{ts} > \text{key_replica_data}[k][1][\text{data}].\text{ts}$
 $\quad \wedge \text{HasQuorum}(k, \text{LAMBDA } r :$ $\parallel$
 $\quad \quad \exists d \in \text{DOMAIN } \text{key_replica_data}[k][r] :$
 $\quad \quad \quad \text{key_replica_data}[k][r][d].\text{ts} = \text{key_replica_data}[k][1][\text{data}].\text{ts})$
 $\wedge \text{client_read_set}' = [\text{client_read_set} \text{ EXCEPT } ![c] = @ \cup \{k\}]$
 $\wedge \text{key_replica_last_read_ts}' = [\text{key_replica_last_read_ts} \text{ EXCEPT } ![k] =$
 $\quad [r \in 1 \dots \text{REPLICA_COUNT} \mapsto$
 $\quad \quad \text{IF } r \leq \text{QUORUM_SIZE} \text{ THEN } \text{client_ts}[c].\text{start_ts} \text{ ELSE } @[r]]]$

$$\begin{aligned}
& \wedge \text{UNCHANGED } \langle \text{next_ts}, \text{client_state}, \text{client_ts}, \text{client_write_set}, \\
& \quad \text{key_replica_data}, \text{key_replica_ts}, \text{key_replica_lock}, \\
& \quad \text{key_replica_si}, \text{client_txn_count} \rangle \\
\vee & \wedge \text{client_state}' = [\text{client_state} \text{ EXCEPT } ![c] = \text{"failed"}] \\
& \wedge \text{UNCHANGED } \langle \text{next_ts}, \text{client_ts}, \text{client_read_set}, \text{client_write_set}, \\
& \quad \text{key_replica_data}, \text{key_replica_lock}, \text{key_replica_ts}, \\
& \quad \text{key_replica_last_read_ts}, \text{key_replica_si}, \text{client_txn_count} \rangle \\
& \wedge \text{client_op_count}' = [\text{client_op_count} \text{ EXCEPT } ![c] = @ + 1]
\end{aligned}$$

$$\begin{aligned}
& \text{Write}(c, k) \triangleq \\
& \wedge \text{client_state}[c] = \text{"active"} \\
& \wedge \text{client_op_count}[c] < \text{MAX_OPS_PER_TXN} \\
& \wedge k \notin \text{client_write_set}[c] \\
& \wedge \vee \wedge \text{HasQuorum}(k, \text{LAMBDA } r : \text{key_replica_lock}[k][r] = \{\}) \\
& \quad \wedge \forall r \in 1 \dots \text{REPLICA_COUNT} : \\
& \quad \quad \vee r > \text{QUORUM_SIZE} \\
& \quad \quad \vee \text{key_replica_ts}[k][r] \leq \text{client_ts}[c].\text{start_ts} \\
& \quad \wedge \text{key_replica_lock}' = [\text{key_replica_lock} \text{ EXCEPT } ![k] = \\
& \quad \quad [r \in 1 \dots \text{REPLICA_COUNT} \mapsto \\
& \quad \quad \quad \text{IF } r \leq \text{QUORUM_SIZE} \text{ THEN } \{c\} \text{ ELSE } \text{key_replica_lock}[k][r]]] \\
& \quad \wedge \text{client_write_set}' = [\text{client_write_set} \text{ EXCEPT } ![c] = @ \cup \{k\}] \\
& \quad \wedge \text{UNCHANGED } \langle \text{next_ts}, \text{client_state}, \text{client_ts}, \text{client_read_set}, \\
& \quad \quad \text{key_replica_data}, \text{key_replica_ts}, \\
& \quad \quad \text{key_replica_last_read_ts}, \\
& \quad \quad \text{key_replica_si}, \text{client_txn_count} \rangle \\
\vee & \wedge \text{client_state}' = [\text{client_state} \text{ EXCEPT } ![c] = \text{"failed"}] \\
& \wedge \text{UNCHANGED } \langle \text{next_ts}, \text{client_ts}, \text{client_read_set}, \text{client_write_set}, \\
& \quad \text{key_replica_data}, \text{key_replica_lock}, \text{key_replica_ts}, \\
& \quad \text{key_replica_last_read_ts}, \text{key_replica_si}, \text{client_txn_count} \rangle \\
& \wedge \text{client_op_count}' = [\text{client_op_count} \text{ EXCEPT } ![c] = @ + 1]
\end{aligned}$$

$$\begin{aligned}
& \text{Commit}(c) \triangleq \\
& \wedge \text{client_state}[c] = \text{"active"} \\
& \wedge \forall k \in \text{client_write_set}[c] : \\
& \quad \text{HasQuorum}(k, \text{LAMBDA } r : c \in \text{key_replica_lock}[k][r]) \\
& \wedge \text{next_ts}' = \text{next_ts} + 1 \\
& \wedge \text{client_state}' = [\text{client_state} \text{ EXCEPT } ![c] = \text{"committed"}] \\
& \wedge \text{client_ts}' = [\text{client_ts} \text{ EXCEPT } ![c].\text{commit_ts} = \text{next_ts}'] \\
& \wedge \text{IF } \text{client_write_set}[c] = \{\} \\
& \quad \text{THEN } \wedge \text{UNCHANGED } \text{key_replica_data} \\
& \quad \quad \wedge \text{UNCHANGED } \text{key_replica_ts} \\
& \quad \quad \wedge \text{UNCHANGED } \text{key_replica_lock}
\end{aligned}$$

$\wedge$ UNCHANGED $key_replica_last_read_ts$
 $\wedge$ UNCHANGED $key_replica_si$
 ELSE $\wedge \forall k \in client_write_set[c] :$
 $\wedge checkSnapshotIsolation(k, next_ts')$
 $\wedge key_replica_data' = [key_replica_data \text{ EXCEPT } ![k] =$
 $\quad [r \in 1 \dots REPLICATION_COUNT \mapsto$
 $\quad \text{IF } r \leq QUORUM_SIZE$
 $\quad \text{THEN } Append(@[r], [ts \mapsto next_ts', val \mapsto \text{"written"}])$
 $\quad \text{ELSE } @[r]]]$
 $\wedge key_replica_ts' = [key_replica_ts \text{ EXCEPT } ![k] =$
 $\quad [r \in 1 \dots REPLICATION_COUNT \mapsto$
 $\quad \text{IF } r \leq QUORUM_SIZE \text{ THEN } next_ts' \text{ ELSE } @[r]]]$
 $\wedge key_replica_lock' = [key_replica_lock \text{ EXCEPT } ![k] =$
 $\quad [r \in 1 \dots REPLICATION_COUNT \mapsto \{\}]]$
 $\wedge$ UNCHANGED $key_replica_last_read_ts$ |
 $\wedge$ UNCHANGED $\langle client_read_set, client_write_set,$
 $\quad client_txn_count, client_op_count \rangle$

$|$
 $Abort(c) \triangleq$
 $\wedge \vee client_state[c] = \text{"active"}$
 $\vee client_state[c] = \text{"failed"}$
 $\wedge client_state' = [client_state \text{ EXCEPT } ![c] = \text{"aborted"}]$
 $\wedge key_replica_lock' = [k \in KEY \mapsto$ |
 $\quad [r \in 1 \dots REPLICATION_COUNT \mapsto$
 $\quad \text{IF } c \in key_replica_lock[k][r] \text{ THEN } \{\} \text{ ELSE } key_replica_lock[k][r]]]$
 $\wedge$ UNCHANGED $\langle next_ts, client_ts, client_read_set, client_write_set,$
 $\quad key_replica_data, key_replica_ts,$
 $\quad key_replica_last_read_ts, key_replica_si,$
 $\quad client_txn_count, client_op_count \rangle$

$|$
 $Reset(c) \triangleq$
 $\wedge \vee client_state[c] = \text{"committed"}$
 $\vee client_state[c] = \text{"aborted"}$ aborted
 $\wedge client_txn_count[c] < MAX_TXN_PER_CLIENT$ |
 $\wedge client_state' = [client_state \text{ EXCEPT } ![c] = \text{"init"}]$
 $\wedge client_ts' = [client_ts \text{ EXCEPT } ![c] = [start_ts \mapsto 0, commit_ts \mapsto 0]]$
 $\wedge client_op_count' = [client_op_count \text{ EXCEPT } ![c] = 0]$
 $\wedge client_read_set' = [client_read_set \text{ EXCEPT } ![c] = \{\}]$
 $\wedge client_write_set' = [client_write_set \text{ EXCEPT } ![c] = \{\}]$
 $\wedge client_txn_count' = [client_txn_count \text{ EXCEPT } ![c] = @ + 1]$ |
 $\wedge$ UNCHANGED $\langle next_ts, key_replica_data, key_replica_lock, key_replica_ts,$
 $\quad key_replica_last_read_ts, key_replica_si \rangle$

$Termination \triangleq$
 $\forall c \in CLIENT : client_txn_count[c] \geq MAX_TXN_PER_CLIENT$
 $\wedge UNCHANGED\ vars$

$Next$
 $Next \triangleq$
 $\exists c \in CLIENT :$
 $\vee Start(c)$
 $\vee \exists k \in KEY : Read(c, k)$
 $\vee \exists k \in KEY : Write(c, k)$
 $\vee Commit(c)$
 $\vee Abort(c)$
 $\vee Reset(c)$
 $\vee Termination$

$Spec \triangleq Init \wedge \Box [Next]_{vars}$

$TypeInvariant \triangleq$
 $\wedge next_ts \in Nat$
 $\wedge client_state \in [CLIENT \rightarrow States]$
 $\wedge client_ts \in [CLIENT \rightarrow [start_ts : Nat, commit_ts : Nat]]$
 $\wedge client_read_set \in [CLIENT \rightarrow SUBSET KEY]$
 $\wedge client_write_set \in [CLIENT \rightarrow SUBSET KEY]$

$WriteConsistency \triangleq$
 $\forall k \in KEY :$
 $\forall r \in 1 \dots REPLICA_COUNT :$
 $\forall n \in 1 \dots (Len(key_replica_data[k][r]) - 1) :$
 $\wedge key_replica_data[k][r][n].ts < key_replica_data[k][r][n+1].ts$

$CommittedConsistency \triangleq$
 $\forall c \in CLIENT :$
 $client_state[c] = \text{"committed"} \Rightarrow$
 $\forall k \in client_write_set[c] :$
 $\wedge HasQuorum(k, LAMBDA r :$
 $\quad \exists n \in DOMAIN\ key_replica_data[k][r] :$
 $\quad \wedge key_replica_data[k][r][n].ts = client_ts[c].commit_ts$
 $\quad \wedge key_replica_lock[k][r] \neq c)$
 $\wedge HasQuorum(k, LAMBDA r :$
 $\quad \exists n \in DOMAIN\ key_replica_data[k][r] :$
 $\quad \wedge key_replica_data[k][r][n].ts = client_ts[c].commit_ts)$

$$\begin{aligned}
\textit{AbortedConsistency} &\triangleq \\
&\forall c \in \textit{CLIENT} : \\
&\quad (\wedge \textit{client_state}[c] = \text{"aborted"} \\
&\quad \wedge \textit{client_ts}[c].\textit{commit_ts} \neq 0 \\
&\quad) \Rightarrow \\
&\quad \forall k \in \textit{client_write_set}[c] : \\
&\quad \quad \forall r \in 1 \dots \textit{REPLICA_COUNT} : \\
&\quad \quad \quad \neg \exists n \in \text{DOMAIN } \textit{key_replica_data}[k][r] : \\
&\quad \quad \quad \textit{key_replica_data}[k][r][n].\textit{ts} = \textit{client_ts}[c].\textit{commit_ts} \\
\\
\textit{SnapshotIsolation} &\triangleq \\
&\forall c1, c2 \in \textit{CLIENT} : \\
&\quad (\wedge \textit{client_state}[c1] = \text{"committed"} \\
&\quad \wedge \textit{client_state}[c2] = \text{"committed"} \\
&\quad \wedge c1 \neq c2 \\
&\quad) \Rightarrow \\
&\quad \vee \textit{client_write_set}[c1] \cap \textit{client_write_set}[c2] = \{\} \\
&\quad \vee \textit{client_ts}[c1].\textit{commit_ts} < \textit{client_ts}[c2].\textit{start_ts} \\
&\quad \vee \textit{client_ts}[c2].\textit{commit_ts} < \textit{client_ts}[c1].\textit{start_ts} \\
\\
\textit{SnapshotReadInvariant} &\triangleq \\
&\forall k \in \textit{KEY} : \\
&\quad \forall r \in 1 \dots \textit{REPLICA_COUNT} : \\
&\quad \quad \textit{key_replica_si}[k][r] = \text{TRUE}
\end{aligned}$$

THEOREM $\textit{Safety} \triangleq \textit{Spec} \Rightarrow \Box(\textit{TypeInvariant} \wedge$
 $\textit{SnapshotIsolation} \wedge$
 $\textit{SnapshotReadInvariant} \wedge$
 $\textit{WriteConsistency} \wedge$
 $\textit{CommittedConsistency} \wedge$
 $\textit{AbortedConsistency})$

MODULE *RedTSER*

EXTENDS *Integers, FiniteSets, Sequences, TLC*

CONSTANTS *KEY*,
 CLIENT

CONSTANTS *REPLICA_COUNT*,
 QUORUM_SIZE, ($3/4 * \text{REPLICA_COUNT}$)
 MAX_TXN_PER_CLIENT,
 MAX_OPS_PER_TXN

ASSUME $\wedge \text{REPLICA_COUNT} > 0$
 $\wedge \text{QUORUM_SIZE} \leq \text{REPLICA_COUNT}$
 $\wedge \text{QUORUM_SIZE} > \text{REPLICA_COUNT} \div 2$

VARIABLES *next_ts*,
 client_state,
 client_ts,
 client_read_set,
 client_write_set,
 key_replica_data,
 key_replica_lock,
 key_replica_ts,
 client_txn_count,
 client_op_count

vars $\triangleq \langle \text{next_ts}, \text{client_state}, \text{client_ts}, \text{client_read_set},$
 client_write_set, *key_replica_data*, *key_replica_lock*,
 key_replica_ts, *client_txn_count*, *client_op_count* $\rangle$

States $\triangleq \{ \text{"init"}, \text{"active"}, \text{"failed"}, \text{"committed"}, \text{"aborted"} \}$

CommittedTxns $\triangleq \{ c \in \text{CLIENT} : \text{client_state}[c] = \text{"committed"} \}$

Init $\triangleq$
 $\wedge \text{next_ts} = 0$
 $\wedge \text{client_state} = [c \in \text{CLIENT} \mapsto \text{"init"}]$
 $\wedge \text{client_ts} = [c \in \text{CLIENT} \mapsto [\text{start_ts} \mapsto 0, \text{commit_ts} \mapsto 0]]$
 $\wedge \text{client_read_set} = [c \in \text{CLIENT} \mapsto \{\}]$
 $\wedge \text{client_write_set} = [c \in \text{CLIENT} \mapsto \{\}]$
 $\wedge \text{key_replica_data} = [k \in \text{KEY} \mapsto$
 $[r \in 1 \dots \text{REPLICA_COUNT} \mapsto \langle [\text{ts} \mapsto 0, \text{val} \mapsto \text{"initial"}] \rangle]]$
 $\wedge \text{key_replica_lock} = [k \in \text{KEY} \mapsto [r \in 1 \dots \text{REPLICA_COUNT} \mapsto \{\}]]$
 $\wedge \text{key_replica_ts} = [k \in \text{KEY} \mapsto [r \in 1 \dots \text{REPLICA_COUNT} \mapsto 0]]$
 $\wedge \text{client_txn_count} = [c \in \text{CLIENT} \mapsto 0]$

$$\begin{aligned}
& \wedge \text{client_op_count} = [c \in \text{CLIENT} \mapsto 0] \\
& \text{HasQuorum}(k, \text{pred}(-)) \triangleq \\
& \quad \text{LET } \text{replicas} \triangleq \{r \in 1 \dots \text{REPLICA_COUNT} : \text{pred}(r)\} \\
& \quad \text{IN } \text{Cardinality}(\text{replicas}) \geq \text{QUORUM_SIZE} \\
& \text{CanAcquireReadLock}(k, r, c) \triangleq \\
& \quad \forall \text{lock} \in \text{key_replica_lock}[k][r] : \\
& \quad \quad \vee \text{lock.client} = c \\
& \quad \quad \vee \text{lock.mode} = \text{"read"} \\
& \text{CanAcquireWriteLock}(k, r, c) \triangleq \\
& \quad \forall \text{lock} \in \text{key_replica_lock}[k][r] : \\
& \quad \quad \text{lock.client} = c \\
& \text{Depends}(t1, t2) \triangleq \\
& \quad \wedge t1 \neq t2 \\
& \quad \wedge \text{client_state}[t1] = \text{"committed"} \\
& \quad \wedge \text{client_state}[t2] = \text{"committed"} \\
& \quad \wedge \vee \exists k1 \in \text{KEY} : \\
& \quad \quad \wedge k1 \in \text{client_write_set}[t1] \\
& \quad \quad \wedge k1 \in \text{client_write_set}[t2] \\
& \quad \vee \exists k2 \in \text{KEY} : \\
& \quad \quad \wedge k2 \in \text{client_read_set}[t1] \\
& \quad \quad \wedge k2 \in \text{client_write_set}[t2] \\
& \quad \vee \exists k3 \in \text{KEY} : \\
& \quad \quad \wedge k3 \in \text{client_write_set}[t1] \\
& \quad \quad \wedge k3 \in \text{client_read_set}[t2] \\
& \text{RECURSIVE } \text{IsAcyclic}(-, -, -, -) \\
& \text{IsAcyclic}(G, \text{visited}, \text{current}, \text{path}) \triangleq \\
& \quad \vee \text{path} = \{\} \\
& \quad \vee \forall \text{next} \in \{t \in \text{CommittedTxns} : \langle \text{current}, t \rangle \in G\} : \\
& \quad \quad \wedge \text{next} \notin \text{visited} \\
& \quad \quad \wedge \text{IsAcyclic}(G, \text{visited} \cup \{\text{next}\}, \text{next}, \text{path} \setminus \{\text{current}\}) \\
& \text{Start}(c) \triangleq \\
& \quad \wedge \text{client_state}[c] = \text{"init"} \\
& \quad \wedge \text{next_ts}' = \text{next_ts} + 1 \\
& \quad \wedge \text{client_state}' = [\text{client_state} \text{ EXCEPT } ![c] = \text{"active"}] \\
& \quad \wedge \text{client_ts}' = [\text{client_ts} \text{ EXCEPT } ![c].\text{start_ts} = \text{next_ts}'] \\
& \quad \wedge \text{client_op_count}' = [\text{client_op_count} \text{ EXCEPT } ![c] = 0] \\
& \quad \wedge \text{UNCHANGED } \langle \text{client_read_set}, \text{client_write_set},
\end{aligned}$$

$key_replica_data, key_replica_ts,$
 $key_replica_lock, client_txn_count\}$

$Read(c, k) \triangleq$
 $\wedge client_state[c] = \text{"active"}$
 $\wedge client_op_count[c] < MAX_OPS_PER_TXN$
 $\wedge k \notin client_read_set[c]$
 $\wedge \vee \wedge HasQuorum(k, \text{LAMBDA } r : CanAcquireReadLock(k, r, c))$
 $\wedge \exists data \in \text{DOMAIN } key_replica_data[k][1] :$
 $\wedge key_replica_data[k][1][data].ts \leq client_ts[c].start_ts$
 $\wedge \neg \exists newer \in \text{DOMAIN } key_replica_data[k][1] :$
 $\wedge key_replica_data[k][1][newer].ts \leq client_ts[c].start_ts$
 $\wedge key_replica_data[k][1][newer].ts > key_replica_data[k][1][data].ts$
 $\wedge HasQuorum(k, \text{LAMBDA } r :$
 $\quad \exists d \in \text{DOMAIN } key_replica_data[k][r] :$
 $\quad \quad key_replica_data[k][r][d].ts = key_replica_data[k][1][data].ts)$
 $\wedge client_read_set' = [client_read_set \text{ EXCEPT } ![c] = @ \cup \{k\}]$
 $\wedge key_replica_lock' = [key_replica_lock \text{ EXCEPT } ![k] =$
 $\quad [r \in 1 \dots REPLICATION_COUNT \mapsto$
 $\quad \text{IF } r \leq QUORUM_SIZE$
 $\quad \quad \text{THEN } @[r] \cup \{[client \mapsto c, mode \mapsto \text{"read"}]\}$
 $\quad \quad \text{ELSE } @[r]]]$
 $\wedge \text{UNCHANGED } \langle next_ts, client_state, client_ts, client_write_set,$
 $\quad key_replica_data, key_replica_ts, client_txn_count \rangle$
 $\vee \wedge client_state' = [client_state \text{ EXCEPT } ![c] = \text{"failed"}]$
 $\wedge \text{UNCHANGED } \langle next_ts, client_ts, client_read_set, client_write_set,$
 $\quad key_replica_data, key_replica_lock,$
 $\quad key_replica_ts, client_txn_count \rangle$
 $\wedge client_op_count' = [client_op_count \text{ EXCEPT } ![c] = @ + 1]$

$Write(c, k) \triangleq$
 $\wedge client_state[c] = \text{"active"}$
 $\wedge client_op_count[c] < MAX_OPS_PER_TXN$
 $\wedge k \notin client_write_set[c]$
 $\wedge \vee \wedge HasQuorum(k, \text{LAMBDA } r : CanAcquireWriteLock(k, r, c))$
 $\wedge key_replica_lock' = [key_replica_lock \text{ EXCEPT } ![k] =$
 $\quad [r \in 1 \dots REPLICATION_COUNT \mapsto$
 $\quad \text{IF } r \leq QUORUM_SIZE$
 $\quad \quad \text{THEN } \{[client \mapsto c, mode \mapsto \text{"write"}]\}$
 $\quad \quad \text{ELSE } @[r]]]$
 $\wedge client_write_set' = [client_write_set \text{ EXCEPT } ![c] = @ \cup \{k\}]$
 $\wedge \text{UNCHANGED } \langle next_ts, client_state, client_ts, client_read_set,$
 $\quad key_replica_data, key_replica_ts, client_txn_count \rangle$

$$\begin{aligned}
& \vee \wedge \text{client_state}' = [\text{client_state} \text{ EXCEPT } ![c] = \text{"failed"}] \\
& \wedge \text{UNCHANGED } \langle \text{next_ts}, \text{client_ts}, \text{client_read_set}, \text{client_write_set}, \\
& \quad \text{key_replica_data}, \text{key_replica_lock}, \\
& \quad \text{key_replica_ts}, \text{client_txn_count} \rangle \\
& \wedge \text{client_op_count}' = [\text{client_op_count} \text{ EXCEPT } ![c] = @ + 1]
\end{aligned}$$

$$\begin{aligned}
& \text{Commit}(c) \triangleq \\
& \wedge \text{client_state}[c] = \text{"active"} \\
& \wedge \forall k \in \text{client_write_set}[c] : \\
& \quad \text{HasQuorum}(k, \text{LAMBDA } r : \\
& \quad \quad \exists \text{lock} \in \text{key_replica_lock}[k][r] : \\
& \quad \quad \wedge \text{lock.client} = c \\
& \quad \quad \wedge \text{lock.mode} = \text{"write"}) \\
& \wedge \forall k \in \text{client_read_set}[c] : \\
& \quad \text{HasQuorum}(k, \text{LAMBDA } r : \\
& \quad \quad \exists \text{lock} \in \text{key_replica_lock}[k][r] : \\
& \quad \quad \wedge \text{lock.client} = c \\
& \quad \quad \wedge \text{lock.mode} = \text{"read"}) \\
& \wedge \text{next_ts}' = \text{next_ts} + 1 \\
& \wedge \text{client_state}' = [\text{client_state} \text{ EXCEPT } ![c] = \text{"committed"}] \\
& \wedge \text{client_ts}' = [\text{client_ts} \text{ EXCEPT } ![c].\text{commit_ts} = \text{next_ts}'] \\
& \wedge \text{IF } \text{client_write_set}[c] = \{\} \\
& \quad \text{THEN } \wedge \text{UNCHANGED } \text{key_replica_data} \\
& \quad \wedge \text{UNCHANGED } \text{key_replica_ts} \\
& \quad \wedge \text{UNCHANGED } \text{key_replica_lock} \\
& \quad \text{ELSE } \wedge \forall k \in \text{client_write_set}[c] : \\
& \quad \quad \wedge \text{key_replica_data}' = [\text{key_replica_data} \text{ EXCEPT } ![k] = \\
& \quad \quad \quad [r \in 1 \dots \text{REPLICA_COUNT} \mapsto \\
& \quad \quad \quad \text{IF } r \leq \text{QUORUM_SIZE} \\
& \quad \quad \quad \text{THEN } \text{Append}(@[r], [ts \mapsto \text{next_ts}', \text{val} \mapsto \text{"written"}]) \\
& \quad \quad \quad \text{ELSE } @[r]]] \\
& \quad \quad \wedge \text{key_replica_ts}' = [\text{key_replica_ts} \text{ EXCEPT } ![k] = \\
& \quad \quad \quad [r \in 1 \dots \text{REPLICA_COUNT} \mapsto \\
& \quad \quad \quad \text{IF } r \leq \text{QUORUM_SIZE} \text{ THEN } \text{next_ts}' \text{ ELSE } @[r]]] \\
& \quad \quad \wedge \text{key_replica_lock}' = [\text{key_replica_lock} \text{ EXCEPT } ![k] = \\
& \quad \quad \quad [r \in 1 \dots \text{REPLICA_COUNT} \mapsto \{\}]] \\
& \wedge \text{UNCHANGED } \langle \text{client_read_set}, \text{client_write_set}, \\
& \quad \text{client_txn_count}, \text{client_op_count} \rangle
\end{aligned}$$

$$\begin{aligned}
& \text{Abort}(c) \triangleq \\
& \wedge \vee \text{client_state}[c] = \text{"active"} \\
& \quad \vee \text{client_state}[c] = \text{"failed"} \\
& \wedge \text{client_state}' = [\text{client_state} \text{ EXCEPT } ![c] = \text{"aborted"}]
\end{aligned}$$

$$\begin{aligned}
& \wedge \text{key_replica_lock}' = [k \in \text{KEY} \mapsto \mid \\
& \quad [r \in 1 \dots \text{REPLICA_COUNT} \mapsto \\
& \quad \quad \text{IF } c \in \text{key_replica_lock}[k][r] \text{ THEN } \{\} \text{ ELSE } \text{key_replica_lock}[k][r]] \\
& \wedge \text{UNCHANGED } \langle \text{next_ts}, \text{client_ts}, \text{client_read_set}, \text{client_write_set}, \\
& \quad \text{key_replica_data}, \text{key_replica_ts}, \\
& \quad \text{client_txn_count}, \text{client_op_count} \rangle \\
& \mid \\
\text{Reset}(c) & \triangleq \\
& \wedge \vee \text{client_state}[c] = \text{"committed"} \\
& \quad \vee \text{client_state}[c] = \text{"aborted"} \quad \text{aborted} \\
& \wedge \text{client_txn_count}[c] < \text{MAX_TXN_PER_CLIENT} \quad \mid \\
& \wedge \text{client_state}' = [\text{client_state} \text{ EXCEPT } ![c] = \text{"init"}] \\
& \wedge \text{client_ts}' = [\text{client_ts} \text{ EXCEPT } ![c] = [\text{start_ts} \mapsto 0, \text{commit_ts} \mapsto 0]] \\
& \wedge \text{client_op_count}' = [\text{client_op_count} \text{ EXCEPT } ![c] = 0] \\
& \wedge \text{client_read_set}' = [\text{client_read_set} \text{ EXCEPT } ![c] = \{\}] \\
& \wedge \text{client_write_set}' = [\text{client_write_set} \text{ EXCEPT } ![c] = \{\}] \\
& \wedge \text{client_txn_count}' = [\text{client_txn_count} \text{ EXCEPT } ![c] = @ + 1] \quad \mid \\
& \wedge \text{UNCHANGED } \langle \text{next_ts}, \text{key_replica_data}, \text{key_replica_lock}, \text{key_replica_ts} \rangle \\
& \mid \\
\text{Termination} & \triangleq \\
& \vee c \in \text{CLIENT} : \text{client_txn_count}[c] \geq \text{MAX_TXN_PER_CLIENT} \\
& \wedge \text{UNCHANGED } \text{vars} \\
& \mid \\
\text{Termination} & \triangleq \\
& \forall c \in \text{CLIENT} : \\
& \quad \vee \text{client_state}[c] = \text{"committed"} \\
& \quad \vee \text{client_state}[c] = \text{"aborted"} \\
& \wedge \text{UNCHANGED } \text{vars} \\
& \mid \\
\text{Next} & \triangleq \\
\text{Next} & \triangleq \\
& \exists c \in \text{CLIENT} : \\
& \quad \vee \text{Start}(c) \\
& \quad \vee \exists k \in \text{KEY} : \text{Read}(c, k) \\
& \quad \vee \exists k \in \text{KEY} : \text{Write}(c, k) \\
& \quad \vee \text{Commit}(c) \\
& \quad \vee \text{Abort}(c) \\
& \quad \vee \text{Reset}(c) \quad \mid \\
& \vee \text{Termination} \\
& \mid \\
\text{Spec} & \triangleq \text{Init} \wedge \Box[\text{Next}]_{\text{vars}} \\
& \mid \\
\text{TypeInvariant} & \triangleq
\end{aligned}$$

$$\begin{aligned}
& \wedge next_ts \in Nat \\
& \wedge client_state \in [CLIENT \rightarrow States] \\
& \wedge client_ts \in [CLIENT \rightarrow [start_ts : Nat, commit_ts : Nat]] \\
& \wedge client_read_set \in [CLIENT \rightarrow SUBSET KEY] \\
& \wedge client_write_set \in [CLIENT \rightarrow SUBSET KEY]
\end{aligned}$$

WriteConsistency $\triangleq$

$$\begin{aligned}
& \forall k \in KEY : \\
& \quad \forall r \in 1 \dots REPLICA_COUNT : \\
& \quad \quad \forall n \in 1 \dots (Len(key_replica_data[k][r]) - 1) : \\
& \quad \quad \quad \wedge key_replica_data[k][r][n].ts < key_replica_data[k][r][n+1].ts
\end{aligned}$$

CommittedConsistency $\triangleq$

$$\begin{aligned}
& \forall c \in CLIENT : \\
& \quad client_state[c] = \text{"committed"} \Rightarrow \\
& \quad \quad \forall k \in client_write_set[c] : \\
& \quad \quad \quad \wedge HasQuorum(k, LAMBDA r : \\
& \quad \quad \quad \quad \exists n \in DOMAIN key_replica_data[k][r] : \\
& \quad \quad \quad \quad \quad \wedge key_replica_data[k][r][n].ts = client_ts[c].commit_ts \\
& \quad \quad \quad \quad \quad \wedge key_replica_lock[k][r] \neq c) \\
& \quad \quad \quad \wedge HasQuorum(k, LAMBDA r : \\
& \quad \quad \quad \quad \exists n \in DOMAIN key_replica_data[k][r] : \\
& \quad \quad \quad \quad \quad \wedge key_replica_data[k][r][n].ts = client_ts[c].commit_ts)
\end{aligned}$$

AbortedConsistency $\triangleq$

$$\begin{aligned}
& \forall c \in CLIENT : \\
& \quad (\wedge client_state[c] = \text{"aborted"} \\
& \quad \quad \wedge client_ts[c].commit_ts \neq 0 \\
& \quad) \Rightarrow \\
& \quad \quad \forall k \in client_write_set[c] : \\
& \quad \quad \quad \forall r \in 1 \dots REPLICA_COUNT : \\
& \quad \quad \quad \quad \neg \exists n \in DOMAIN key_replica_data[k][r] : \\
& \quad \quad \quad \quad \quad key_replica_data[k][r][n].ts = client_ts[c].commit_ts
\end{aligned}$$

Serializability $\triangleq$

LET

$$\begin{aligned}
Edges \triangleq \{ & t \in CommittedTxns \times CommittedTxns : \\
& \quad \wedge t[1] \neq t[2] \\
& \quad \wedge Depends(t[1], t[2]) \\
& \quad \wedge client_ts[t[1]].commit_ts < client_ts[t[2]].commit_ts \}
\end{aligned}$$

IN

$\forall start \in CommittedTxns :$
 $IsAcyclic(Edges, \{start\}, start, CommittedTxns)$

THEOREM $Safety \triangleq Spec \Rightarrow \Box (TypeInvariant \wedge WriteConsistency \wedge$
 $CommittedConsistency \wedge$
 $AbortedConsistency \wedge Serializability)$